



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

Ingeniería de Software

Ingeniería de Sistemas de Información

Ingeniería de Ciberseguridad

Ciencias de la Computación

Período: 2026-10

Curso: 1ASI0400 | Diseño de Base de datos

NRC: 6008

Docente: Elizabeth Del Pilar Sáenz Musayón

INFORME DE TRABAJO FINAL

Nombre del startup: ControlBet

Nombre del producto: BetCore DB: Modelo de gestión y monitoreo de datos para casas de apuestas deportivas

Relación de integrantes

Código	Apellidos y nombres
U202423813	Macalupu Marchan, Yeissen Beckam
U20241F697	Chavez Pino, Paula Brenda
U202424466	Gallegos De La Cruz, Giovanni Marcelo
U20241E550	Huerta Cardenas, Brayan Benjamin
U202423883	Rivera Rupay, Fabricio Jose

INDICE

REGISTRO DE VERSIONES DEL INFORME.....	4
STUDENT OUTCOME.....	10
CAPÍTULO I: INTRODUCCIÓN	15
1.1. Startup Profile	15
1.1.1. Descripción de la Startup	15
1.1.2. Perfiles de integrantes del equipo	15
1.2. Solution Profile	17
1.2.1. Antecedentes y problemática	18
1.2.2. Propuesta de valor.....	20
1.3. Segmentos objetivo	22
CAPÍTULO II: RECOPIACIÓN Y ANÁLISIS DE REQUISITOS	23
2.1. Entrevistas.....	23
2.1.1. Diseño de entrevistas	23
2.1.2. Registro de entrevistas	26
2.1.3. Análisis de entrevistas.....	31
2.2. Requisitos.....	34
CAPÍTULO III: DISEÑO DE BASE DE DATOS	40
3.1. Entidades.....	40
3.2. Atributos	43
3.3. Enfoque relacional	51
3.3.1. Diagrama entidad-relación conceptual.....	51
3.3.2. Diagrama entidad-relación lógico	52
3.4. Enfoque no relacional	52
3.4.1. Colecciones	52
3.4.2. Validación de esquema.....	54
3.4.3. Patrones de modelo de datos	64
CAPÍTULO IV: IMPLEMENTACIÓN DE BASE DE DATOS	68
4.1. Sistema de gestión de base de datos.....	68
4.1.1. Evaluación y elección del sistema de gestión de base de datos relacional.....	68
4.2.1. Evaluación y elección del sistema de gestión de base de datos no relacional.....	70
4.2. Diagramas de datos	75
4.2.1. Diagrama entidad-relación físico	75
4.2.2. Diagrama de documentos.....	75
4.3. Scripts de creación de la base de datos	76

4.3.1. Scripts de creación y carga de datos de la base de datos relacional	76
4.3.1.1. Creación de tablas	76
4.3.1.2. Creación de datos	84
4.3.2. Scripts de creación y carga de datos de la base de datos no relacional	91
4.3.2.1. Creación de base de datos y colecciones.....	91
4.3.2.2. Inserción de datos para las colecciones.....	99
4.4. Consultas.....	126
4.4.1. Consultas para la base de datos relacional	126
4.4.1.1. Consultas y Subqueries	126
4.4.1.2. Procedimientos almacenados	134
4.4.1.3. Funciones	144
4.4.1.4. Triggers	153
4.4.1.4. Índices	165
4.4.1.5. Estadísticas.....	172
4.4.2. Consultas para la base de datos no relacional	177
CONCLUSIONES	185
Conclusiones y recomendaciones	185
Video About-the-Team.....	188
Exposición del Equipo	189
Bibliografía	190
Anexos	191
Anexo A: Evidencia de entrevistas realizadas	191
Anexo B: Diagrama entidad-relación lógico	195
Anexo C: Diagrama entidad-relación lógico	196
Anexo D: Diagrama entidad-relación físico	196
Anexo E: Diagrama entidad-relación físico.....	197
Anexo F: Enlaces de video de exposición	197
Anexo G: Enlaces a evidencias y artefactos del proyecto.....	198

REGISTRO DE VERSIONES DEL INFORME

Versión	Fecha	Autor	Descripción
1.1	12/04/2026	Macalupu Marchan, Yeissen Beckam	Redacción de la sección 1.1.1 Descripción del startup.
1.2	14/04/2026	Gallegos De La Cruz, Giovanni Marcelo	Desarrollo de la sección 1.2.1 Antecedentes y problemática.
1.3	15/04/2026	Chavez Pino, Paula Brenda	Incorporación de sustento estadístico del sector de apuestas deportivas con fuente oficial.
1.4	15/04/2026	Huerta Cardenas, Brayan Benjamin	Corrección de la problemática mediante la técnica 5W + 2H en formato por puntos.
1.5	17/04/2026	Macalupu Marchan, Yeissen Beckam	Redacción de la sección 1.2.2 Propuesta de valor.
1.6	17/04/2026	Rivera Rupay, Fabricio Jose	Definición y ajuste de la sección 1.3 Segmentos objetivo.
1.7	17/04/2026	Huerta Cardenas, Brayan Benjamin	Diseño de entrevistas para los segmentos objetivo del Capítulo II.
1.8	18/04/2026	Gallegos De La Cruz, Giovanni Marcelo	Revisión general de coherencia y mejora de redacción del Capítulo I y 2.1.1.
1.9	19/04/2026	Chavez Pino, Paula Brenda	Registro y resumen de entrevistas realizadas a representantes de los segmentos objetivo.

2.0	19/04/2026	Rivera Rupay, Fabricio Jose	Desarrollo del análisis de entrevistas por segmento objetivo.
2.1	20/04/2026	Huerta Cardenas, Brayan Benjamin	Elaboración de la sección 2.2 Requisitos mediante User Stories y requisitos funcionales.
2.2	21/04/2026	Macalupu Marchan, Yeissen Beckam	Identificación y descripción de las entidades principales del modelo de datos BetCore DB.
2.3	27/04/2026	Gallegos De La Cruz, Giovanni Marcelo	Desarrollo de la sección 3.2 Atributos con descripciones de los principales campos del modelo.
2.4	5/05/2026	Chavez Pino, Paula Brenda	Incorporación del enfoque relacional y descripción del diagrama entidad-relación lógico.
2.5	6/05/2026	Rivera Rupay, Fabricio Jose	Evaluación comparativa de MySQL, PostgreSQL y SQL Server para la selección del DBMS relacional.
2.6	9/05/2026	Rivera Rupay, Fabricio Jose	Justificación de la elección de SQL Server como sistema de gestión de base de datos relacional.
2.7	10/05/2026	Macalupu Marchan, Yeissen Beckam	Revisión final de coherencia, nomenclatura técnica y alineación del informe con el enfoque de diseño de base de datos.
2.8	12/05/2026	Huerta Cardenas,	Desarrollo de los scripts de creación de la base de datos

		Brayan Benjamin	relacional en SQL Server, incluyendo tablas, claves primarias, claves foráneas y restricciones.
2.9	14/05/2026	Gallegos De La Cruz, Giovanni Marcelo	Elaboración de los scripts de carga de datos para la base de datos relacional, considerando registros de prueba para las entidades principales del modelo.
3.0	16/05/2026	Macalupu Marchan, Yeissen Beckam	Desarrollo de consultas SQL orientadas al análisis operativo y financiero de BetCore DB, incluyendo filtros, agrupaciones, subconsultas y consultas de apoyo a la toma de decisiones.
3.1	18/05/2026	Chavez Pino, Paula Brenda	Implementación y documentación de procedimientos almacenados y funciones para automatizar operaciones y consultas frecuentes dentro de la base de datos relacional.
3.2	20/05/2026	Rivera Rupay, Fabricio Jose	Desarrollo de triggers para reforzar el control de operaciones, validaciones automáticas y trazabilidad dentro de la base de datos relacional.
3.3	22/05/2026	Gallegos De La Cruz, Giovanni Marcelo	Elaboración de índices y estadísticas en SQL Server para mejorar el rendimiento de

			las consultas y optimizar el acceso a los datos del modelo relacional.
3.4	25/05/2026	Macalupu Marchan, Yeissen Beckam	Incorporación de la sección de base de datos no relacional, incluyendo la identificación de colecciones, estructura documental y justificación del uso de MongoDB.
3.5	27/05/2026	Rivera Rupay, Fabricio Jose	Diseño del modelo documental en Hackolade, considerando las colecciones apuestas y movimientos_financieros, sus campos, documentos embebidos y referencias lógicas.
3.6	29/05/2026	Chavez Pino, Paula Brenda	Desarrollo de la sección de validación de esquema para la base de datos no relacional, incluyendo reglas simples para campos requeridos, tipos de datos, enumeraciones y formatos de fecha.
3.7	31/05/2026	Huerta Cardenas, Brayan Benjamin	Evaluación y elección del sistema de gestión de base de datos no relacional, justificando el uso de MongoDB como alternativa documental para BetCore DB.
3.8	02/06/2026	Gallegos De La Cruz, Giovanni Marcelo	Elaboración del diagrama de documentos de la base de datos no relacional y redacción de la explicación de las

			colecciones, arreglos y relaciones lógicas.
3.9	01/07/2026	Rivera Rupay, Fabricio Jose	Desarrollo de los scripts de creación y carga de datos en MongoDB, incluyendo la creación lógica de la base de datos, colecciones e inserciones con insertMany.
4.0	01/07/2026	Macalupu Marchan, Yeissen Beckam	Desarrollo de consultas para la base de datos no relacional en MongoDB, considerando consultas sobre apuestas, usuarios, locales, montos y movimientos financieros.
4.1	01/07/2026	Chavez Pino, Paula Brenda	Revisión de evidencias de implementación en MongoDB desde Visual Studio Code, incluyendo ejecución de insertOne, validación de documentos y salida de resultados.
4.2	05/07/2026	Huerta Cardenas, Brayan Benjamin	Redacción de conclusiones y recomendaciones generales del proyecto, considerando la integración entre el modelo relacional en SQL Server y el modelo documental en MongoDB.
4.3	05/07/2026	Macalupu Marchan, Yeissen Beckam	Actualización del README del repositorio en GitHub, incorporando la documentación general del proyecto, estructura del

			repositorio, tecnologías, SQL Server, MongoDB y evidencias.
4.4	08/07/2026	Todos los integrantes	Revisión final del informe, corrección de redacción, verificación de coherencia entre secciones, actualización de evidencias y preparación de la entrega final.

STUDENT OUTCOME

El desarrollo del proyecto BetCore DB: Modelo de gestión y monitoreo de datos para casas de apuestas deportivas permite evidenciar la integración de competencias propias de Ingeniería de Software y Ciencias de la Computación. Desde la perspectiva de Ingeniería de Software, el equipo aplicó estrategias de aprendizaje para comprender el dominio del problema, levantar requisitos, analizar necesidades de los usuarios y justificar decisiones de diseño. Desde la perspectiva de Ciencias de la Computación, el proyecto permitió aplicar fundamentos de modelado de datos, diseño relacional, normalización, integridad referencial y estructuración lógica de información para producir una solución basada en computación.

En ese sentido, el trabajo realizado contribuye al logro del ABET - EAC - Student Outcome 7, relacionado con la capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, y al ABET - CAC - Student Outcome 6, relacionado con la capacidad de aplicar teoría de ciencias de la computación y fundamentos del desarrollo de software para producir soluciones basadas en computación.

Criterio específico	Acciones realizadas	Conclusiones
ABET - EAC - Student Outcome 7 Actualiza conceptos y conocimientos necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de ingeniería de software.	Macalupu Marchan, Yeissen Beckam – TB1: Participó en la definición del enfoque del proyecto, la descripción de la startup ControlBet y la orientación de BetCore DB como una solución de gestión de datos para casas de apuestas deportivas. Además, colaboró en la revisión de coherencia del documento para mantener el alcance centrado en el diseño de base de datos. Chavez Pino, Paula Brenda – TB1: Contribuyó en la incorporación de información	El equipo fortaleció su capacidad para adquirir nuevos conocimientos sobre el dominio de apuestas deportivas, gestión de información operativa y transaccional, levantamiento de requisitos y diseño de bases de datos. Asimismo, se evidenció la aplicación de técnicas de investigación, análisis del problema y documentación académica para justificar la propuesta de BetCore DB.

	de sustento sobre el crecimiento del sector de apuestas deportivas, reforzando la justificación del problema y la necesidad de una solución basada en datos. Gallegos De La Cruz, Giovanni Marcelo – TB1: Participó en el desarrollo de los antecedentes y problemática, aplicando la técnica 5W + 2H para analizar el contexto del problema y organizar la información de manera estructurada. Huerta Cardenas, Brayan Benjamin – TB1: Colaboró en el diseño de entrevistas dirigidas a los segmentos objetivo, considerando preguntas orientadas a identificar necesidades, procesos actuales y datos relevantes para el modelo de base de datos. Rivera Rupay, Fabricio Jose – TB1: Participó en la definición y ajuste de los segmentos objetivo, diferenciando a los actores	
--	--	--

	internos de las casas de apuestas y a los clientes apostadores como fuentes de información necesarias para el análisis de requisitos.	
ABET - EAC - Student Outcome 7 Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de ingeniería de software.	Macalupu Marchan, Yeissen Beckam – TB1: Identificó la necesidad de alinear el proyecto con el alcance del curso, evitando presentar BetCore DB como una plataforma final y enfocándolo como una arquitectura de base de datos. Chavez Pino, Paula Brenda – TB1: Reforzó el aprendizaje sobre búsqueda y uso de fuentes confiables para sustentar el contexto del problema, especialmente en relación con el crecimiento del sector de apuestas deportivas. Gallegos De La Cruz, Giovanni Marcelo – TB1: Aplicó criterios de análisis y organización de información para mejorar la claridad de la problemática y la conexión entre antecedentes, necesidad del negocio y solución propuesta.	El proyecto permitió reconocer que el diseño de una solución de datos requiere aprendizaje continuo, investigación del dominio, revisión de metodologías y adaptación de conocimientos técnicos. El equipo comprendió que una base de datos bien diseñada no solo depende de la creación de tablas, sino también de entender el problema, los usuarios, las reglas de negocio y la trazabilidad de la información.

	Huerta Cardenas, Brayan Benjamin – TB1: Desarrolló habilidades relacionadas con la formulación de preguntas de entrevista, identificación de información principal y complementaria, y conexión entre necesidades de usuarios y requisitos funcionales. Rivera Rupay, Fabricio Jose – TB1: Reconoció la importancia de identificar correctamente los perfiles involucrados en el dominio del problema, considerando tanto usuarios internos como clientes apostadores para enriquecer el análisis.	
ABET - CAC - Student Outcome 6 Aplica la teoría de ciencias de la computación y fundamentos del desarrollo de software para producir soluciones basadas en computación.	Macalupu Marchan, Yeissen Beckam – TB1: Participó en la identificación de entidades principales y en la organización conceptual del modelo de datos de BetCore DB, considerando actores, apuestas, transacciones y eventos deportivos. Chavez Pino, Paula Brenda – TB1: Contribuyó en el análisis de información obtenida del	El equipo aplicó fundamentos de ciencias de la computación y desarrollo de software al representar un dominio real mediante entidades, atributos, relaciones y requisitos funcionales. La propuesta de BetCore DB evidencia el uso de conceptos de modelado de datos, abstracción, organización de información e integridad relacional para

	contexto y entrevistas para relacionarla con posibles entidades, atributos y requisitos funcionales del modelo. Gallegos De La Cruz, Giovanni Marcelo – TB1: Apoyó en la estructuración lógica de la problemática y en la conexión entre necesidades del dominio, requisitos y decisiones preliminares del diseño de base de datos. Huerta Cardenas, Brayan Benjamin – TB1: Colaboró en la identificación de datos relevantes a partir de las entrevistas, permitiendo reconocer elementos como usuarios, apuestas, pagos, cuotas, comprobantes, saldos y transacciones financieras. Rivera Rupay, Fabricio Jose – TB1: Participó en la organización de segmentos y necesidades de información, aportando a la definición de relaciones entre actores del negocio y datos necesarios para la solución.	construir una solución basada en computación.
--	---	--

CAPÍTULO I: INTRODUCCIÓN

1.1. Startup Profile

1.1.1. Descripción de la Startup

ControlBet es una startup orientada al diseño de soluciones de gestión de datos para casas de apuestas deportivas, con énfasis en la organización, gestión y análisis inteligente de datos operativos y transaccionales, respondiendo ante la necesidad de contar con herramientas que permitan a las casas de apuestas administrar de manera eficiente grandes volúmenes de información relacionados con usuarios, eventos deportivos, apuestas realizadas, movimientos económicos, resultados y comportamiento de las operaciones en tiempo real.

La propuesta del startup plantea BetCore DB, un modelo de gestión y monitoreo de datos orientado a casas de apuestas deportivas. Esta solución se enfoca en el diseño de una arquitectura de base de datos capaz de organizar información operativa y transaccional, permitiendo mejorar la trazabilidad de apuestas, el control de movimientos financieros, el análisis de información para la toma de decisiones estratégicas y reducir riesgos asociados a pérdidas monetarias. De esta manera, el startup busca aportar valor mediante una solución basada en datos bien diseñada, sin enfocarse en el desarrollo de una plataforma final, sino en la estructuración eficiente de la información del negocio.

1.1.2. Perfiles de integrantes del equipo



Macalupu Marchan, Yeissen Beckam – U202423813.

Soy estudiante de la carrera de Ciencias de la Computación en la Universidad Peruana de Ciencias Aplicadas (UPC), me desempeño como desarrollador FullStack con en el mercado TI. Trabajando con el stack MERN, y como profesional puedo aportar algunos de mis conocimientos en este proyecto de casa de apuestas aplicando algunas metodologías o herramientas.

**Chavez Pino, Paula Brenda – U20241697**

Soy estudiante de la carrera de Ciencias de la Computación en la Universidad Peruana de Ciencias Aplicadas (UPC), actualmente cursando el cuarto ciclo. Cuento con conocimientos en programación, especialmente en los lenguajes C++ y Python, que he aplicado en proyectos académicos enfocados en la resolución de problemas y el desarrollo de soluciones eficientes.

Me destaco por mi capacidad para trabajar en equipo y asumir roles de liderazgo cuando es necesario, fomentando una comunicación clara y un ambiente colaborativo. Soy una persona responsable, proactiva y comprometida con mis objetivos, siempre buscando aprender nuevas herramientas y fortalecer mis habilidades técnicas y blandas.

Además, valoro el bienestar y la organización dentro del equipo, ya que considero que un buen entorno de trabajo es clave para lograr resultados de calidad. Estoy interesada en seguir desarrollándome en el área tecnológica y contribuir activamente en proyectos que generen impacto.

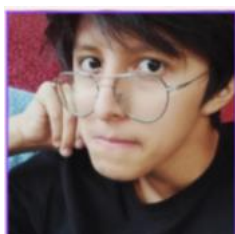
**Gallegos De La Cruz, Giovanni Marcelo – U202424466**

Estudiante de Ingeniería de Software con interés en el desarrollo backend, automatización de procesos, bases de datos y soluciones digitales. Cuenta con conocimientos en redes, simulación de sistemas y configuración de entornos de comunicación, utilizando herramientas como Cisco Packet Tracer para el diseño, prueba y monitoreo de flujos de datos. Además, posee experiencia inicial en n8n, APIs REST, PostgreSQL y modelos de lenguaje como ChatGPT y Gemini, aplicándolos en proyectos académicos y personales orientados a la automatización y mejora de procesos. Se caracteriza por su aprendizaje autodidacta, trabajo colaborativo y disposición para asumir nuevos desafíos tecnológicos.



Rivera Rupay, Fabricio Jose – U202423883.

Mi nombre es Fabricio Rivera, estudio en la carrera de Ingeniería de Software en la Universidad Peruana de Ciencias Aplicadas. Tengo conocimientos en lenguaje C++ y Python. Además, poseo conocimientos básicos en Internet de las Cosas (IoT), con experiencia en el uso de Cisco Packet Tracer para simulaciones de redes y en el manejo de bases de datos SQL utilizando Oracle. Me gusta trabajar en equipo y en mis tiempos libres me gusta jugar videojuegos, escuchar música y ver fútbol. Me considero responsable, solidario, un poco perfeccionista y ambicioso.



Huerta Cardenas, Brayan Benjamin – U20241E550

Soy Brayan Benjamin Huerta Cardenas, estudiante de Ingeniería de Software. Tengo conocimientos en lenguajes de programación y en la simulación de redes con máquinas virtuales en Cisco Packet Tarcer y tengo pasión por el aprendizaje continuo, especialmente en áreas vinculadas a la ciencia y la tecnología. Me considero una persona con aptitudes técnicas y una actitud positiva que me permite adaptarme y destacar en diversos entornos. Mi enfoque proactivo, especialmente en el trabajo colaborativo, y mi habilidad para resolver problema de manera eficiente.

1.2. Solution Profile

La presente sección describe el perfil de la solución propuesta por ControlBet, su propósito es contextualizar el problema identificado en el sector, explicar las principales dificultades relacionadas con la organización de información operativa y transaccional, y presentar la base conceptual que justifica el diseño de una arquitectura de base de datos relacional.

Asimismo, esta sección permite establecer la relación entre las necesidades del negocio, los usuarios involucrados y la propuesta de valor del proyecto, manteniendo como eje central la mejora en la gestión de datos, la trazabilidad de operaciones, el control financiero y el soporte para la toma de decisiones estratégicas.

1.2.1. Antecedentes y problemática

Antecedentes

En la actualidad, el sector de las apuestas deportivas ha experimentado un crecimiento sostenido como consecuencia del aumento del consumo digital, el acceso masivo a internet, la disponibilidad de eventos deportivos en tiempo real y la expansión de servicios vinculados al entretenimiento y las transacciones en línea. En el contexto peruano, este crecimiento también se evidencia mediante información oficial del Ministerio de Comercio Exterior y Turismo (Mincetur), según la cual la recaudación acumulada entre enero y noviembre de 2025 por impuestos vinculados a casinos, máquinas tragamonedas, juegos a distancia y apuestas deportivas alcanzó los S/ 419.5 millones.

Este escenario refleja la creciente necesidad de contar con estructuras de datos capaces de organizar información del negocio. Las casas de apuestas manejan diariamente datos relacionados con usuarios, eventos deportivos, cuotas, apuestas registradas, movimientos económicos, resultados y estados de pago. Por ello, la gestión adecuada de la información se convierte en un factor clave para mantener el control operativo, reducir errores y apoyar la toma de decisiones.

Problemática

A pesar del crecimiento del sector, muchas casas de apuestas pequeñas y medianas aún presentan dificultades para organizar y controlar sus datos de manera eficiente. En algunos casos, la información se encuentra dispersa en distintos medios, se registra de forma parcialmente manual o no cuenta con una estructura clara que permita relacionar adecuadamente apuestas, usuarios, pagos, eventos deportivos y resultados.

Esta situación genera problemas como duplicidad de registros, baja trazabilidad de operaciones, demoras en la validación de resultados, dificultad para consultar información histórica y mayor riesgo de inconsistencias financieras.

Como consecuencia, pueden generarse errores en el seguimiento de transacciones, inconsistencias en los registros, demoras en la validación de resultados y dificultades para identificar situaciones que podrían derivar en pérdidas monetarias. En un entorno altamente competitivo y dinámico, la capacidad de administrar datos en tiempo real representa una ventaja estratégica para la sostenibilidad del negocio.

- **¿Who? (¿Quiénes están involucrados?)**
Casas de apuestas deportivas pequeñas y medianas, encargados, asistentes, propietarios, personal financiero, analistas operativos y usuarios apostadores.
- **¿What? (¿Qué problema existe?)**
Existe una gestión deficiente de los datos operativos y transaccionales, lo que dificulta el control de apuestas, pagos, saldos, resultados deportivos y movimientos financieros.
- **¿Where? (¿Dónde ocurre?)**
Ocurre en el entorno operativo de las casas de apuestas deportivas, principalmente en los procesos de registro de apuestas, control financiero, validación de resultados, seguimiento de transacciones y consulta de información histórica.
- **¿When? (¿Cuándo ocurre?)**
Se presenta durante la operación diaria del negocio, especialmente en horarios de alta demanda, eventos deportivos importantes o momentos en los que aumenta el volumen de apuestas y transacciones.
- **¿Why? (¿Por qué ocurre?)**
Ocurre porque muchas organizaciones no cuentan con una arquitectura de datos estructurada que permita integrar, relacionar y consultar la información de manera ordenada, consistente y confiable.
- **¿How? (¿Cómo impacta?)**

Impacta generando errores de registro, duplicidad de datos, demoras en validaciones, dificultad para detectar inconsistencias, baja trazabilidad y menor capacidad para analizar el comportamiento operativo y financiero del negocio en tiempo real.

- **¿How much? (¿Cuánto afecta?)**

Puede afectar económicamente mediante pérdidas monetarias, pagos incorrectos, decisiones tardías, menor eficiencia operativa y limitaciones para escalar el negocio de manera sostenible.

1.2.2. Propuesta de valor

La propuesta de valor de ControlBet se centra en el diseño de BetCore DB, un modelo de gestión y monitoreo de datos para casas de apuestas deportivas. Esta solución surge frente a la necesidad de organizar de manera eficiente la información operativa y transaccional del negocio, especialmente en contextos donde los registros de apuestas, pagos, usuarios, cuotas, eventos deportivos y resultados pueden encontrarse dispersos o poco estructurados.

El problema principal que se busca resolver es la dificultad que tienen algunas casas de apuestas pequeñas y medianas para controlar sus datos de forma ordenada, confiable y trazable. Esta situación puede generar errores en el registro de apuestas, inconsistencias en movimientos financieros, demoras en la validación de resultados y limitaciones para consultar información histórica que apoye la toma de decisiones.

Ante ello, BetCore DB propone una arquitectura de base de datos relacional orientada a estructurar, relacionar y consultar información del negocio. Su finalidad no es desarrollar una plataforma final, sino establecer una base de datos robusta que permita mejorar el control interno, facilitar el monitoreo de operaciones y servir como soporte para futuros sistemas de gestión o análisis.

Los principales usuarios beneficiados por esta solución son los propietarios, encargados y asistentes de casas de apuestas deportivas, quienes necesitan supervisar operaciones, controlar pagos, revisar apuestas y consultar reportes. También se consideran como usuarios relevantes los clientes

apostadores, ya que sus datos, apuestas, comprobantes, saldos e historial deben ser correctamente registrados para garantizar transparencia y trazabilidad.

Los objetivos de ControlBet, son diseñar una solución de datos clara, escalable y consistente; reducir la redundancia e inconsistencia de información; mejorar la trazabilidad de las apuestas y transacciones financieras; y facilitar el análisis de datos para apoyar decisiones estratégicas dentro del negocio.

Entre los principales beneficios se encuentran el mayor control sobre las apuestas registradas, mejor seguimiento de pagos y movimientos monetarios, reducción de errores operativos, acceso más ordenado a información histórica, mayor transparencia para los clientes y una base estructurada que permita al negocio crecer de manera más segura y eficiente.

Lean UX Canvas – BetCore DB

Elemento	Desarrollo
Business Outcomes	Reducir inconsistencias, mejorar trazabilidad, facilitar consultas históricas y apoyar decisiones estratégicas.
Users	Propietarios, encargados, asistentes, personal financiero y clientes apostadores.
User Outcomes & Benefits	Mayor control operativo, mejor seguimiento de pagos, historial de apuestas más claro y reducción de errores.
Solutions	Diseño de una arquitectura de base de datos relacional para estructurar y relacionar información crítica del negocio.
Hypotheses	Si una casa de apuestas cuenta con una base de datos bien estructurada, podrá mejorar el control de sus operaciones y reducir pérdidas monetarias.

What's the most important thing we need to learn first?	Identificar qué datos son indispensables para registrar, consultar y controlar apuestas, pagos, usuarios y resultados.
What's the least amount of work we need to do to learn the next most important thing?	Entrevistar usuarios clave, analizar procesos actuales y construir un modelo entidad-relación inicial.

1.3. Segmentos objetivo

ControlBet se dirige a dos segmentos principales dentro del dominio de las apuestas deportivas.

El primer segmento está conformado por encargados, asistentes o propietarios de casas de apuestas pequeñas y medianas en etapa de crecimiento, que requieren optimizar la gestión de su información operativa y transaccional mediante una base de datos estructurada. Este grupo necesita organizar registros de apuestas, pagos, ingresos, resultados y movimientos financieros para mejorar su control interno y reducir errores operativos.

El segundo segmento está conformado por clientes o personas que realizan apuestas deportivas, quienes interactúan directamente con los procesos del negocio al registrar apuestas, realizar pagos, consultar resultados y solicitar validaciones de sus jugadas. Este segmento resulta relevante porque permite identificar qué datos del usuario apostador deben ser almacenados, cómo se relacionan con las apuestas realizadas y qué información es importante para garantizar trazabilidad y seguimiento.

En el contexto peruano, este mercado presenta un crecimiento sostenido. Según información oficial del Ministerio de Comercio Exterior y Turismo (Mincetur), durante 2025 se registraron 4,583 locales autorizados vinculados al rubro y se incorporaron 1,756 nuevas salas de apuestas, evidenciando la expansión del sector y la necesidad de soluciones tecnológicas orientadas al control de operaciones. Asimismo, desde el punto de vista demográfico, el segmento se concentra principalmente en zonas urbanas con alta actividad comercial, mayor acceso digital y

presencia de establecimientos formales, como Lima Metropolitana y principales ciudades del país.

CAPÍTULO II: RECOPIACIÓN Y ANÁLISIS DE REQUISITOS

2.1. Entrevistas

En esta sección se presenta el proceso de recolección de información realizado mediante entrevistas a representantes de los segmentos objetivo del proyecto. Estas entrevistas permitieron conocer las necesidades, dificultades y expectativas relacionadas con la gestión de datos en casas de apuestas deportivas.

2.1.1. Diseño de entrevistas

Con la finalidad de recopilar información relevante sobre la problemática identificada en el sector de apuestas deportivas, se diseñó un proceso de entrevistas dirigido a los segmentos objetivo definidos previamente. Estas entrevistas permitirán conocer la situación actual de los negocios, los procesos operativos que manejan, las dificultades relacionadas con el control de datos y las necesidades existentes respecto a una mejor gestión de información.

Asimismo, la información obtenida servirá como base para identificar los principales requerimientos del sistema y reconocer entidades fundamentales para el diseño de la base de datos, tales como usuarios, apuestas, pagos, locales, reportes, movimientos financieros y resultados deportivos.

Propietarios, asistentes o encargados de casas de apuestas

Introducción

- Buenos días/tardes/noches. En esta reunión realizaremos una entrevista para conocer las necesidades que tienen las casas de apuestas deportivas.
- Con la finalidad es identificar oportunidades de mejora relacionadas con el control de operaciones, apuestas y movimientos financieros.
- Para el registro de la entrevista, por favor indíquenos sus nombres, apellidos, edad, distrito y cargo dentro del negocio.

- A continuación, iniciaremos con las preguntas.

Preguntas generales

- ¿Cuánto tiempo lleva operando su casa de apuestas deportivas?
- ¿Cuántos trabajadores aproximadamente participan en la operación diaria del negocio?

Preguntas principales

- ¿Cómo registran actualmente las apuestas realizadas por los clientes?
- ¿Qué información consideran importante almacenar de cada apuesta?
- ¿Cómo controlan los ingresos y pagos diarios del negocio?
- ¿Qué dificultades tienen al momento de buscar información histórica de apuestas o transacciones?
- ¿Manejan reportes diarios, semanales o mensuales? ¿Qué datos incluyen?
- ¿Cómo realizan el seguimiento de resultados deportivos vinculados a apuestas activas?
- ¿Han tenido pérdidas económicas por errores de registro o falta de control?
- ¿Qué tipo de alertas o indicadores les gustaría visualizar para mejorar decisiones?

Preguntas complementarias

- ¿Desean controlar accesos por roles (administrador, caja, supervisor)?
- ¿Qué información de clientes suelen registrar?
- ¿Con qué frecuencia revisan reportes financieros?
- ¿Qué tan importante sería consultar información en tiempo real?

Clientes o personas que realizan apuestas deportivas

Introducción

- Buenos días/tardes/noches. En esta reunión realizaremos una entrevista para conocer la experiencia y las necesidades de las personas que realizan apuestas deportivas.

- Con la finalidad es comprender cómo interactúan con este tipo de servicios, qué información consideran importante durante el proceso de apuesta y qué dificultades suelen presentarse en estas casas de apuestas.
- Para el registro de la entrevista, por favor indíquenos sus nombres, apellidos, edad, distrito y ocupación.
- A continuación, iniciaremos con las preguntas.

Preguntas generales

- ¿Con qué frecuencia realiza apuestas deportivas?
- ¿En qué tipo de eventos deportivos apuesta con mayor frecuencia?
- ¿Suele realizar sus apuestas de manera presencial, virtual o ambas?

Preguntas principales

- ¿Qué información considera más importante cuando realiza una apuesta?
- ¿Qué datos suelen solicitarle al momento de registrar una apuesta?
- ¿Cómo verifica normalmente el estado de sus apuestas, si están activas, ganadas, perdidas o pendientes?
- ¿Ha tenido problemas con el registro de una apuesta, un pago o la validación de resultados?
- ¿Qué tan importante considera recibir información clara sobre cuotas, montos apostados, premios potenciales y resultados?
- ¿Qué aspectos generan mayor desconfianza o incomodidad durante el proceso de apuesta?
- ¿Qué mejoras considera necesarias para que el registro y seguimiento de sus apuestas sea más ordenado y transparente?

Preguntas complementarias

- ¿Suele conservar comprobantes o registros de sus apuestas?
- ¿Le parece importante que el sistema almacene su historial de apuestas anteriores?
- ¿Ha tenido que reclamar alguna vez por un error en montos, pagos o resultados?

2.1.2. Registro de entrevistas

Segmento #1: Propietarios, Asistentes u Encargados de casas de apuestas

Entrevistado 1: Gamaniel Cruz Valerio

- **Sexo:** Masculino
- **Distrito:** San Juan de Lurigancho
- **Ocupación:** Encargado de la casa de apuestas
- **Link:** <https://youtu.be/FOZ8yaOpJDI>
- **Duración:** 18 minutos y 35 segundos

Resumen:

El entrevistado, Gamaniel, de 27 años, se desempeña como encargado en un local de la casa de apuestas "Te Apuesto", perteneciente a La Tinka, donde cuenta con tres años de experiencia. El local opera con un equipo de dos personas y maneja dos modalidades de apuesta: una a través de una aplicación móvil y otra de forma presencial mediante la emisión de tickets físicos. Para la atención en el local, disponen de dos terminales, uno de autoservicio para los clientes y otro de uso exclusivo para los trabajadores.

Durante la atención presencial, es un requisito obligatorio solicitar el DNI del cliente al registrar la apuesta para garantizar que sea mayor de edad. Sin embargo, el alto volumen diario de transacciones dificulta el manejo de la información, lo que en ocasiones genera errores humanos al momento de ingresar los datos. En este proceso, el ticket físico resulta una herramienta indispensable, ya que su código de barras permite acceder a toda la información de la apuesta al ser escaneado, incluyendo el evento deportivo, la cuota de ganancia, el monto a cobrar y los datos del usuario.

A nivel de gestión de datos, la empresa recopila información detallada como el DNI, correos electrónicos y datos biométricos. Esta base de datos se utiliza, entre otras cosas, para enviar bonos promocionales a los clientes vía correo electrónico. En cuanto al manejo administrativo y financiero, las ventas y pagos diarios se registran mediante hojas de Excel. El sistema operativo de las máquinas funciona bajo perfiles o roles asignados a cada trabajador y permite generar distintos reportes, categorizando la información entre apuestas

deportivas y juegos virtuales. Adicionalmente, existe una normativa estricta respecto a los retiros: cuando un premio excede los 1,000 soles, el local ya no realiza el pago en efectivo, sino que La Tinka asume la responsabilidad de abonar el dinero directamente a través de la aplicación.

Entrevistado 2: Manuel Sanchez

- **Sexo:** Masculino
- **Distrito:** Surquillo
- **Ocupación:** Administrador de local de apuestas
- **Link:** https://youtu.be/sZN-aLleVYY?si=95O2da4x-jWfa_FX
- **Duración:** 3 minutos y 28 segundos

Resumen:

Manuel Sanchez es administrador de un local de apuestas deportivas ubicado en el distrito de Surquillo, Lima, y cuenta con experiencia directa en la gestión operativa y financiera del negocio. El local opera con una modalidad de atención presencial, donde el registro de apuestas se realiza a través de terminales de uso interno, y los clientes pueden acercarse a las ventanillas para realizar sus jugadas. Manuel destaca que, en el contexto de Surquillo, el flujo de clientes es constante durante los días con partidos de fútbol, lo que genera un volumen considerable de transacciones en horarios de alta demanda.

En cuanto al registro de información, Manuel señala que cada apuesta requiere la identificación del cliente mediante su documento de identidad, especialmente para el cobro de premios y para cumplir con los controles internos del negocio. La información que consideran más relevante almacenar incluye el tipo de evento deportivo, la cuota seleccionada, el monto apostado y el estado del ticket. Sin embargo, reconoce que parte de este registro aún se realiza de manera manual o a través de hojas de cálculo, lo que dificulta el acceso rápido a la información histórica cuando se necesita revisar operaciones pasadas o resolver dudas de los clientes.

Respecto al control financiero, Manuel menciona que los ingresos y egresos diarios se registran al cierre de cada jornada, pero que en ocasiones se presentan inconsistencias entre el registro físico de los tickets y los reportes

del sistema. Esta situación genera demoras al momento de cuadrar cuentas y puede derivar en errores en los pagos de premios. Asimismo, señala que el seguimiento de resultados deportivos vinculados a apuestas activas no siempre se realiza de manera automatizada, lo que obliga al personal a verificar manualmente los marcadores al término de cada partido.

En cuanto a las mejoras que considera necesarias, Manuel expresa la importancia de contar con un sistema que centralice la información de apuestas, pagos y clientes en una sola plataforma, evitando la duplicidad de registros y reduciendo los errores humanos. Indica que le resultaría muy útil contar con alertas que notifiquen automáticamente cuando un resultado es validado y cuando los premios están listos para ser cobrados, así como reportes diarios que reflejen con precisión el estado financiero del local. Finalmente, subraya que el acceso a información en tiempo real y la posibilidad de consultar el historial de operaciones de cada cliente representaría una ventaja significativa para la gestión del negocio.

Segmento #2: Clientes o personas que realizan apuestas deportivas

Entrevistado 3: Byron Tirado

- **Sexo:** Masculino
- **Distrito:** San Juan de Lurigancho
- **Ocupación:** Administrador de Negocio Familiar
- **Link:** https://youtu.be/ohdtMp_Q5e4?si=Nxul8DlFHF42eebd
- **Duración:** 4 minutos y 25 segundos

Resumen:

Byron es un apostador frecuente que realiza entre una y dos apuestas diarias, enfocándose principalmente en la NBA y el fútbol. Aunque inició en la modalidad presencial, actualmente prefiere las apuestas virtuales por comodidad. Su proceso incluye una investigación previa sobre el desempeño y la competencia de los equipos. Ha reportado inconvenientes con la lentitud de las plataformas en vivo y retrasos significativos en la validación de montos o pagos. Sugiere que las casas de apuestas mejoren la eficacia de sus servidores y diseñen interfaces más dinámicas para facilitar las apuestas al instante.

Entrevistado 4: Jefferson Pérez Pino

- **Sexo:** Masculino
- **Distrito:** El Tambo
- **Ocupación:** Docente de Ingles
- **Link:** <https://youtu.be/X34vMN5ummw>
- **Duración:** 4 minutos y 13 segundos

Resumen:

Se realizó una entrevista a Jeferson Pérez, profesor de inglés residente en Huancayo, quien participa en apuestas deportivas de manera ocasional. Indicó que suele apostar principalmente durante eventos deportivos de gran relevancia como la Champions League, el Mundial y la Liga 1, enfocándose exclusivamente en el fútbol. Asimismo, señaló que realiza sus apuestas de manera virtual a través de plataformas en línea. El entrevistado mencionó que verifica el estado de sus apuestas (activas, ganadas, perdidas o pendientes) directamente en la plataforma digital, valorando la rapidez y accesibilidad de esta información. En su experiencia personal, no ha tenido inconvenientes relacionados con el registro de apuestas, pagos o validación de resultados, por lo que no ha necesitado realizar reclamos.

Sin embargo, comentó que algunos de sus conocidos han presentado problemas, especialmente bloqueos en sus cuentas. Según su percepción, esto podría estar relacionado con la realización de apuestas de montos elevados, lo que genera cierta desconfianza en el funcionamiento de estas plataformas. También mencionó que, en algunos casos, las cuentas pueden ser limitadas a montos muy bajos, lo cual desincentiva la participación del usuario. En cuanto a la confianza y transparencia, considera fundamental que las plataformas brinden información clara sobre las cuotas, montos apostados, premios potenciales y resultados. Asimismo, destacó la importancia de contar con registros actualizados y de fácil acceso, que permitan al usuario llevar un mejor control de sus apuestas y tomar decisiones informadas.

Finalmente, señaló que no suele tener la necesidad de guardar comprobantes físicos, ya que confía en el registro digital de la plataforma. No

obstante, considera importante que el sistema almacene un historial completo de apuestas, lo cual contribuiría a mejorar la organización, el seguimiento y la transparencia del servicio.

Entrevistado 5: Luis Fernando Ramírez Torres

- **Sexo:** Masculino
- **Distrito:** Los Olivos
- **Ocupación:** Técnico administrativo
- **Link:** <https://youtu.be/ldQ-6H2F2Go>
- **Duración:** 7 minutos y 54 segundos

Resumen:

Luis Fernando Ramírez Torres, de 28 años, vive en Los Olivos y trabaja como técnico administrativo. Es una persona que realiza apuestas deportivas con frecuencia moderada, aproximadamente dos o tres veces por semana, especialmente cuando hay partidos importantes de fútbol. Sus apuestas se enfocan principalmente en la Liga 1 de Perú, Champions League, Premier League y partidos de selecciones, aunque ocasionalmente también apuesta en básquet.

El entrevistado utiliza tanto la modalidad virtual como presencial, aunque prefiere apostar de manera virtual por la rapidez y comodidad. Sin embargo, también acude a locales físicos cuando desea pagar en efectivo o recibir un comprobante físico.

Durante el proceso de apuesta, considera importante revisar la cuota, el monto apostado, el premio potencial, el tipo de apuesta y el estado del evento. Además, señala que al registrar una apuesta normalmente se solicitan datos como el usuario, monto, evento deportivo, equipo o resultado elegido, tipo de apuesta y, en algunos casos, el DNI.

Luis Fernando manifiesta que ha tenido problemas con la validación de resultados, especialmente cuando una apuesta queda pendiente durante varias horas después de finalizado el partido. También menciona que a veces no se explica claramente por qué una apuesta combinada se marca como perdida, lo cual genera confusión y desconfianza.

Para él, es muy importante que el sistema muestre información clara sobre cuotas, montos apostados, premios potenciales y resultados. Considera que una mejora necesaria sería contar con un historial detallado de apuestas, donde se pueda visualizar la fecha, hora, monto, cuota, posible premio y resultado final. También considera útil recibir avisos cuando una apuesta cambie de estado o cuando un pago esté disponible.

Finalmente, el entrevistado suele conservar comprobantes cuando apuesta presencialmente y revisa el historial cuando apuesta virtualmente. Además, considera importante que el sistema almacene el historial de apuestas anteriores, ya que esto permite controlar mejor el dinero apostado, revisar resultados y realizar reclamos en caso de errores.

2.1.3. Análisis de entrevistas

A partir de las entrevistas registradas, se realizó un análisis por cada segmento objetivo con la finalidad de identificar las características más representativas del problema, lo que permitirá reconocer las necesidades clave para la construcción de arquetipos y para la identificación de entidades relevantes en el diseño de la base de datos.

Segmento #1: Propietarios, asistentes o encargados de casas de apuestas

En este segmento, el análisis se basa en la entrevista realizada a un administrador y encargado de una casa de apuestas con experiencia operativa directa en el negocio. A partir de ello, se identifican las siguientes características:

Características objetivas

- 100% de los entrevistados de este segmento indicó que en el negocio se manejan modalidades de apuesta presenciales y digitales. En el caso analizado, el local opera con atención presencial mediante tickets físicos y también mediante aplicación móvil.
- 100% indicó que se registran datos relevantes del cliente, tales como DNI, correo electrónico y otros datos asociados al usuario, lo que demuestra la importancia de contar con una estructura clara para el almacenamiento de información personal y transaccional.

- 100% manifestó que las ventas y pagos diarios todavía se apoyan en herramientas como hojas de Excel, lo cual evidencia una gestión parcialmente manual y una oportunidad de mejora en la organización y trazabilidad de la información.
- 100% afirmó que el negocio utiliza roles de acceso para operar el sistema y genera reportes diferenciados, lo cual justifica la futura incorporación de entidades como usuario, rol, reporte y movimiento financiero dentro del modelo de datos.

Características subjetivas

- 100% percibe que el alto volumen de transacciones diarias dificulta el manejo de la información y puede generar errores humanos en el registro de datos. Esto revela una necesidad fuerte de control, consistencia y validación en la base de datos.
- 100% considera que el ticket físico y su código de barras son elementos clave para el seguimiento de la apuesta, lo que refleja la importancia de la trazabilidad y del acceso rápido a la información asociada a cada operación.
- 100% valora el uso de la información no solo para registrar apuestas, sino también para apoyar procesos del negocio como promociones y seguimiento de clientes, lo que evidencia una visión operativa y comercial del dato.

Interpretación del segmento

Este segmento representa a un perfil orientado al control operativo y financiero del negocio. Sus principales preocupaciones se relacionan con la organización de la información, la reducción de errores, la trazabilidad de las apuestas y la generación de reportes. A partir de esta entrevista, se refuerza la necesidad de considerar entidades como Usuario, Cliente, Apuesta, Comprobante/Ticket, Rol, Reporte, Transacción y Pago. Asimismo, este análisis sirve de base para construir un arquetipo de tipo gestor operativo, cuya prioridad es supervisar el negocio con información ordenada, accesible y confiable.

Segmento #2: Clientes o personas que realizan apuestas deportivas

En este segmento, el análisis se basa en cinco entrevistas válidas registradas. A partir de ellas, se identifican patrones comunes y diferencias de comportamiento relevantes para el proyecto.

Características objetivas

- 80% de los entrevistados realiza actualmente sus apuestas por canales virtuales, aunque uno de ellos señaló haber iniciado en modalidad presencial y luego migrado a la modalidad digital por comodidad. Esto demuestra una preferencia clara por el acceso remoto y la inmediatez del servicio.
- 100% de los entrevistados tiene al fútbol como referencia importante dentro de sus apuestas. Además, 50% también apuesta en otros deportes, como la NBA, lo que indica que el sistema debe poder registrar distintos tipos de eventos deportivos y no limitarse a una sola disciplina.
- 60% corresponde a un perfil de apostador frecuente, que realiza entre una y dos apuestas diarias; el otro 40% corresponde a un perfil ocasional, que apuesta principalmente en eventos de gran relevancia. Esto muestra que dentro del mismo segmento existen distintos niveles de intensidad de uso.
- 60% reportó problemas directos relacionados con la lentitud de la plataforma y retrasos en validación de montos o pagos, mientras que el otro 40% no reportó inconvenientes personales, aunque sí mencionó casos cercanos de bloqueos y limitaciones de cuenta. Esto revela que la confiabilidad operativa es una variable para el usuario final.

Características subjetivas

- 100% valora la comodidad y rapidez al momento de apostar, lo que sugiere que el usuario espera procesos simples, respuesta inmediata y consulta ágil de información.

- 100% asocia la confianza en la casa de apuestas con la transparencia de la información, especialmente en aspectos como cuotas, montos apostados, premios potenciales, resultados e historial de operaciones.
- 60% expresó explícitamente la necesidad de mejorar la infraestructura tecnológica, especialmente en el rendimiento de plataformas en vivo; el otro 40% destacó la importancia de que la información se mantenga actualizada y disponible para la toma de decisiones. En ambos casos, se evidencia que la calidad del servicio depende del acceso oportuno a los datos.
- 60% indicó de manera expresa que no necesita comprobantes físicos porque confía en el registro digital, pero al mismo tiempo resaltó la importancia de contar con un historial completo de apuestas, lo cual refuerza la necesidad de almacenamiento y seguimiento digital.

Interpretación del segmento

Este segmento representa a usuarios que priorizan la rapidez, claridad y confiabilidad en sus operaciones. A partir del análisis, se observa que los clientes requieren consultar fácilmente sus apuestas, estados, montos, cuotas y resultados, así como disponer de un historial de operaciones que les permita validar sus movimientos. A partir de estas entrevistas se refuerza la necesidad de incluir entidades como Cliente/Usuario, Apuesta, Detalle de Apuesta, Evento Deportivo, Cuota, Resultado, Historial de Apuestas, Pago y Comprobante. Además, permite identificar dos arquetipos preliminares: el apostador frecuente digital, que valora velocidad y operatividad constante, y el apostador ocasional orientado a eventos relevantes, que prioriza confianza y transparencia en el servicio.

2.2. Requisitos

A partir de las entrevistas realizadas a los segmentos objetivo, se identificaron los siguientes requisitos funcionales y no funcionales necesarios para la solución propuesta. Estos requisitos responden a la necesidad de mejorar la organización de la información operativa y transaccional, fortalecer la trazabilidad de las apuestas, facilitar el control financiero y permitir un acceso más claro y confiable a los datos relevantes del negocio.

Requisitos funcionales

Código	User Story	Requisito funcional
RF-01	Como administrador o encargado, quiero registrar personas, usuarios y trabajadores para mantener organizada la información de los actores que intervienen en la operación del negocio.	El sistema debe permitir registrar datos de identificación, contacto y estado de personas, usuarios y trabajadores asociados a la casa de apuestas.
RF-02	Como encargado del local, quiero validar la identidad y mayoría de edad del cliente apostador para cumplir con las reglas del negocio antes de registrar una apuesta.	El sistema debe permitir almacenar datos de identificación del cliente, como tipo y número de documento, fecha de nacimiento y país asociado.
RF-03	Como administrador, quiero gestionar roles de trabajadores para asignar responsabilidades según la función operativa dentro del negocio.	El sistema debe permitir registrar roles como administrador, cajero o supervisor, y asociarlos a los trabajadores correspondientes.
RF-04	Como administrador o encargado, quiero registrar locales y trabajadores asignados para controlar qué personal opera en cada establecimiento.	El sistema debe permitir registrar locales y relacionarlos con los trabajadores asignados, incluyendo fecha de asignación y estado.
RF-05	Como encargado o supervisor, quiero registrar deportes, países, ligas, equipos y eventos deportivos para organizar la información deportiva vinculada con las apuestas.	El sistema debe permitir almacenar y relacionar la información de deportes, países, ligas, equipos y eventos deportivos.

RF-06	Como operador, quiero registrar mercados de apuesta y cuotas por evento para estructurar correctamente las opciones disponibles para apostar.	El sistema debe permitir registrar mercados de apuesta asociados a eventos deportivos y cuotas relacionadas con cada mercado.
RF-07	Como cliente apostador, quiero realizar apuestas sobre cuotas disponibles para que mi operación quede registrada de manera ordenada y trazable.	El sistema debe permitir registrar apuestas indicando usuario, monto apostado, tipo de apuesta, cuota total, fecha, estado, moneda y código de ticket.
RF-08	Como encargado, quiero consultar el detalle de una apuesta para verificar las cuotas seleccionadas y el resultado de cada selección.	El sistema debe permitir registrar y consultar el detalle de cada apuesta, incluyendo cuota seleccionada, valor de cuota y resultado individual.
RF-09	Como cliente, quiero consultar mi historial de apuestas para hacer seguimiento a mis operaciones anteriores y validar resultados.	El sistema debe permitir consultar las apuestas registradas por usuario, incluyendo fecha, monto, estado, cuota total y código de ticket.
RF-10	Como encargado o cajero, quiero registrar transacciones financieras para controlar depósitos, retiros, pagos de premios, reembolsos y ajustes de saldo.	El sistema debe permitir registrar transacciones financieras asociadas a un usuario, indicando tipo de transacción, método de pago, moneda, monto, fecha y estado.
RF-11	Como encargado, quiero consultar el estado de pagos y movimientos financieros para	El sistema debe permitir consultar el estado y detalle de las transacciones financieras asociadas a cada usuario.

	tener claridad sobre depósitos, retiros y premios.	
RF-12	Como encargado, quiero emitir comprobantes de transacciones financieras para respaldar las operaciones económicas realizadas.	El sistema debe permitir almacenar comprobantes asociados a transacciones financieras, incluyendo número de comprobante, fecha de emisión y enlace o ruta del documento digital.
RF-13	Como encargado, quiero contar con un código de ticket por apuesta para identificar y consultar fácilmente cada operación realizada.	El sistema debe permitir registrar un código único de ticket asociado a cada apuesta.
RF-14	Como operador o supervisor, quiero registrar resultados de eventos deportivos para actualizar el estado de las apuestas relacionadas.	El sistema debe permitir registrar resultados de eventos y mantener actualizado el estado de las apuestas según el resultado correspondiente.
RF-15	Como cliente, quiero contar con información clara sobre cuotas, montos apostados, premios potenciales y resultados para tener mayor confianza en el proceso de apuesta.	El sistema debe almacenar información esencial de cada apuesta, incluyendo monto apostado, cuota total, estado, fecha y resultado asociado.
RF-16	Como responsable financiero, quiero mantener actualizado el saldo del usuario para controlar el dinero disponible, retenido y total.	El sistema debe permitir registrar saldos por usuario y moneda, así como mantener un historial de cambios asociados a transacciones financieras.

Requisitos no funcionales

Código	Requisito no funcional	Descripción
RNF-01	Seguridad de la información	La base de datos debe proteger la información sensible de usuarios, trabajadores, apuestas y transacciones financieras mediante el uso de credenciales, roles y restricciones de acceso.
RNF-02	Integridad de los datos	La base de datos debe mantener la consistencia de la información mediante claves primarias, claves foráneas, restricciones de unicidad y reglas de integridad referencial.
RNF-03	Trazabilidad	La solución debe permitir el seguimiento de operaciones importantes, como apuestas registradas, cambios de saldo, transacciones financieras y emisión de comprobantes.
RNF-04	Rendimiento en consultas	La base de datos debe permitir consultas eficientes sobre apuestas, usuarios, eventos deportivos, saldos y transacciones, especialmente en escenarios de alta actividad operativa.
RNF-05	Escalabilidad	El modelo de base de datos debe permitir el crecimiento del negocio, incorporando nuevos usuarios, locales, eventos deportivos, mercados de apuesta y transacciones sin afectar su estructura principal.
RNF-06	Disponibilidad de la información	La información operativa y transaccional debe estar organizada de manera que pueda ser consultada oportunamente por los actores responsables del control y análisis del negocio.
RNF-07	Mantenibilidad	El diseño de la base de datos debe utilizar una nomenclatura clara, relaciones bien definidas y una estructura ordenada que facilite futuras modificaciones o ampliaciones.

RNF-08	Compatibilidad con SQL Server	La implementación física del modelo debe ser compatible con SQL Server, respetando su sintaxis, restricciones, tipos de datos y mecanismos de integridad.
RNF-09	Confiabilidad de los datos financieros	La base de datos debe asegurar que los registros de pagos, retiros, depósitos, saldos y premios se almacenen de forma consistente para reducir riesgos de pérdidas monetarias.
RNF-10	Respaldo y recuperación	La solución debe considerar la posibilidad de realizar copias de seguridad y recuperación de datos para evitar pérdida de información crítica del negocio.
RNF-11	Control de acceso por roles	El sistema debe controlar el acceso a la información según los roles asignados a cada usuario, permitiendo que administradores, encargados, cajeros o supervisores accedan solo a la información correspondiente a sus funciones.
RNF-12	Disponibilidad operativa	El sistema debe estar disponible durante los horarios de operación del negocio, especialmente en eventos deportivos de alta demanda, donde aumenta el volumen de apuestas y transacciones.

CAPÍTULO III: DISEÑO DE BASE DE DATOS

3.1. Entidades

En esta sección se identifican las principales entidades que forman parte de BetCore DB. Estas entidades representan objetos o conceptos relevantes dentro del proceso de gestión de datos de una casa de apuestas deportivas, considerando la información obtenida en las entrevistas, los requisitos funcionales y la necesidad de estructurar adecuadamente los datos operativos y transaccionales del negocio.

Entidad	Descripción
Persona	Representa a la persona como entidad base dentro del modelo. Permite almacenar información general de identificación que posteriormente puede relacionarse con usuarios del sistema o trabajadores de la casa de apuestas.
Trabajador	Representa al personal interno de la casa de apuestas, como encargados, asistentes o administradores. Esta entidad permite diferenciar al personal operativo del cliente apostador.
Usuario	Representa a las personas registradas dentro de la solución de datos para interactuar con las operaciones del negocio. Incluye tanto a clientes apostadores como a otros actores que requieran trazabilidad dentro del modelo. Se relaciona con apuestas, transacciones financieras y saldo.
Rol	Representa la función o perfil asignado a un trabajador dentro del negocio, por ejemplo administrador, cajero o supervisor. Es importante para organizar responsabilidades y control de acceso según el tipo de actor.
Local	Representa el establecimiento físico donde opera la casa de apuestas. Permite identificar la sede donde se

	registran apuestas, pagos u otras operaciones del negocio.
TrabajadorxLocal	Representa la relación entre un trabajador y el local donde desempeña sus funciones. Permite registrar en qué sede opera cada trabajador y mantener control organizacional.
Cuota	Representa el valor asociado a una opción de apuesta dentro de un mercado. Es fundamental porque determina la relación entre el monto apostado y la posible ganancia del usuario.
MercadoApuesta	Representa el tipo de mercado disponible dentro de un evento, por ejemplo ganador del partido, total de goles o marcador exacto. Esta entidad es importante porque organiza las distintas opciones sobre las que se puede apostar.
Apuesta	Representa la operación principal del negocio, es decir, la jugada realizada por un usuario sobre uno o más eventos deportivos. Es una entidad central porque concentra la información relacionada con el monto apostado, estado, cuota y resultado esperado.
EstadoApuesta	Representa la condición actual de una apuesta, por ejemplo pendiente, ganada, perdida o anulada. Su propósito es facilitar el seguimiento y trazabilidad del ciclo de vida de la apuesta.
DetalleApuesta	Representa el detalle específico de cada apuesta. Permite descomponer una apuesta en sus elementos particulares, como selecciones, cuotas asociadas o componentes internos de la jugada.
Transacción Financiera	Representa los movimientos monetarios asociados a los usuarios, tales como depósitos, retiros, pagos de premios o ajustes. Es fundamental para controlar el flujo económico del negocio y reducir inconsistencias financieras.

TipoTransaccion	Representa la clasificación de una transacción financiera, por ejemplo depósito, retiro, pago o reembolso. Facilita la organización y análisis de los distintos movimientos económicos.
EstadoTransaccion	Representa la condición en la que se encuentra una transacción financiera, como pendiente, procesada o anulada. Su finalidad es permitir el seguimiento del estado de cada movimiento monetario.
Comprobante	Representa el documento o evidencia asociada a una transacción financiera. Puede utilizarse para respaldar operaciones económicas y brindar trazabilidad documental dentro del negocio.
MetodoPago	Representa la forma a través de la cual se realiza una transacción financiera, por ejemplo efectivo, transferencia, billetera digital u otro medio autorizado.
HistorialPago	Representa el registro histórico de cambios en el saldo de un usuario. Su función es permitir trazabilidad sobre variaciones monetarias producidas por apuestas, pagos, depósitos o retiros.
ApuestaTransaccion	Representa la relación entre una apuesta y una transacción financiera. Esta entidad permite vincular la operación deportiva con el movimiento económico correspondiente, como el cargo de la apuesta o el pago de un premio.
Saldo	Representa el balance económico asociado a un usuario. Esta entidad permite conocer la disponibilidad de fondos y mantener control sobre el dinero relacionado con apuestas y transacciones.
Moneda	Representa el tipo de divisa utilizada en las apuestas y transacciones financieras, por ejemplo soles o dólares. Permite mantener consistencia en las operaciones monetarias del negocio.

Deporte	Representa la disciplina deportiva sobre la cual se pueden organizar eventos y apuestas, como fútbol, básquet o tenis. Es una entidad base para estructurar la oferta deportiva.
Equipo	Representa a los equipos o participantes que intervienen en los eventos deportivos. Es una entidad clave porque permite identificar a los actores deportivos sobre los cuales se realizan las apuestas.
País	Representa el país asociado a ligas, equipos u otros elementos del dominio deportivo. Esta entidad ayuda a normalizar la información geográfica del modelo.
Liga	Representa la competición o torneo al que pertenece un evento deportivo. Su finalidad es organizar eventos dentro de un contexto competitivo determinado.
Evento	Representa cada partido o competencia deportiva sobre la cual los usuarios pueden realizar apuestas. Esta entidad almacena la ocurrencia específica de un encuentro dentro de una liga o deporte determinado.
EventoParticipante	Representa la relación entre un evento y los equipos o participantes que intervienen en él. Permite establecer qué equipos forman parte de cada evento deportivo.

3.2. Atributos

En esta sección se describen los principales atributos de las entidades identificadas, los atributos representan las características necesarias para almacenar, organizar y consultar información relacionada con usuarios, trabajadores, apuestas, eventos deportivos, transacciones financieras, saldos y comprobantes.

Entidad	Atributo	Descripción
Persona	id_persona	Identificador único de la persona (Primary Key).

	nombres	Nombres de la persona registrada.
	apellidos	Apellidos de la persona registrada.
	tipo_documento	Tipo de documento de identidad, como DNI, CE o pasaporte.
	numero_documento	Número de documento de identidad.
	correo	Correo electrónico de contacto.
	telefono	Número telefónico de la persona.
	fecha_nacimiento	Fecha de nacimiento, útil para validar la mayoría de edad.
	id_pais	Identificador del país asociado a la persona.
Usuario	id_usuario	Identificador único del usuario.
	fecha_registro	Fecha en que el usuario fue registrado.
	password_hash	Contraseña cifrada del usuario.
	estado	Estado del usuario, activo o inactivo.
	id_persona	Identificador de la persona asociada al usuario.
Trabajador	id_trabajador	Identificador único del trabajador.
	username	Nombre de usuario utilizado para el acceso interno.
	password_hash	Contraseña cifrada del trabajador.
	estado	Estado del trabajador dentro del negocio.

	id_rol	Identificador del rol asignado al trabajador.
	id_persona	Identificador de la persona asociada al trabajador.
Rol	id_rol	Identificador único del rol.
	nombre_rol	Nombre del rol, por ejemplo administrador, cajero o supervisor.
	descripcion	Descripción breve de las funciones del rol.
Local	id_local	Identificador único del local.
	nombre_local	Nombre comercial del establecimiento.
	direccion	Dirección física del local.
	distrito	Distrito donde se ubica el local.
	ciudad	Ciudad donde opera el local.
	telefono	Número de contacto del establecimiento.
	horario_atencion	Horario de funcionamiento del local.
	estado	Estado del local, activo o inactivo.
TrabajadorxLocal	id_trabajador	Identificador del trabajador asignado al local.
	id_local	Identificador del local asignado.
	fecha_asignacion	Fecha en que el trabajador fue asignado al local.
	estado	Estado de la asignación entre trabajador y local.
Pais	id_pais	Identificador único del país.
	nombre_pais	Nombre del país registrado.
Deporte	id_deporte	Identificador único del deporte.

	nombre_deporte	Nombre de la disciplina deportiva.
	estado	Estado del deporte dentro del catálogo.
Liga	id_liga	Identificador único de la liga.
	nombre_liga	Nombre de la competición deportiva.
	id_deporte	Identificador del deporte al que pertenece la liga.
	id_pais	Identificador del país asociado a la liga.
Equipo	id_equipo	Identificador único del equipo.
	nombre_equipo	Nombre del equipo o participante deportivo.
	id_deporte	Identificador del deporte al que pertenece el equipo.
	id_pais	Identificador del país de origen del equipo.
Evento	id_evento	Identificador único del evento deportivo.
	nombre_evento	Nombre descriptivo del partido o competencia.
	fecha_hora	Fecha y hora programada del evento.
	estado	Estado del evento, como programado, finalizado o suspendido.
	id_liga	Identificador de la liga a la que pertenece el evento.
EventoParticipante	id_evento	Identificador del evento deportivo.
	id_equipo	Identificador del equipo participante.

	condicion	Condición del equipo dentro del evento, por ejemplo local o visitante.
MercadoApuesta	id_mercado	Identificador único del mercado de apuesta.
	nombre_mercado	Nombre del mercado, por ejemplo ganador del partido o total de goles.
	estado	Estado del mercado de apuesta.
	descripcion	Descripción breve del mercado.
	id_evento	Identificador del evento asociado al mercado.
Cuota	id_cuota	Identificador único de la cuota.
	nombre_opcion	Nombre de la opción apostable dentro del mercado.
	valor	Valor numérico de la cuota.
	estado	Estado de la cuota, activa o inactiva.
	id_mercado	Identificador del mercado al que pertenece la cuota.
EstadoApuesta	id_estado_apuesta	Identificador único del estado de apuesta.
	nombre_estado	Nombre del estado, por ejemplo pendiente, ganada, perdida o anulada.
Apuesta	id_apuesta	Identificador único de la apuesta.
	codigo_ticket	Código único del ticket de apuesta.
	tipo_apuesta	Tipo de apuesta realizada, por ejemplo simple o combinada.
	monto_apostado	Monto económico apostado por el usuario.

	cuota_total	Valor total de la cuota asociada a la apuesta.
	fecha_apuesta	Fecha y hora en que se registró la apuesta.
	id_estado_apuesta	Identificador del estado actual de la apuesta.
	id_moneda	Identificador de la moneda utilizada.
	id_usuario	Identificador del usuario que realiza la apuesta.
	id_local	Identificador del local donde se registra la apuesta, si corresponde.
DetalleApuesta	id_apuesta	Identificador de la apuesta asociada.
	id_cuota	Identificador de la cuota seleccionada.
	valor_cuota	Valor de la cuota al momento de registrar la apuesta.
	resultado_detalle	Resultado individual del detalle de apuesta.
TipoTransaccion	id_tipo_transaccion	Identificador único del tipo de transacción.
	nombre_tipo	Nombre del tipo de movimiento financiero, como depósito, retiro o pago.
	naturaleza	Indica si la transacción representa ingreso o egreso.
	afecta_saldo	Indica si la transacción modifica el saldo del usuario.
EstadoTransaccion	id_estado	Identificador único del estado de transacción.

	nombre_estado	Nombre del estado, como pendiente, procesada, rechazada o anulada.
MetodoPago	id_metodo_pago	Identificador único del método de pago.
	nombre_metodo	Nombre del método de pago, como efectivo, transferencia o billetera digital.
	tipo_metodo	Clasificación del método de pago.
	estado	Estado del método de pago, activo o inactivo.
Moneda	id_moneda	Identificador único de la moneda.
	codigo_moneda	Código de la moneda, por ejemplo PEN o USD.
	nombre_moneda	Nombre de la moneda.
	tipo_cambio	Valor de cambio referencial, si aplica.
Transaccion-Financiera	id_transaccion	Identificador único de la transacción financiera.
	monto_total	Monto total registrado en la operación.
	monto_netto	Monto final procesado después de ajustes, si corresponde.
	fecha_solicitud	Fecha en que se solicita la transacción.
	fecha_proceso	Fecha en que la transacción es procesada.
	referencia_externa	Código o referencia externa de la operación.
	observacion	Comentario adicional sobre la transacción.

	id_tipo_transaccion	Identificador del tipo de transacción.
	id_estado	Identificador del estado de la transacción.
	id_metodo_pago	Identificador del método de pago utilizado.
	id_moneda	Identificador de la moneda de la transacción.
	id_usuario	Identificador del usuario asociado a la transacción.
ApuestaTransaccion	id_apuesta	Identificador de la apuesta relacionada.
	id_transaccion	Identificador de la transacción financiera relacionada.
Comprobante	id_comprobante	Identificador único del comprobante.
	numero_comprobante	Número único del comprobante emitido.
	fecha_emision	Fecha de emisión del comprobante.
	url_pdf	Ruta o enlace del comprobante digital.
	id_transaccion	Identificador de la transacción asociada al comprobante.
Saldo	id_saldo	Identificador único del saldo.
	monto_disponible	Monto disponible para realizar apuestas o retiros.
	monto retenido	Monto retenido por operaciones pendientes.
	monto_total	Suma total del saldo disponible y retenido.
	ultima_actualizacion	Fecha y hora de la última actualización del saldo.

	id_moneda	Identificador de la moneda del saldo.
	id_persona	Identificador del usuario asociado al saldo.
HistorialSaldo	id_historial_saldo	Identificador único del historial de saldo.
	saldo_anterior	Saldo registrado antes de la operación.
	saldo_posterior	Saldo registrado después de la operación.
	fecha_registro	Fecha en que se registró el cambio de saldo.
	id_saldo	Identificador del saldo afectado.
	id_transaccion	Identificador de la transacción que generó el cambio.

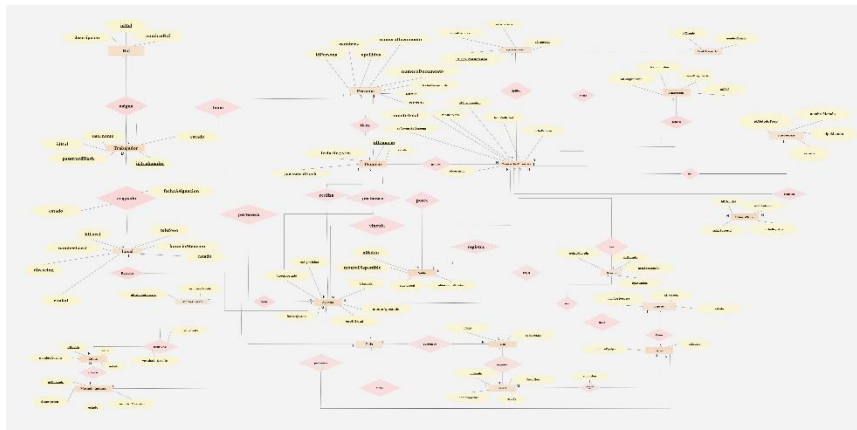
3.3. Enfoque relacional

En esta sección se presenta el enfoque relacional utilizado para organizar la información mediante entidades, atributos y relaciones claramente definidas. Este enfoque permite representar de manera estructurada los datos relacionados con usuarios, trabajadores, locales, apuestas, eventos deportivos, cuotas, transacciones financieras y saldos.

3.3.1. Diagrama entidad-relación conceptual

Figura 1

Diagrama entidad-relación conceptual del sistema

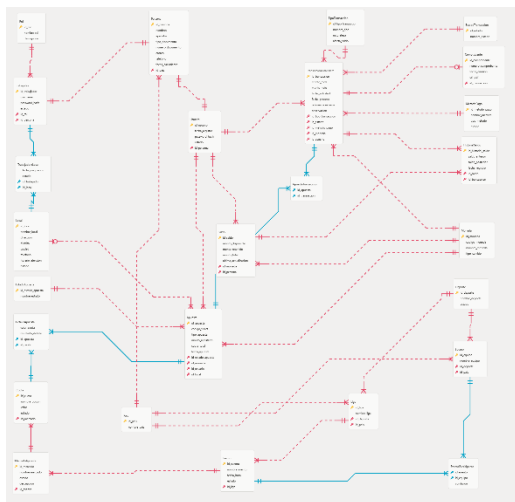


Nota. Debido a la complejidad del modelo, se recomienda visualizarlo en su tamaño original. Haga clic [aquí](#) para acceder a la versión en alta resolución.

3.3.2. Diagrama entidad-relación lógico

Figura 2

Diagrama entidad-relación lógico del sistema



Nota. Debido a la complejidad del modelo, se recomienda visualizarlo en su tamaño original. Haga clic [aquí](#) para acceder a la versión en alta resolución.

3.4. Enfoque no relacional

En esta sección se presenta el diseño de la base de datos no relacional para BetCore DB, considerando un enfoque documental orientado a representar la información del negocio mediante colecciones y documentos. Este diseño permite organizar datos con estructuras flexibles, embebiendo información relacionada cuando resulta conveniente para mejorar la consulta, trazabilidad y análisis de las operaciones.

3.4.1. Colecciones

En esta sección se identifican las colecciones principales de la base de datos no relacional de BetCore DB. Para este diseño se seleccionaron aquellas entidades que pueden ser representadas como documentos, debido a que

contienen información flexible, dinámica y relacionada directamente con las operaciones del negocio.

A diferencia del modelo relacional, donde la información se distribuye en múltiples tablas, en el enfoque documental se busca agrupar datos relacionados dentro de un mismo documento cuando esto facilita la lectura y el análisis. Por ello, se definieron dos colecciones principales: **apuestas** y **movimientos_financieros**.

Colección	Descripción	Justificación
apuestas	Almacena los documentos relacionados con las apuestas deportivas realizadas por los usuarios. Cada documento contiene el identificador de la apuesta, código de ticket, datos resumidos del usuario, local, selecciones realizadas, moneda, montos, estado de la apuesta y fechas de auditoría.	Esta colección concentra la información principal del negocio, ya que permite conocer quién apostó, en qué local, a qué deporte o evento apostó, qué opción seleccionó y cuánto apostó. Se utiliza un diseño documental porque una apuesta agrupa información que normalmente se consulta junta.
movimientos_financieros	Almacena los documentos relacionados con las operaciones económicas del negocio, como apuestas,	Esta colección permite controlar el impacto financiero de las operaciones. Se separa de

	depósitos, retiros, premios, reembolsos, comisiones y ajustes. Cada documento contiene la transacción, referencias al usuario y a la apuesta cuando corresponda, moneda, tipo de operación, estado, montos, método de pago, comprobante y fechas de auditoría.	apuestas para evitar duplicar toda la información del ticket dentro de cada movimiento financiero. En su lugar, se utilizan referencias implícitas como <code>usuario_id</code> y <code>apuesta_id</code> .
--	--	---

La separación en estas dos colecciones permite mantener un diseño claro y fácil de gestionar. La colección **apuestas** se enfoca en la operación principal del negocio, mientras que **movimientos_financieros** se orienta al control económico y transaccional. De esta forma, se evita una estructura excesivamente compleja y se mantiene una organización adecuada para el análisis de datos.

3.4.2. Validación de esquema

En esta sección se presentan las reglas de validación definidas para los documentos de las colecciones identificadas en la base de datos no relacional. Estas validaciones permiten controlar que los documentos almacenados cumplan con una estructura mínima, tipos de datos adecuados, valores permitidos y restricciones necesarias para mantener la consistencia de la información.

La validación de esquema es importante porque evita el registro de documentos incompletos o con datos incorrectos, especialmente en información sensible del negocio como apuestas, montos, estados, usuarios, monedas, transacciones financieras y comprobantes.

Validaciones aplicadas

Colección	Validaciones principales	Objetivo
apuestas	Campos requeridos, tipos de datos, longitud mínima y máxima en campos de texto, valores permitidos para tipo de documento, moneda, estado de apuesta y resultado de selección, monto mínimo, cuota mínima, mínimo una selección y formato de fecha para fecha_apuesta.	Asegurar que cada documento de apuesta conserve la información necesaria para identificar al usuario, el local, la selección realizada, el monto apostado, la moneda utilizada, el estado de la apuesta y la trazabilidad operativa.
movimientos _financieros	Campos requeridos, tipos de datos, valores permitidos para tipo de movimiento, estado, moneda, montos no negativos, referencias mediante usuario_id y apuesta_id, método de pago,	Garantizar que los movimientos económicos se registren con datos consistentes para auditoría, control financiero y relación con las apuestas cuando corresponda.

	comprobante y formato de fecha para fecha_operacion.	
--	---	--

Validación de esquema para la colección apuestas

Esta colección almacena los documentos relacionados con las apuestas deportivas realizadas por los usuarios. La validación asegura que cada apuesta tenga un ticket, usuario, local, selecciones, moneda, montos, estado y fecha de registro correctamente definidos.

```
db.createCollection("apuestas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [
        "id_apuesta",
        "codigo_ticket",
        "usuario",
        "selecciones",
        "moneda",
        "monto_apostado",
        "estado_apuesta",
        "fecha_apuesta"
      ],
    },
    properties: {
      _id: {
        bsonType: "objectId",
        description: "Identificador único del documento."
      },
      id_apuesta: {
        bsonType: "int",
        minimum: 1,
        description: "Identificador interno de la apuesta."
      },
      codigo_ticket: {
        bsonType: "string",
        minLength: 8,
        maxLength: 20,
        description: "Código único del ticket de apuesta."
      },
      usuario: {
```

```

    bsonType: "object",
    required: [
      "id_usuario",
      "nombre_completo",
      "tipo_documento",
      "numero_documento"
    ],
    properties: {
      id_usuario: {
        bsonType: "int",
        minimum: 1,
        description: "Identificador del usuario que realizó
la apuesta."
      },
      nombre_completo: {
        bsonType: "string",
        minLength: 3,
        maxLength: 100,
        description: "Nombre completo del usuario."
      },
      tipo_documento: {
        bsonType: "string",
        enum: ["DNI", "CE", "PASAPORTE"],
        description: "Tipo de documento del usuario."
      },
      numero_documento: {
        bsonType: "string",
        minLength: 8,
        maxLength: 12,
        description: "Número de documento del usuario."
      },
      correo: {
        bsonType: "string",
        maxLength: 120,
        description: "Correo electrónico del usuario."
      }
    }
  },
  local: {
    bsonType: "object",
    properties: {
      id_local: {
        bsonType: "int",
        minimum: 1,
        description: "Identificador del local."
      },
      nombre_local: {
        bsonType: "string",

```

```

        minLength: 3,
        maxLength: 80,
        description: "Nombre del local."
    },
    distrito: {
        bsonType: "string",
        minLength: 3,
        maxLength: 80,
        description: "Distrito donde se ubica el local."
    },
    ciudad: {
        bsonType: "string",
        minLength: 3,
        maxLength: 80,
        description: "Ciudad donde se ubica el local."
    }
},
selecciones: {
    bsonType: "array",
    minItems: 1,
    description: "Lista de selecciones realizadas en la
apuesta.",
    items: {
        bsonType: "object",
        required: [
            "deporte",
            "evento",
            "mercado",
            "opcion",
            "cuota",
            "resultado"
        ],
        properties: {
            deporte: {
                bsonType: "string",
                minLength: 3,
                maxLength: 50,
                description: "Deporte relacionado con la
apuesta."
            },
            evento: {
                bsonType: "string",
                minLength: 5,
                maxLength: 120,
                description: "Evento deportivo seleccionado."
            },
            mercado: {

```

```

        bsonType: "string",
        minLength: 3,
        maxLength: 80,
        description: "Mercado de apuesta seleccionado."
    },
    opcion: {
        bsonType: "string",
        minLength: 2,
        maxLength: 100,
        description: "Opción elegida por el usuario."
    },
    cuota: {
        bsonType: "double",
        minimum: 1,
        maximum: 100,
        description: "Valor de la cuota al momento de
apostar."
    },
    resultado: {
        bsonType: "string",
        enum: ["PENDIENTE", "ACERTADO", "FALLADO",
"ANULADO"],
        description: "Resultado de la selección."
    }
}
},
moneda: {
    bsonType: "object",
    required: [
        "id_moneda",
        "codigo"
    ],
    properties: {
        id_moneda: {
            bsonType: "int",
            minimum: 1,
            description: "Identificador de la moneda."
        },
        codigo: {
            bsonType: "string",
            enum: ["PEN", "USD", "EUR"],
            description: "Código de la moneda."
        },
        simbolo: {
            bsonType: "string",
            enum: ["S/", "$", "€"],
            description: "Símbolo de la moneda."
        }
    }
}
}
}

```

```

    }
  }
},
monto_apostado: {
  bsonType: "double",
  minimum: 0,
  description: "Monto apostado por el usuario."
},
monto_ganancia: {
  bsonType: "double",
  minimum: 0,
  description: "Monto ganado cuando corresponda."
},
estado_apuesta: {
  bsonType: "string",
  enum: ["PENDIENTE", "GANADA", "PERDIDA", "ANULADA",
"CANCELADA"],
  description: "Estado actual de la apuesta."
},
fecha_apuesta: {
  bsonType: "date",
  description: "Fecha y hora de registro de la apuesta."
},
created_at: {
  bsonType: "date",
  description: "Fecha de creación del documento."
},
updated_at: {
  bsonType: "date",
  description: "Fecha de última actualización del
documento."
},
fuente: {
  bsonType: "string",
  maxLength: 50,
  description: "Fuente u origen del registro."
}
}
},
validationLevel: "strict",
validationAction: "error"
});

```

Validación de esquema para la colección movimientos_financieros

Esta colección almacena documentos relacionados con depósitos, retiros, premios, reembolsos, comisiones y ajustes. La validación asegura que cada movimiento financiero tenga identificadores, tipo de operación, estado, moneda, montos y fecha de operación correctamente definidos.

```
db.createCollection("movimientos_financieros", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [
        "id_transaccion",
        "usuario_id",
        "moneda",
        "tipo",
        "estado",
        "monto",
        "fecha_operacion"
      ],
      properties: {
        _id: {
          bsonType: "objectId",
          description: "Identificador único del documento."
        },
        id_transaccion: {
          bsonType: "int",
          minimum: 1,
          description: "Identificador interno de la transacción
financiera."
        },
        usuario_id: {
          bsonType: "int",
          minimum: 1,
          description: "Referencia al usuario relacionado con el
movimiento financiero."
        },
        apuesta_id: {
          bsonType: "int",
          minimum: 1,
          description: "Referencia a la apuesta relacionada con
el movimiento financiero."
        },
        moneda: {
          bsonType: "object",
          required: [
            "codigo"
```

```

    ],
    properties: {
      codigo: {
        bsonType: "string",
        enum: ["PEN", "USD", "EUR"],
        description: "Código de la moneda utilizada."
      },
      simbolo: {
        bsonType: "string",
        enum: ["S/", "$", "€"],
        description: "Símbolo de la moneda."
      }
    }
  },
  tipo: {
    bsonType: "string",
    enum: [
      "APUESTA",
      "DEPOSITO",
      "RETIRO",
      "PREMIO",
      "REEMBOLSO",
      "COMISION",
      "AJUSTE"
    ],
    description: "Tipo de movimiento financiero."
  },
  estado: {
    bsonType: "string",
    enum: ["PENDIENTE", "COMPLETADA", "FALLIDA",
"RECHAZADA"],
    description: "Estado del movimiento financiero."
  },
  monto: {
    bsonType: "double",
    minimum: 0,
    description: "Monto principal del movimiento
financiero."
  },
  saldo_anterior: {
    bsonType: "double",
    minimum: 0,
    description: "Saldo antes de aplicar el movimiento."
  },
  saldo_posterior: {
    bsonType: "double",
    minimum: 0,
    description: "Saldo después de aplicar el movimiento."
  }
}

```

```

    },
    comision_aplicada: {
      bsonType: "double",
      minimum: 0,
      description: "Comisión aplicada al movimiento."
    },
    metodo_pago: {
      bsonType: "string",
      minLength: 3,
      maxLength: 60,
      description: "Método de pago utilizado."
    },
    numero_comprobante: {
      bsonType: "string",
      minLength: 4,
      maxLength: 30,
      description: "Número de comprobante de la operación."
    },
    descripcion: {
      bsonType: "string",
      maxLength: 200,
      description: "Descripción breve del movimiento
financiero."
    },
    fecha_operacion: {
      bsonType: "date",
      description: "Fecha principal de la operación
financiera."
    },
    created_at: {
      bsonType: "date",
      description: "Fecha de creación del documento."
    },
    updated_at: {
      bsonType: "date",
      description: "Fecha de última actualización del
documento."
    }
  }
},
validationLevel: "strict",
validationAction: "error"
});

```

3.4.3. Patrones de modelo de datos

En esta sección se presentan los patrones de modelado de datos aplicados en el diseño no relacional. Estos patrones permiten organizar los documentos de manera más eficiente, evitando una conversión directa del modelo relacional y adaptando la estructura al enfoque documental.

Para el diseño de las colecciones **apuestas** y **movimientos_financieros**, se aplicaron patrones que permiten agrupar información relacionada dentro de un mismo documento, reducir datos redundantes y mantener referencias simples entre documentos cuando la información no necesita repetirse completamente.

Embedded Document Pattern

El Embedded Document Pattern consiste en incluir documentos dentro de otros documentos cuando la información está directamente relacionada y suele consultarse en conjunto. Este patrón permite reducir la necesidad de realizar consultas adicionales, ya que los datos relevantes se encuentran dentro del mismo documento principal.

En el caso de BetCore DB, este patrón se aplica principalmente en la colección apuestas, donde se incluyen subdocumentos como usuario, local, moneda y el arreglo selecciones.

Ejemplo:

```
{
  "id_apuesta": 1,
  "codigo_ticket": "TCK-2024-0001",
  "usuario": {
    "id_usuario": 1,
    "nombre_completo": "Juan Pérez",
```

```

    "tipo_documento":"DNI",
    "numero_documento":"76543210"
  },
  "local":{
    "id_local":1,
    "nombre_local":"ControlBet S.J.L",
    "distrito":"San Juan de Lurigancho"
  },
  "selecciones":[
    {
      "deporte":"Fútbol",
      "evento":"Universitario vs Alianza Lima",
      "mercado":"Ganador del partido",
      "opcion":"Universitario gana",
      "cuota":1.85
    }
  ]
}

```

Este patrón es adecuado porque una apuesta necesita conservar información relacionada con el usuario, el local y las selecciones realizadas. Además, permite responder rápidamente preguntas como qué usuario apostó, a qué evento apostó, qué opción seleccionó y cuánto apostó.

Subset Pattern

El Subset Pattern consiste en almacenar solo una parte de la información de una entidad dentro de un documento, evitando copiar todos sus datos. Este patrón ayuda a reducir la redundancia y mantiene los documentos más livianos.

En BetCore DB, este patrón se aplica en la colección apuestas, especialmente en los subdocumentos usuario, local y moneda. No se almacena toda la información completa del usuario o del local, sino únicamente los campos necesarios para identificar la operación y facilitar la consulta.

Ejemplo:

```
"usuario":{
  "id_usuario":1,
  "nombre_completo":"Juan Pérez",
  "tipo_documento":"DNI",
  "numero_documento":"76543210",
  "correo":"juan.perez@gmail.com"
}
```

En este caso, no se incluyen todos los datos personales del usuario, como dirección completa, fecha de nacimiento u otros datos administrativos. Solo se conservan los datos necesarios para reconocer al usuario dentro de la apuesta. Esto permite mantener trazabilidad sin sobrecargar el documento.

Reference Pattern

El Reference Pattern consiste en relacionar documentos mediante identificadores, sin insertar toda la información de una colección dentro de otra. Este patrón es útil cuando se desea evitar duplicidad y cuando la información relacionada puede identificarse mediante un campo de referencia.

En el diseño de BetCore DB, este patrón se aplica principalmente en la colección movimientos_financieros, donde se utilizan los campos usuario_id y apuesta_id.

Ejemplo:

```
{
  "id_transaccion":1,
  "usuario_id":1,
  "apuesta_id":1,
  "tipo":"APUESTA",
  "estado":"COMPLETADA",
  "monto":50.00
}
```

En este caso, `usuario_id` permite identificar al usuario relacionado con el movimiento financiero, mientras que `apuesta_id` permite reconocer la apuesta vinculada. De esta manera, no se repite toda la información del usuario ni todo el documento de la apuesta dentro de la colección financiera.

Aplicación de patrones por colección

Colección	Colección	Colección
apuestas	Embedded Document Pattern, Subset Pattern	Se embeben datos relacionados como usuario, local, moneda y selecciones porque se consultan junto con la apuesta. Además, se almacena solo un subconjunto de información para evitar documentos demasiado grandes.
movimientos _financieros	Reference Pattern, Subset Pattern	Se utilizan referencias como <code>usuario_id</code> y <code>apuesta_id</code> para evitar repetir información de la apuesta o del usuario. También se mantiene solo la información financiera necesaria para auditoría y control económico.

En resumen, el diseño de BetCore DB aplica patrones de modelado documental para equilibrar la facilidad de consulta con la reducción de redundancia. La colección **apuestas** prioriza la lectura rápida de información operativa y deportiva, mientras que **movimientos_financieros** mantiene una estructura financiera más limpia mediante referencias implícitas.

CAPÍTULO IV: IMPLEMENTACIÓN DE BASE DE DATOS

4.1. Sistema de gestión de base de datos

En esta sección se presenta la elección del DBMS que se utilizará para la solución propuesta por ControlBet, denominada BetCore DB. Esta selección es importante porque el gestor de base de datos debe permitir implementar de manera adecuada el modelo relacional diseñado, garantizando la organización, integridad y consulta eficiente de la información operativa y transaccional de una casa de apuestas deportivas.

4.1.1. Evaluación y elección del sistema de gestión de base de datos relacional

Para seleccionar el sistema de gestión de base de datos relacional más adecuado para BetCore DB, se evaluaron tres alternativas: MySQL, PostgreSQL y SQL Server. La evaluación consideró criterios relacionados con escalabilidad, rendimiento, integridad transaccional, seguridad, compatibilidad con el modelo diseñado, facilidad de implementación y mantenimiento.

Criterio	MySQL	PostgreSQL	SQL Server
Escalabilidad	Adecuado para aplicaciones web y bases de datos relacionales de tamaño medio.	Escalable y apropiado para modelos complejos y consultas avanzadas.	Adecuado para entornos transaccionales y empresariales, con capacidad de crecimiento según las necesidades del negocio.
Rendimiento	Buen rendimiento en operaciones comunes y consultas simples.	Alto rendimiento en consultas complejas y manejo avanzado de datos.	Buen rendimiento en operaciones transaccionales, consultas estructuradas y procesamiento de información financiera.

Integridad transaccional	Soporta transacciones y claves foráneas mediante motores como InnoDB.	Ofrece soporte sólido para transacciones, restricciones e integridad referencial.	Permite manejar transacciones, claves primarias, claves foráneas y restricciones de integridad necesarias para el modelo.
Seguridad	Cuenta con gestión de usuarios, permisos y privilegios.	Permite control de roles, permisos y autenticación.	Ofrece mecanismos de seguridad, control de accesos, permisos y administración de usuarios.
Compatibilidad con el proyecto	Requeriría adaptar parte del script y la sintaxis utilizada.	Requeriría ajustes en sintaxis y configuración del entorno.	Es altamente compatible con el script desarrollado, ya que utiliza sintaxis propia de SQL Server como GO, CONSTRAINT, PRIMARY KEY y FOREIGN KEY.
Facilidad de implementación y mantenimiento	Es accesible, pero no corresponde al entorno principal definido para el proyecto.	Es robusto, aunque requiere mayor adaptación para este caso.	Es la alternativa más conveniente porque permite ejecutar directamente el script en SQL Server Management

			Studio y facilita la administración del modelo relacional.
--	--	--	--

Después de evaluar las alternativas, se selecciona SQL Server como sistema de gestión de base de datos relacional para la implementación de la solución propuesta.

La elección se justifica porque SQL Server se adapta adecuadamente a las necesidades del proyecto, ya que permite representar el modelo relacional diseñado para una casa de apuestas deportivas, garantizando la integridad de la información mediante claves primarias, claves foráneas y restricciones. Esto es fundamental para mantener relaciones consistentes entre entidades como Usuario, Apuesta, DetalleApuesta, Cuota, TransaccionFinanciera, Saldo, HistorialSaldo y Comprobante.

Asimismo, SQL Server resulta conveniente para el manejo de operaciones transaccionales, debido a que el modelo debe registrar movimientos económicos como depósitos, retiros, pagos de premios, ajustes de saldo y transacciones asociadas a apuestas. Estas operaciones requieren consistencia, trazabilidad y control, aspectos que se reflejan en las tablas financieras y sus relaciones dentro del script desarrollado.

4.2.1. Evaluación y elección del sistema de gestión de base de datos no relacional

En esta sección se evalúa y selecciona el sistema de gestión de base de datos no relacional que mejor se adapta a las necesidades de BetCore DB. Para esta parte del proyecto se consideraron los principales tipos de bases de datos NoSQL: clave-valor, familias de columnas, grafos y documentos.

De acuerdo con la naturaleza del proyecto, el enfoque más adecuado es el modelo documental, debido a que permite representar la información mediante documentos flexibles, con estructuras anidadas y arreglos. Este enfoque se adapta correctamente a la gestión de apuestas deportivas, ya que

una apuesta puede contener información relacionada con el usuario, local, moneda, selecciones, evento deportivo, monto apostado, estado y fechas de auditoría.

Evaluación de alternativas NoSQL

Tipo de base de datos NoSQL	Descripción	Evaluación para BetCore DB
Clave-valor	Organiza los datos mediante pares de clave y valor. Es útil para accesos rápidos a datos simples.	No se considera la mejor opción, porque el proyecto requiere representar documentos con estructuras internas como usuario, local, moneda y selecciones.
Familia de columnas	Organiza los datos en columnas agrupadas, útil para grandes volúmenes analíticos.	No es la opción más adecuada, ya que el proyecto necesita manejar documentos con datos relacionados y no únicamente columnas distribuidas.
Grafos	Representa datos mediante nodos y relaciones. Es útil cuando las conexiones entre entidades son el elemento principal.	No es la prioridad del proyecto, porque BetCore DB se enfoca más en registrar operaciones, apuestas y movimientos financieros que en analizar redes complejas de relaciones.
Documental	Organiza la información en documentos flexibles, similares a JSON, permitiendo documentos embebidos y arreglos.	Es la opción más adecuada, porque permite modelar apuestas y movimientos financieros de forma clara, flexible y cercana a las consultas del negocio.

Sistema de gestión seleccionado

El sistema de gestión de base de datos no relacional seleccionado es MongoDB, debido a que trabaja bajo un enfoque documental y permite almacenar información en documentos con estructuras flexibles. Esto resulta adecuado para BetCore DB, ya que las colecciones apuestas y

movimientos_financieros requieren manejar datos anidados, arreglos y referencias implícitas entre documentos.

MongoDB permite representar una apuesta como un documento que contiene subdocumentos como usuario, local y moneda, además de un arreglo de selecciones. Asimismo, permite representar los movimientos financieros en una colección separada, utilizando campos como usuario_id y apuesta_id para mantener una relación lógica sin duplicar información innecesaria.

Justificación de la elección

Se eligió MongoDB por las siguientes razones:

Criterio	Justificación
Modelo documental	Permite representar documentos con estructuras flexibles, adecuadas para apuestas y operaciones financieras.
Documentos embebidos	Facilita incluir datos relacionados dentro de un mismo documento, como usuario, local, moneda y selecciones de una apuesta.
Uso de arreglos	Permite registrar múltiples selecciones dentro de una misma apuesta, lo cual es útil para apuestas simples y combinadas.
Flexibilidad del esquema	Permite adaptar la estructura de los documentos según las necesidades del proyecto, sin depender de una estructura rígida como en el modelo relacional.
Validación de esquema	Permite definir reglas de validación para controlar campos requeridos, tipos de datos, valores permitidos y rangos mínimos o máximos.
Escalabilidad	Su enfoque documental facilita el crecimiento progresivo de la base de datos y la incorporación de nuevos campos en el futuro.

Como evidencia de implementación del sistema de gestión de base de datos no relacional seleccionado, se incluirá una captura de la ejecución de un comando insertOne en MongoDB desde Visual Studio Code. Esta evidencia

permite demostrar que la colección apuestas fue creada correctamente y que acepta documentos que cumplen con la estructura y validaciones establecidas para el proyecto.

```
db.apuestas.insertOne({
  id_apuesta: 1001,
  codigo_ticket: "APU2024070401",
  usuario: {
    id_usuario: 501,
    nombre_completo: "Juan Carlos Mendoza Peña",
    tipo_documento: "DNI",
    numero_documento: "12345678",
    correo: "juan.mendoza@email.com"
  },
  local: {
    id_local: 15,
    nombre_local: "Casa de Apuestas Central",
    distrito: "San Isidro",
    ciudad: "Lima"
  },
  selecciones: [
    {
      deporte: "Fútbol",
      evento: "Perú vs Chile - Eliminatorias 2026",
      mercado: "Ganador del Partido",
      opcion: "Perú",
      cuota: 2.45,
      resultado: "PENDIENTE"
    },
    {
      deporte: "Tenis",
      evento: "Wimbledon - Final Masculina",
      mercado: "Ganador",
      opcion: "Jannik Sinner",
      cuota: 1.95,
      resultado: "ACERTADO"
    },
    {
      deporte: "Baloncesto",
      evento: "NBA - Lakers vs Celtics",
      mercado: "Over/Under Total Puntos",
      opcion: "Over 215",
      cuota: 1.80,
      resultado: "PENDIENTE"
    }
  ]
},
```

```

moneda: {
  id_moneda: 1,
  codigo: "PEN",
  simbolo: "S/"
},
monto_apostado: 150.50,
monto_ganancia: 387.45,
estado_apuesta: "PENDIENTE",
fecha_apuesta: new Date("2024-07-04T14:30:00Z"),
created_at: new Date(),
updated_at: new Date(),
fuente: "Presencial"
})

```

Figura 3

Diagrama entidad-relación físico del sistema

```

{
  _id: ObjectId('6a49703db050fc82c0176854'),
  id_apuesta: NumberInt('1001'),
  codigo_ticket: 'APU2024070401',
  usuario: {
    id_usuario: NumberInt('501'),
    nombre_completo: 'Juan Carlos Mendoza Peña',
    tipo_documento: 'DNI',
    numero_documento: '12345678',
    correo: 'juan.mendoza@email.com'
  },
  local: {
    id_local: NumberInt('15'),
    nombre_local: 'Casa de Apuestas Central',
    distrito: 'San Isidro',
    ciudad: 'Lima'
  },
  selecciones: [
    {
      deporte: 'Fútbol',
      evento: 'Perú vs Chile - Eliminatorias 2026',
      mercado: 'Ganador del Partido',
      opcion: 'Perú',
      cuota: Double('2.45'),
      resultado: 'PENDIENTE'
    },
    {
      deporte: 'Tenis',
      evento: 'Wimbledon - Final Masculina',
      mercado: 'Ganador',
      opcion: 'Jannik Sinner',
      cuota: Double('1.95'),
      resultado: 'ACERTADO'
    },
    {
      deporte: 'Baloncesto',
      evento: 'NBA - Lakers vs Celtics',
      mercado: 'Over/Under Total Puntos',
      opcion: 'Over 215',
      cuota: Double('1.8'),
      resultado: 'PENDIENTE'
    }
  ],
  moneda: {
    id_moneda: NumberInt('1'),
    codigo: 'PEN',
    simbolo: 'S/'
  },
  monto_apostado: Double('150.5'),
  monto_ganancia: Double('387.45'),
  estado_apuesta: 'PENDIENTE',
  fecha_apuesta: ISODate('2024-07-04T14:30:00.000Z'),
  created_at: ISODate('2026-07-04T20:42:37.494Z'),
  updated_at: ISODate('2026-07-04T20:42:37.494Z'),
  fuente: 'Presencial'
}

```

Nota. La figura muestra la ejecución correcta de un comando insertOne en MongoDB desde Visual Studio Code, donde se registra un documento de apuesta con usuario, local, selecciones, moneda, montos, estado y fechas de auditoría. Elaboración propia.

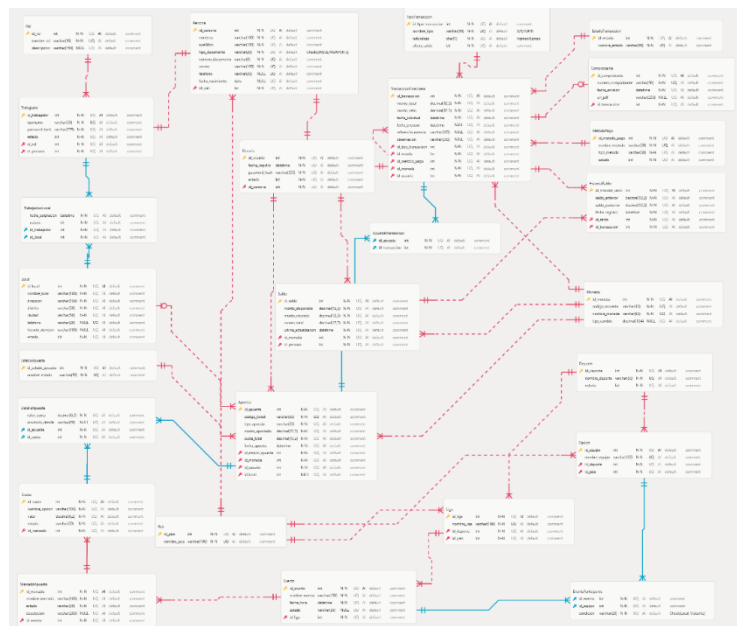
4.2. Diagramas de datos

En esta sección se presentan los diagramas de datos elaborados para representar la estructura física de la base de datos. Para su desarrollo se utilizó Visual Studio Code con la extensión ERD Editor, seleccionando el DBMS MSSQL, debido a que la implementación del modelo relacional se realizará en SQL Server.

4.2.1. Diagrama entidad-relación físico

Figura 3

Diagrama entidad-relación físico del sistema



Nota. Debido a la cantidad de tablas y la densidad de los detalles técnicos del modelo físico, se recomienda visualizar el diagrama en su tamaño original. Haga clic [aquí](#) para acceder a la versión en alta resolución.

4.2.2. Diagrama de documentos

Figura 3

Diagrama de documentos de la base de datos no relacional de BetCore DB

betcore_db			
apuestas			
_id	str	*	
id_apuesta	int	*	
codigo_ticket	str	*	
usuario	obj	*	
id_usuario	int	*	
nombre_completo	str	*	
tipo_documento	str	*	
numero_documento	str	*	
correo	email		
local	obj		
id_local	int		
nombre_local	str		
distrito	str		
ciudad	str		
selecciones	arr	*	
[0]	obj		
deporte	str	*	
evento	str	*	
mercado	str	*	
opcion	str	*	
cuota	num	*	
resultado	str	*	
moneda	obj	*	
id_moneda	int	*	
codigo	str	*	
simbolo	str		
monto_apostado	num	*	
monto_ganancia	num		
estado_apuesta	str	*	
fecha_apuesta	date-time	*	
created_at	date-time		
updated_at	date-time		
fuente	str		
movimientos_financieros			
_id	str	*	
id_transaccion	int	*	
usuario_id	int	*	
apuesta_id	int	*	
moneda	obj	*	
codigo	str	*	
simbolo	str		
tipo	str	*	
estado	str	*	
monto	num	*	
saldo_anterior	num		
saldo_posterior	num		
comision_aplicada	num		
metodo_pago	str		
numero_comprobante	str		
descripcion	str		
fecha_operacion	date-time	*	
created_at	date-time		
updated_at	date-time		

Nota. Elaboración propia en Hackolade. Debido a la densidad de los detalles técnicos del modelo documental, se recomienda visualizar el diagrama en su tamaño original. Haga [clíc](#) aquí para acceder a la versión en alta resolución.

4.3. Scripts de creación de la base de datos

En esta sección se presentan los scripts utilizados para la creación y carga de la base de datos para BetCore DB. Estos scripts permiten implementar las estructuras definidas en el modelo físico, incluyendo tablas, atributos, claves primarias, claves foráneas, restricciones de integridad y registros de prueba necesarios para validar el funcionamiento del diseño.

4.3.1. Scripts de creación y carga de datos de la base de datos relacional

4.3.1.1. Creación de tablas

```
--CREACION DE LA BASE DE DATOS
CREATE DATABASE apuestas_deportivas;
GO
```

```
USE apuestas_deportivas;
```

```

GO

-- 1: CREACIÓN DE TABLAS
CREATE TABLE Pais (
    id_pais      int          NOT NULL IDENTITY(1,1),
    nombre_pais  varchar(100) NOT NULL,
    CONSTRAINT PK_Pais PRIMARY KEY (id_pais),
    CONSTRAINT UQ_nombre_pais UNIQUE (nombre_pais)
)
GO

CREATE TABLE Moneda (
    id_moneda     int          NOT NULL IDENTITY(1,1),
    codigo_moneda varchar(10)   NOT NULL,
    nombre_moneda varchar(50)   NOT NULL,
    tipo_cambio   decimal(10,4),
    CONSTRAINT PK_Moneda PRIMARY KEY (id_moneda),
    CONSTRAINT UQ_Moneda_codigo_moneda UNIQUE (codigo_moneda),
    CONSTRAINT UQ_Moneda_nombre_moneda UNIQUE (nombre_moneda),
    CONSTRAINT CK_Moneda_nombre_moneda CHECK ( nombre_moneda IN ( 'SOL
PERUANO', 'DOLAR AMERICANO', 'EURO' ) ),
    CONSTRAINT CK_Moneda_codigo_moneda CHECK ( codigo_moneda IN ( 'PEN',
'USD', 'EUR' ) ),
    CONSTRAINT CK_Moneda_tipo_cambio CHECK ( tipo_cambio IS NULL OR
tipo_cambio > 0 )
)
GO

CREATE TABLE Deporte (
    id_deporte     int          NOT NULL IDENTITY(1,1),
    nombre_deporte varchar(50)   NOT NULL,
    estado          bit          NOT NULL,
    CONSTRAINT PK_Deporte PRIMARY KEY (id_deporte),
    CONSTRAINT UQ_nombre_deporte UNIQUE (nombre_deporte)
)
GO

CREATE TABLE Rol (
    id_rol         int          NOT NULL IDENTITY(1,1),
    nombre_rol     varchar(50)   NOT NULL,
    descripcion    varchar(150),
    CONSTRAINT PK_Rol PRIMARY KEY (id_rol),
    CONSTRAINT UQ_Rol_nombre_rol UNIQUE (nombre_rol),
    CONSTRAINT CK_Rol_nombre_rol CHECK ( nombre_rol IN ( 'ADMINISTRADOR',
'CAJERO', 'APOSTADOR', 'ANALISTA' ) )
)
GO

CREATE TABLE EstadoApuesta (
    id_estado_apuesta int          NOT NULL IDENTITY(1,1),
    nombre_estado     varchar(50)   NOT NULL,
    CONSTRAINT PK_EstadoApuesta PRIMARY KEY (id_estado_apuesta),
    CONSTRAINT CK_EstadoApuesta_nombre_estado CHECK (nombre_estado IN
('PENDIENTE', 'GANADA', 'PERDIDA', 'ANULADA', 'CANCELADA')),
    CONSTRAINT UQ_EstadoApuesta_nombre_estado UNIQUE (nombre_estado)
)
GO

CREATE TABLE EstadoTransaccion (
    id_estado     int          NOT NULL IDENTITY(1,1),
    nombre_estado varchar(50)   NOT NULL,
    CONSTRAINT PK_EstadoTransaccion PRIMARY KEY (id_estado),

```

```

CONSTRAINT CK_EstadoTransaccion_nombre_estado_transaccion CHECK
(nombre_estado IN ('PENDIENTE', 'COMPLETADA', 'FALLIDA')),
CONSTRAINT UQ_EstadoTransaccion_nombre_estado UNIQUE (nombre_estado)
)
GO

```

```

CREATE TABLE MetodoPago (
    id_metodo_pago int NOT NULL IDENTITY(1,1),
    nombre_metodo varchar(50) NOT NULL,
    tipo_metodo varchar(30) NOT NULL,
    estado bit NOT NULL DEFAULT 1,
    CONSTRAINT PK_MetodoPago PRIMARY KEY (id_metodo_pago),
    CONSTRAINT CK_MetodoPago_nombre_metodo CHECK ( nombre_metodo IN
('TARJETA CREDITO', 'TARJETA DEBITO', 'YAPE', 'PLIN', 'TRANSFERENCIA
BANCARIA', 'PAYPAL', 'EFECTIVO' )),
    CONSTRAINT CK_MetodoPago_tipo_metodo CHECK ( tipo_metodo IN
('DIGITAL', 'BANCARIO', 'EFECTIVO' ) ),
    CONSTRAINT UQ_EstadoApuesta_nombre_metodo UNIQUE (nombre_metodo)
)
GO

```

```

CREATE TABLE TipoTransaccion (
    id_tipo_transaccion int NOT NULL IDENTITY(1,1),
    nombre_tipo varchar(50) NOT NULL,
    naturaleza VARCHAR(30) NOT NULL,
    afecta_saldo bit NOT NULL,
    CONSTRAINT PK_TipoTransaccion PRIMARY KEY (id_tipo_transaccion),
    CONSTRAINT CK_TipoTransaccion_nombre_tipo CHECK ( nombre_tipo IN (
'DEPÓSITO', 'RETIRO', 'APUESTA', 'PREMIO', 'BONIFICACION', 'COMISION',
'DEVOLUCION' )),
    CONSTRAINT CK_TipoTransaccion_naturaleza CHECK ( naturaleza IN
('INGRESO', 'EGRESO' ) ),
    CONSTRAINT UQ_EstadoApuesta_nombre_tipo UNIQUE (nombre_tipo)
)
GO

```

-- 2: PERSONAS, USUARIOS Y LOCALES
-- Gestión de clientes y trabajadores.

```

CREATE TABLE Persona (
    id_persona int NOT NULL IDENTITY(1,1),
    nombres varchar(100) NOT NULL,
    apellidos varchar(100) NOT NULL,
    tipo_documento varchar(25) NOT NULL,
    numero_documento varchar(20) NOT NULL,
    correo varchar(100) NOT NULL,
    telefono varchar(20) ,
    fecha_nacimiento date ,
    id_pais int NOT NULL,
    CONSTRAINT PK_Persona PRIMARY KEY (id_persona),
    CONSTRAINT CK_Persona_tipo_documento CHECK ( tipo_documento IN ('DNI',
'PASAPORTE', 'CARNET DE EXTRANJERIA', 'CC', 'CI')),
    CONSTRAINT UQ_Persona_documento UNIQUE (tipo_documento,
numero_documento),
    CONSTRAINT UQ_Persona_correo UNIQUE (correo),
    CONSTRAINT CK_Persona_correo CHECK (correo LIKE '%@%.%'),
    CONSTRAINT UQ_Persona_telefono UNIQUE (telefono)
)
GO

```

```

CREATE TABLE Usuario (
    id_usuario int NOT NULL IDENTITY(1,1),
    fecha_registro datetime NOT NULL,
    password_hash varchar(255) NOT NULL,

```

```

        estado          bit          NOT NULL,
        id_persona      int          NOT NULL,
        CONSTRAINT PK_Usuario PRIMARY KEY (id_usuario)
    )
GO

CREATE TABLE Trabajador (
    id_trabajador int          NOT NULL IDENTITY(1,1),
    username       varchar(50) NOT NULL,
    password_hash  varchar(255) NOT NULL,
    estado         bit         NOT NULL,
    id_rol         int         NOT NULL,
    id_persona     int         NOT NULL,
    CONSTRAINT PK_Trabajador PRIMARY KEY (id_trabajador)
)
GO

CREATE TABLE Local (
    id_local        int          NOT NULL IDENTITY(1,1),
    nombre_local    varchar(100) NOT NULL,
    direccion       varchar(150) NOT NULL,
    distrito       varchar(50)  NOT NULL,
    ciudad         varchar(50)  NOT NULL,
    telefono       varchar(20)  ,
    horario_atencion varchar(100),
    estado         bit         NOT NULL,
    CONSTRAINT PK_Local PRIMARY KEY (id_local),
    CONSTRAINT UQ_Local_telefono UNIQUE (telefono),
    CONSTRAINT UQ_Local_direccion UNIQUE (direccion)
)
GO

CREATE TABLE TrabajadorxLocal (
    fecha_asignacion datetime NOT NULL,
    estado         bit         NOT NULL,
    id_trabajador  int         NOT NULL,
    id_local       int         NOT NULL,
    CONSTRAINT PK_TrabajadorxLocal PRIMARY KEY (id_trabajador, id_local)
)
GO

-- 3: GESTIÓN DE EVENTOS DEPORTIVOS Y CUOTAS
-- Ligas, Equipos y Eventos a los que se puede apostar.
CREATE TABLE Liga (
    id_liga        int          NOT NULL IDENTITY(1,1),
    nombre_liga    varchar(100) NOT NULL,
    id_deporte     int          NOT NULL,
    id_pais        int          NOT NULL,
    CONSTRAINT PK_Liga PRIMARY KEY (id_liga),
    CONSTRAINT UQ_nombre_liga UNIQUE (nombre_liga)
)
GO

CREATE TABLE Equipo (
    id_equipo      int          NOT NULL IDENTITY(1,1),
    nombre_equipo  varchar(100) NOT NULL,
    id_deporte     int          NOT NULL,
    id_pais        int          NOT NULL,
    CONSTRAINT PK_Equipo PRIMARY KEY (id_equipo),
    CONSTRAINT UQ_nombre_equipo UNIQUE (nombre_equipo)
)
GO

```

```

CREATE TABLE Evento (
    id_evento      int          NOT NULL IDENTITY(1,1),
    nombre_evento  varchar(150) NOT NULL,
    fecha_hora     datetime     NOT NULL,
    estado         varchar(30)  NOT NULL DEFAULT 'PROGRAMADO',
    id_liga        int          NOT NULL,
    CONSTRAINT PK_Evento PRIMARY KEY (id_evento),
    CONSTRAINT CK_Evento_estado_evento CHECK (estado IN ('PROGRAMADO', 'EN
CURSO', 'FINALIZADO', 'SUSPENDIDO', 'CANCELADO', 'POSTERGADO'))
)
GO

CREATE TABLE EventoParticipante (
    id_evento int          NOT NULL,
    id_equipo int          NOT NULL,
    condicion varchar(20) NOT NULL,
    CONSTRAINT PK_EventoParticipante PRIMARY KEY (id_evento, id_equipo),
    CONSTRAINT CK_EventoParticipante_condicion CHECK (condicion IN
('LOCAL', 'VISITANTE'))
)
GO

CREATE TABLE MercadoApuesta (
    id_mercado      int          NOT NULL IDENTITY(1,1),
    nombre_mercado  varchar(100) NOT NULL,
    estado          varchar(20)  NOT NULL,
    descripcion     varchar(200),
    id_evento       int          NOT NULL,
    CONSTRAINT PK_MercadoApuesta PRIMARY KEY (id_mercado),
    CONSTRAINT CK_MercadoApuesta_estado CHECK (estado IN ('ACTIVO',
'SUSPENDIDO', 'CERRADO'))
)
GO

CREATE TABLE Cuota (
    id_cuota      int          NOT NULL IDENTITY(1,1),
    nombre_opcion varchar(100) NOT NULL,
    valor         decimal(6,2) NOT NULL,
    estado        varchar(20)  NOT NULL,
    id_mercado    int          NOT NULL,
    CONSTRAINT PK_Cuota PRIMARY KEY (id_cuota),
    CONSTRAINT CK_Cuota_estado_cuota CHECK (estado IN ('ACTIVA',
'SUSPENDIDA', 'CERRADA')),
    CONSTRAINT CK_Cuota_valor_cuota CHECK (valor >= 1.01)
)
GO

-- 4: FINANZAS Y APUESTAS
-- Registro de saldos, transacciones, tickets y comprobantes.
CREATE TABLE Saldo (
    id_saldo      int          NOT NULL IDENTITY(1,1),
    monto_disponible decimal(12,2) NOT NULL,
    monto_retenido decimal(12,2) NOT NULL,
    monto_total   decimal(12,2) NOT NULL,
    ultima_actualizacion datetime NOT NULL,
    id_moneda     int          NOT NULL,
    id_usuario    int          NOT NULL,

    CONSTRAINT PK_Saldo PRIMARY KEY (id_saldo),
    CONSTRAINT CK_Saldo_monto_disponible CHECK (monto_disponible >= 0),
    CONSTRAINT CK_Saldo_monto_retenido   CHECK (monto_retenido >= 0),
    CONSTRAINT CK_Saldo_monto_total     CHECK (monto_total >= 0)
)

```

GO

```

CREATE TABLE TransaccionFinanciera (
    id_transaccion      int          NOT NULL IDENTITY(1,1),
    monto_total         decimal(12,2) NOT NULL,
    monto_netto         decimal(12,2) NOT NULL,
    fecha_solicitud     datetime     NOT NULL,
    fecha_proceso       datetime     ,
    referencia_externa  varchar(100) ,
    observacion         varchar(200) ,
    id_tipo_transaccion int          NOT NULL,
    id_estado           int          NOT NULL,
    id_metodo_pago      int          NOT NULL,
    id_moneda           int          NOT NULL,
    id_usuario          int          NOT NULL,
    CONSTRAINT PK_TransaccionFinanciera PRIMARY KEY (id_transaccion),
    CONSTRAINT CK_TransaccionFinanciera_monto_total CHECK (monto_total >
0),
    CONSTRAINT CK_TransaccionFinanciera_monto_netto CHECK (monto_netto > 0)
)
GO

```

```

CREATE TABLE HistorialSaldo (
    id_historial_saldo int          NOT NULL IDENTITY(1,1),
    saldo_anterior     decimal(12,2) NOT NULL,
    saldo_posterior    decimal(12,2) NOT NULL,
    fecha_registro     datetime     NOT NULL,
    id_saldo           int          NOT NULL,
    id_transaccion     int          NOT NULL,
    CONSTRAINT PK_HistorialSaldo PRIMARY KEY (id_historial_saldo),
    CONSTRAINT CK_HistorialSaldo_saldo_anterior CHECK (saldo_anterior
>=0),
    CONSTRAINT CK_HistorialSaldo_saldo_posterior CHECK (saldo_posterior
>=0)
)
GO

```

```

CREATE TABLE Comprobante (
    id_comprobante      int          NOT NULL IDENTITY(1,1),
    numero_comprobante  varchar(50)  NOT NULL,
    fecha_emision       datetime     NOT NULL,
    url_pdf             varchar(255),
    id_transaccion      int          NOT NULL,
    CONSTRAINT PK_Comprobante PRIMARY KEY (id_comprobante),
    CONSTRAINT UQ_numero_comprobante UNIQUE (numero_comprobante)
)
GO

```

```

CREATE TABLE Apuesta (
    id_apuesta          int IDENTITY(1,1) NOT NULL,
    codigo_ticket        varchar(50)  NOT NULL,
    tipo_apuesta        varchar(20)   NOT NULL,
    monto_apostado      decimal(12,2) NOT NULL,
    cuota_total         decimal(10,2) NOT NULL,
    fecha_apuesta       datetime     NOT NULL,
    id_estado_apuesta   int          NOT NULL,
    id_moneda           int          NOT NULL,
    id_usuario          int          NOT NULL,
    id_local            int          ,

    CONSTRAINT PK_Apuesta PRIMARY KEY (id_apuesta),
    CONSTRAINT UQ_Apuesta_codigo_ticket UNIQUE (codigo_ticket),
    CONSTRAINT CK_Apuesta_monto_apostado CHECK (monto_apostado >= 1),

```

```

        CONSTRAINT CK_Apuesta_cuota_total CHECK (cuota_total >= 1.01),
        CONSTRAINT CK_Apuesta_tipo_apuesta CHECK (tipo_apuesta IN ('SIMPLE',
'MULTIPLE', 'EN VIVO'))
    )
GO

CREATE TABLE DetalleAposta (
    valor_cuota          decimal(6,2) NOT NULL,
    resultado_detalle    varchar(20) ,
    id_apuesta           int           NOT NULL,
    id_cuota             int           NOT NULL,
    CONSTRAINT PK_DetalleAposta PRIMARY KEY (id_apuesta, id_cuota),
    CONSTRAINT CK_DetalleAposta_resultado_detalle CHECK (
resultado_detalle IN ('GANADO', 'PERDIDO', 'ANULADO', 'PENDIENTE') )
)
GO

CREATE TABLE ApuestaTransaccion (
    id_apuesta           int NOT NULL,
    id_transaccion       int NOT NULL,
    CONSTRAINT PK_ApuestaTransaccion PRIMARY KEY (id_apuesta,
id_transaccion)
)
GO

-- CREACIÓN DE RELACIONES (LLAVES FORÁNEAS)
-- Se ejecutan al final para evitar errores de dependencias.
ALTER TABLE Persona ADD CONSTRAINT FK_Pais_TO_Persona FOREIGN KEY
(id_pais) REFERENCES Pais (id_pais);
GO
ALTER TABLE Usuario ADD CONSTRAINT FK_Persona_TO_Usuario FOREIGN KEY
(id_persona) REFERENCES Persona (id_persona);
GO
ALTER TABLE Trabajador ADD CONSTRAINT FK_Rol_TO_Trabajador FOREIGN KEY
(id_rol) REFERENCES Rol (id_rol);
GO
ALTER TABLE Trabajador ADD CONSTRAINT FK_Persona_TO_Trabajador FOREIGN
KEY (id_persona) REFERENCES Persona (id_persona);
GO
ALTER TABLE TrabajadorxLocal ADD CONSTRAINT
FK_Trabajador_TO_TrabajadorxLocal FOREIGN KEY (id_trabajador) REFERENCES
Trabajador (id_trabajador);
GO
ALTER TABLE TrabajadorxLocal ADD CONSTRAINT FK_Local_TO_TrabajadorxLocal
FOREIGN KEY (id_local) REFERENCES Local (id_local);
GO
ALTER TABLE Liga ADD CONSTRAINT FK_Deporte_TO_Liga FOREIGN KEY
(id_deporte) REFERENCES Deporte (id_deporte);
GO
ALTER TABLE Liga ADD CONSTRAINT FK_Pais_TO_Liga FOREIGN KEY (id_pais)
REFERENCES Pais (id_pais);
GO
ALTER TABLE Equipo ADD CONSTRAINT FK_Deporte_TO_Equipo FOREIGN KEY
(id_deporte) REFERENCES Deporte (id_deporte);
GO
ALTER TABLE Equipo ADD CONSTRAINT FK_Pais_TO_Equipo FOREIGN KEY
(id_pais) REFERENCES Pais (id_pais);
GO
ALTER TABLE Evento ADD CONSTRAINT FK_Liga_TO_Evento FOREIGN KEY
(id_liga) REFERENCES Liga (id_liga);
GO

```

```

ALTER TABLE EventoParticipante ADD CONSTRAINT
FK_Evento_TO_EventoParticipante FOREIGN KEY (id_evento) REFERENCES
Evento (id_evento);
GO
ALTER TABLE EventoParticipante ADD CONSTRAINT
FK_Equipo_TO_EventoParticipante FOREIGN KEY (id_equipo) REFERENCES
Equipo (id_equipo);
GO
ALTER TABLE MercadoApuesta ADD CONSTRAINT FK_Evento_TO_MercadoApuesta
FOREIGN KEY (id_evento) REFERENCES Evento (id_evento);
GO
ALTER TABLE Cuota ADD CONSTRAINT FK_MercadoApuesta_TO_Cuota FOREIGN KEY
(id_mercado) REFERENCES MercadoApuesta (id_mercado);
GO
ALTER TABLE Saldo ADD CONSTRAINT FK_Moneda_TO_Saldo FOREIGN KEY
(id_moneda) REFERENCES Moneda (id_moneda);
GO
ALTER TABLE Saldo ADD CONSTRAINT FK_Usuario_TO_Saldo FOREIGN KEY
(id_usuario) REFERENCES Usuario (id_usuario);
GO
ALTER TABLE TransaccionFinanciera ADD CONSTRAINT
FK_TipoTransaccion_TO_TransaccionFinanciera FOREIGN KEY
(id_tipo_transaccion) REFERENCES TipoTransaccion (id_tipo_transaccion);
GO
ALTER TABLE TransaccionFinanciera ADD CONSTRAINT
FK_EstadoTransaccion_TO_TransaccionFinanciera FOREIGN KEY (id_estado)
REFERENCES EstadoTransaccion (id_estado);
GO
ALTER TABLE TransaccionFinanciera ADD CONSTRAINT
FK_MetodoPago_TO_TransaccionFinanciera FOREIGN KEY (id_metodo_pago)
REFERENCES MetodoPago (id_metodo_pago);
GO
ALTER TABLE TransaccionFinanciera ADD CONSTRAINT
FK_Moneda_TO_TransaccionFinanciera FOREIGN KEY (id_moneda) REFERENCES
Moneda (id_moneda);
GO
ALTER TABLE TransaccionFinanciera ADD CONSTRAINT
FK_Usuario_TO_TransaccionFinanciera FOREIGN KEY (id_usuario) REFERENCES
Usuario (id_usuario);
GO
ALTER TABLE HistorialSaldo ADD CONSTRAINT FK_Saldo_TO_HistorialSaldo
FOREIGN KEY (id_saldo) REFERENCES Saldo (id_saldo);
GO
ALTER TABLE HistorialSaldo ADD CONSTRAINT
FK_TransaccionFinanciera_TO_HistorialSaldo FOREIGN KEY (id_transaccion)
REFERENCES TransaccionFinanciera (id_transaccion);
GO
ALTER TABLE Comprobante ADD CONSTRAINT
FK_TransaccionFinanciera_TO_Comprobante FOREIGN KEY (id_transaccion)
REFERENCES TransaccionFinanciera (id_transaccion);
GO
ALTER TABLE Apuesta ADD CONSTRAINT FK_EstadoApuesta_TO_Apuesta FOREIGN
KEY (id_estado_apuesta) REFERENCES EstadoApuesta (id_estado_apuesta);
GO
ALTER TABLE Apuesta ADD CONSTRAINT FK_Moneda_TO_Apuesta FOREIGN KEY
(id_moneda) REFERENCES Moneda (id_moneda);
GO
ALTER TABLE Apuesta ADD CONSTRAINT FK_Usuario_TO_Apuesta FOREIGN KEY
(id_usuario) REFERENCES Usuario (id_usuario);
GO
ALTER TABLE Apuesta ADD CONSTRAINT FK_Local_TO_Apuesta FOREIGN KEY
(id_local) REFERENCES Local (id_local);

```

```

GO
ALTER TABLE DetalleApuesta ADD CONSTRAINT FK_Apuesta_TO_DetalleApuesta
FOREIGN KEY (id_apuesta) REFERENCES Apuesta (id_apuesta);
GO
ALTER TABLE DetalleApuesta ADD CONSTRAINT FK_Cuota_TO_DetalleApuesta
FOREIGN KEY (id_cuota) REFERENCES Cuota (id_cuota);
GO
ALTER TABLE ApuestaTransaccion ADD CONSTRAINT
FK_Apuesta_TO_ApuestaTransaccion FOREIGN KEY (id_apuesta) REFERENCES
Apuesta (id_apuesta);
GO
ALTER TABLE ApuestaTransaccion ADD CONSTRAINT
FK_TransaccionFinanciera_TO_ApuestaTransaccion FOREIGN KEY
(id_transaccion) REFERENCES TransaccionFinanciera (id_transaccion);
GO

```

4.3.1.2. Creación de datos

```

-- INSERCCION DE DATOS
USE apuestas_deportivas
GO

-- 1. TABLA PAIS
INSERT INTO Pais (nombre_pais) VALUES
('Perú'),
('Argentina'),
('España'),
('Estados Unidos'),
('Brasil'),
('Inglaterra'),
('Chile'),
('Colombia'),
('México'),
('Uruguay'),
('Paraguay'),
('Bolivia'),
('Italia'),
('Francia'),
('Alemania');
GO

-- 2. TABLA MONEDA
INSERT INTO Moneda (codigo_moneda, nombre_moneda, tipo_cambio) VALUES
('PEN', 'SOL PERUANO', NULL),
('USD', 'DOLAR AMERICANO', 3.7500),
('EUR', 'EURO', 4.1000)
GO

-- 3. TABLA DEPORTE
INSERT INTO Deporte (nombre_deporte, estado) VALUES
('Fútbol', 1),
('Básquet', 1),
('Vóley', 1)
GO

-- 4. TABLA ROL
INSERT INTO Rol (nombre_rol, descripcion) VALUES
('ADMINISTRADOR', 'Gestiona el sistema completo, usuarios y
configuraciones'),
('CAJERO', 'Procesa depósitos, retiros y pagos en ventanilla'),
('APOSTADOR', 'Usuario final que realiza las apuestas'),
('ANALISTA', 'Define cuotas, mercados y eventos deportivos')

```

GO

-- 5. TABLA EstadoApuesta

INSERT INTO EstadoApuesta (nombre_estado) VALUES

('PENDIENTE'),
('GANADA'),
('PERDIDA'),
('ANULADA'),
('CANCELADA')

GO

-- 6. TABLA EstadoTransaccion

INSERT INTO EstadoTransaccion (nombre_estado) VALUES

('PENDIENTE'),
('COMPLETADA'),
('FALLIDA')

GO

-- 7. TABLA MetodoPago

INSERT INTO MetodoPago (nombre_metodo, tipo_metodo, estado) VALUES

('TARJETA CREDITO', 'BANCARIO', 1),
('TARJETA DEBITO', 'BANCARIO', 1),
('YAPE', 'DIGITAL', 1),
('PLIN', 'DIGITAL', 1),
('TRANSFERENCIA BANCARIA', 'BANCARIO', 1),
('PAYPAL', 'DIGITAL', 1),
('EFECTIVO', 'EFECTIVO', 1)

GO

-- 8. TABLA TipoTransaccion

INSERT INTO TipoTransaccion (nombre_tipo, naturaleza, afecta_saldo)
VALUES

('DEPOSITO', 'INGRESO', 1),
('PREMIO', 'INGRESO', 1),
('BONIFICACION', 'INGRESO', 1),
('DEVOLUCION', 'INGRESO', 1),
('RETIRO', 'EGRESO', 1),
('APUESTA', 'EGRESO', 1),
('COMISION', 'EGRESO', 1)

GO

-- 9. TABLA Persona

INSERT INTO Persona (nombres, apellidos, tipo_documento,
numero_documento, correo, telefono, fecha_nacimiento, id_pais) VALUES

('Byron', 'Tirado Quispe', 'DNI', '74521369', 'byron.tirado@gmail.com',
'987111222', '1995-03-20', 1),
('Jefferson', 'Pérez Pino', 'DNI', '70011223',
'jefferson.perez@gmail.com', '987333444', '1990-08-15', 1),
('Luis Fernando', 'Ramírez Torres', 'DNI', '71452389',
'luis.ramirez@gmail.com', '987555666', '1997-11-02', 1),
('María Elena', 'Quispe Salinas', 'DNI', '73219456',
'maria.quispe@gmail.com', '987888999', '1993-04-08', 1),
('Gamaniel', 'Cruz Valerio', 'DNI', '72345678',
'gamaniel.cruz@gmail.com', '987654321', '1998-05-12', 1),
('Paula', 'Chávez Pino', 'DNI', '77889900', 'paula.chavez@gmail.com',
'986111000', '2003-06-25', 1),
('Giovanni', 'Gallegos De La Cruz', 'DNI', '75432198',
'giovanni.gallegos@gmail.com', '986222111', '2001-09-30', 1),
('Brayan', 'Huerta Cárdenas', 'DNI', '78901234',
'brayan.huerta@gmail.com', '986333222', '2002-12-05', 1),
('Fabricio', 'Rivera Rupay', 'DNI', '79012345',
'fabricio.rivera@gmail.com', '986444333', '2001-04-18', 1),

```
( 'Carlos', 'Mendoza Vega', 'DNI', '76543299',
'carlos.mendoza@gmail.com', '985111111', '1996-02-14', 1),
( 'Andrea', 'Soto Pinedo', 'DNI', '71234588', 'andrea.soto@gmail.com',
'985222222', '1994-07-23', 1),
( 'Diego', 'Vargas Ríos', 'DNI', '73456722', 'diego.vargas@gmail.com',
'985333333', '1989-10-11', 1),
( 'Sofía', 'Castillo Mora', 'DNI', '74567833',
'sofia.castillo@gmail.com', '985444444', '1998-12-30', 1),
( 'Marco', 'Sánchez Lara', 'DNI', '75678244', 'marco.sanchez@gmail.com',
'985555111', '1991-05-08', 1),
( 'Valeria', 'Rojas Núñez', 'CARNET DE EXTRANJERIA', 'CE789456123',
'valeria.rojas@gmail.com', '+57312456789', '1996-09-17', 8),
( 'Joaquín', 'Flores Bautista', 'DNI', '77890466',
'joaquin.flores@gmail.com', '985777777', '1993-01-22', 1),
( 'Camila', 'Espinoza Vela', 'DNI', '78901577',
'camila.espinoza@gmail.com', '985888888', '1995-11-04', 1),
( 'Lionel', 'Suárez Martínez', 'PASAPORTE', 'AR123456',
'lionel.suarez@gmail.com', '+54911234567', '1992-07-22', 2);
```

```
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
```

```
-- 10. TABLA Trabajador
```

```
INSERT INTO Trabajador (username, password_hash, estado, id_rol,
id_persona) VALUES
```

```
( 'byron.tirado', 'hashed_byron123', 1, 2, 1),
( 'jefferson.perez', 'hashed_jefferson123', 1, 1, 2),
( 'luis.ramirez', 'hashed_luis123', 1, 4, 3),
( 'maria.quispe', 'hashed_maria123', 1, 2, 4),
( 'gamaniel.cruz', 'hashed_gamaniel123', 1, 4, 5),
( 'paula.chavez', 'hashed_paula123', 1, 2, 6),
( 'giovanni.gallegos', 'hashed_giovanni123', 1, 4, 7),
( 'brayan.huerta', 'hashed_brayan123', 1, 2, 8),
( 'fabricio.rivera', 'hashed_fabricio123', 1, 1, 9),
( 'carlos.mendoza', 'hashed_carlos123', 1, 4, 10),
( 'andrea.soto', 'hashed_andrea123', 1, 2, 11),
( 'diego.vargas', 'hashed_diego123', 1, 4, 12),
( 'sofia.castillo', 'hashed_sofia123', 1, 2, 13),
( 'marco.sanchez', 'hashed_marco123', 1, 1, 14),
( 'valeria.rojas', 'hashed_valeria123', 1, 2, 15),
( 'joaquin.flores', 'hashed_joaquin123', 1, 4, 16),
( 'camila.espinoza', 'hashed_camila123', 1, 2, 17),
( 'andres.quispe', 'hashed_andres123', 1, 4, 21),
( 'patricia.morales', 'hashed_patricia123', 1, 2, 22),
( 'luis.torres', 'hashed_luist123', 1, 1, 23)
```

```
GO
```

```
UPDATE Trabajador
```

```
SET password_hash = CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', username
+ '123'), 2);
```

```
GO
```

```
-- 11. TABLA Usuario
```

```
INSERT INTO Usuario (fecha_registro, password_hash, estado, id_persona)
VALUES
```

```
( '2023-01-15', 'hashed_pwd_18', 1, 18),
( '2023-02-20', 'hashed_pwd_19', 1, 19),
( '2023-03-10', 'hashed_pwd_20', 1, 20),
( '2022-11-05', 'hashed_pwd_24', 1, 24),
( '2022-08-17', 'hashed_pwd_25', 1, 25),
( '2023-04-22', 'hashed_pwd_26', 1, 26),
( '2021-12-30', 'hashed_pwd_27', 1, 27),
( '2022-06-14', 'hashed_pwd_28', 1, 28),
```

```
( '2023-05-08', 'hashed_pwd_29', 1, 29),
( '2022-09-19', 'hashed_pwd_30', 1, 30);
```

```
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
```

-- 12. Tabla Local

```
INSERT INTO Local (nombre_local, direccion, distrito, ciudad, telefono,
horario_atencion, estado) VALUES
( 'Apuesta Ganadora - Miraflores', 'Av. Larco 345',
'Miraflores', 'Lima', '014456789', 'Lun-Dom 9:00-22:00', 1),
( 'Apuesta Ganadora - San Isidro', 'Av. Conquistadores 567', 'San
Isidro', 'Lima', '014567890', 'Lun-Dom 9:00-22:00', 1),
( 'Apuesta Ganadora - Surco', 'Av. Primavera 890',
'Santiago de Surco', 'Lima', '014678901', 'Lun-Dom 9:00-23:00', 1),
( 'Apuesta Ganadora - San Borja', 'Av. San Luis 234', 'San
Borja', 'Lima', '014789012', 'Lun-Dom 9:00-22:00', 1),
( 'Apuesta Ganadora - La Molina', 'Av. La Molina 456', 'La
Molina', 'Lima', '014890123', 'Lun-Dom 10:00-22:00', 1),
( 'Apuesta Ganadora - Callao', 'Av. Sáenz Peña 123',
'Callao', 'Callao', '015123456', 'Lun-Dom 9:00-22:00', 1),
( 'Apuesta Ganadora - Trujillo', 'Jr. Pizarro 678',
'Trujillo', 'Trujillo', '044234567', 'Lun-Dom 9:00-21:00', 1),
( 'Apuesta Ganadora - Arequipa', 'Av. Ejército 345',
'Cayma', 'Arequipa', '054345678', 'Lun-Dom 9:00-21:00', 1),
( 'Apuesta Ganadora - Cusco', 'Av. El Sol 789',
'Cusco', 'Cusco', '084456789', 'Lun-Dom 9:00-21:00', 1),
( 'Apuesta Ganadora - Piura', 'Av. Grau 456',
'Piura', 'Piura', '073567890', 'Lun-Dom 9:00-21:00', 1)
GO
```

-- 13. TABLA TrabajadorxLocal

```
INSERT INTO TrabajadorxLocal (fecha_asignacion, estado, id_trabajador,
id_local) VALUES
( '2023-01-10', 1, 1, 1),
( '2023-01-10', 1, 2, 1),
( '2023-01-15', 1, 3, 2),
( '2023-01-15', 1, 4, 2),
( '2023-02-01', 1, 5, 3),
( '2023-02-01', 1, 6, 3),
( '2023-02-10', 1, 7, 4),
( '2023-02-10', 1, 8, 4),
( '2023-03-01', 1, 9, 5),
( '2023-03-01', 1, 10, 5),
( '2023-03-15', 1, 11, 6),
( '2023-03-15', 1, 12, 6),
( '2023-04-01', 1, 13, 7),
( '2023-04-01', 1, 14, 7),
( '2023-04-15', 1, 15, 8),
( '2023-04-15', 1, 16, 8),
( '2023-05-01', 1, 17, 9),
( '2023-05-01', 1, 18, 9),
( '2023-05-15', 1, 19, 10),
( '2023-05-15', 1, 20, 10)
GO
```

-- 14. TABLA Liga

```
INSERT INTO Liga (nombre_liga, id_deporte, id_pais) VALUES
-- FÚTBOL (id_deporte = 1)
( 'Liga 1 Perú', 1, 1),
( 'Premier League', 1, 6),
( 'La Liga', 1, 3),
( 'Serie A', 1, 13),
```

```

('Bundesliga', 1, 15),
('Ligue 1', 1, 14),
('Liga Profesional Argentina', 1, 2),
('Primera División Chile', 1, 7),
('Liga BetPlay Colombia', 1, 8),
('Liga MX', 1, 9),
('Brasileirao Serie A', 1, 5),
('UEFA Champions League', 1, 3),
('Copa Libertadores', 1, 2),
('Eredivisie', 1, 4),
('Primeira Liga Portugal', 1, 3),

-- BASKET (id_deporte = 2)
('NBA', 2, 4),
('Liga ACB España', 2, 3),
('EuroLeague', 2, 13),
('NBB Brasil', 2, 5),
('Liga Nacional Argentina', 2, 2),
('BSL Turquía', 2, 13),
('LNB Perú', 2, 1),
('Lega Basket Italia', 2, 13),
('Pro A Francia', 2, 14),
('BBL Alemania', 2, 15),

-- VÓLEY (id_deporte = 3)
('Liga Nacional Vóley Perú', 3, 1),
('Superliga Femenina España', 3, 3),
('Serie A1 Italia', 3, 13),
('Ligue A Francia', 3, 14),
('Bundesliga Vóley', 3, 15)
GO

-- 15. Tabla Equipo
INSERT INTO Equipo (nombre_equipo, id_deporte, id_pais) VALUES
-- FÚTBOL PERÚ (id_deporte=1, id_pais=1)
('Universitario de Deportes', 1, 1),
('Alianza Lima', 1, 1),
('Sporting Cristal', 1, 1),
('FBC Melgar', 1, 1),
('Cesar Vallejo', 1, 1),

-- FÚTBOL INGLATERRA (id_deporte=1, id_pais=6)
('Manchester City', 1, 6),
('Liverpool', 1, 6);

-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 16. TABLA Evento
INSERT INTO Evento (nombre_evento, fecha_hora, estado, id_liga) VALUES

-- LIGA 1 PERÚ (id_liga=1)
('Universitario vs Alianza Lima', '2024-03-15 15:00:00',
'FINALIZADO', 1),
('Sporting Cristal vs FBC Melgar', '2024-03-22 20:00:00',
'FINALIZADO', 1),
('Alianza Lima vs Cesar Vallejo', '2024-04-05 18:00:00',
'FINALIZADO', 1),
('FBC Melgar vs Universitario', '2024-04-19 15:30:00',
'FINALIZADO', 1),
('Cesar Vallejo vs Sporting Cristal', '2024-05-03 20:00:00',
'FINALIZADO', 1),

```

```

('Universitario vs Sporting Cristal', '2024-06-15 15:00:00',
'PROGRAMADO', 1),
('Alianza Lima vs FBC Melgar', '2024-06-22 20:00:00',
'PROGRAMADO', 1);
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
-- 17. TABLA EventoParticipante
INSERT INTO EventoParticipante (id_evento, id_equipo, condicion) VALUES

-- LIGA 1 PERÚ
-- Evento 1: Universitario vs Alianza Lima
(1, 1, 'LOCAL'),
(1, 2, 'VISITANTE'),
-- Evento 2: Sporting Cristal vs FBC Melgar
(2, 3, 'LOCAL'),
(2, 4, 'VISITANTE'),
-- Evento 3: Alianza Lima vs Cesar Vallejo
(3, 2, 'LOCAL'),
(3, 5, 'VISITANTE');
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 18. TABLA MercadoApuesta
INSERT INTO MercadoApuesta (nombre_mercado, estado, descripcion,
id_evento) VALUES

-- EVENTO 1: Universitario vs Alianza Lima (FINALIZADO)
('Ganador del Partido', 'CERRADO', 'Apuesta al equipo ganador del
partido', 1),
('Total de Goles', 'CERRADO', 'Apuesta sobre el total de goles
en el partido', 1),
('Ambos Equipos Anotan', 'CERRADO', 'Apuesta si ambos equipos marcan
al menos un gol', 1);
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 19. TABLA Cuota
INSERT INTO Cuota (nombre_opcion, valor, estado, id_mercado) VALUES

-- MERCADO 1: Ganador del Partido - Evento 1 (Universitario vs Alianza
Lima) CERRADO
('Universitario', 2.10, 'CERRADA', 1),
('Empate', 3.20, 'CERRADA', 1),
('Alianza Lima', 3.50, 'CERRADA', 1);
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 20. TABLA Saldo
INSERT INTO Saldo (monto_disponible, monto_retenido, monto_total,
ultima_actualizacion, id_moneda, id_usuario) VALUES

-- SALDOS EN PEN (id_moneda=1)
(500.00, 0.00, 500.00, '2024-05-01 10:00:00', 1, 1),
(1200.50, 50.00, 1250.50, '2024-05-02 11:30:00', 1, 2),
(350.00, 0.00, 350.00, '2024-05-03 09:15:00', 1, 3),
(2500.00, 100.00, 2600.00, '2024-05-04 14:20:00', 1, 4),
(750.00, 0.00, 750.00, '2024-05-05 16:45:00', 1, 5);
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 21. TABLA TransaccionFinanciera

```

```
INSERT INTO TransaccionFinanciera (monto_total, monto_netto,
fecha_solicitud, fecha_proceso, referencia_externa, observacion,
id_tipo_transaccion, id_estado, id_metodo_pago, id_moneda, id_usuario)
VALUES
```

```
-- DEPÓSITOS COMPLETADOS (id_tipo=1, id_estado=2)
(500.00, 500.00, '2024-01-05 10:00:00', '2024-01-05 10:05:00', 'DEP-
001', 'Depósito inicial', 1, 2, 3, 1, 1),
(1250.00, 1250.00, '2024-01-06 11:00:00', '2024-01-06 11:03:00', 'DEP-
002', 'Depósito inicial', 1, 2, 4, 1, 2),
(350.00, 350.00, '2024-01-07 09:00:00', '2024-01-07 09:02:00', 'DEP-
003', 'Depósito inicial', 1, 2, 7, 1, 3),
(2600.00, 2600.00, '2024-01-08 14:00:00', '2024-01-08 14:04:00', 'DEP-
004', 'Depósito inicial', 1, 2, 1, 1, 4);
```

```
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
```

```
-- 22. Tabla HistorialSaldo -----
```

```
INSERT INTO HistorialSaldo (saldo_anterior, saldo_posterior,
fecha_registro, id_saldo, id_transaccion) VALUES
```

```
-- DEPÓSITOS INICIALES (saldo_anterior = 0 porque son primeros
movimientos)
(0.00, 500.00, '2024-01-05 10:05:00', 1, 1),
(0.00, 1250.50, '2024-01-06 11:03:00', 2, 2),
(0.00, 350.00, '2024-01-07 09:02:00', 3, 3),
(0.00, 2600.00, '2024-01-08 14:04:00', 4, 4);
```

```
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
```

```
-- 23. Tabla Comprobante -----
```

```
INSERT INTO Comprobante (numero_comprobante, fecha_emision, url_pdf,
id_transaccion) VALUES
```

```
-- DEPÓSITOS (id_transaccion 1-30)
('COMP-DEP-001', '2024-01-05 10:05:00',
'https://docs.apuestaganadora.com/comprobantes/DEP-001.pdf', 1),
('COMP-DEP-002', '2024-01-06 11:03:00',
'https://docs.apuestaganadora.com/comprobantes/DEP-002.pdf', 2),
('COMP-DEP-003', '2024-01-07 09:02:00',
'https://docs.apuestaganadora.com/comprobantes/DEP-003.pdf', 3),
('COMP-DEP-004', '2024-01-08 14:04:00',
'https://docs.apuestaganadora.com/comprobantes/DEP-004.pdf', 4),
('COMP-DEP-005', '2024-01-09 16:02:00',
'https://docs.apuestaganadora.com/comprobantes/DEP-005.pdf', 5);
```

```
-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO
```

```
-- 24. TABLA Apuesta
```

```
INSERT INTO Apuesta
(codigo_ticket, tipo_apuesta, monto_apostado, cuota_total,
fecha_apuesta, id_estado_apuesta, id_moneda, id_usuario, id_local)
VALUES
```

```
-- APUESTAS YA RELACIONADAS CON TRANSACCIONES APU-001 A APU-010
('TCK-APU-001', 'EN VIVO', 50.00, 2.10, '2024-03-15 15:30:00', 2, 1,
1, 1),
('TCK-APU-002', 'EN VIVO', 100.00, 1.90, '2024-03-22 20:30:00', 3, 1,
4, 2),
```

```

('TCK-APU-003', 'EN VIVO', 75.00, 2.20, '2024-03-17 17:30:00', 2, 1,
8, NULL),
('TCK-APU-004', 'EN VIVO', 200.00, 3.20, '2024-03-24 21:00:00', 3, 1,
13, 3);

-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 25. TABLA DetalleApuesta
INSERT INTO DetalleApuesta
(valor_cuota, resultado_detalle, id_apuesta, id_cuota) VALUES

-- Apuestas ganadas / perdidas / anuladas ya finalizadas
(2.10, 'GANADO', 1, 1),
(1.90, 'PERDIDO', 2, 11),
(2.20, 'GANADO', 3, 22),
(3.20, 'PERDIDO', 4, 33),
(3.10, 'GANADO', 5, 72);

-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

-- 26. TABLA ApuestaTransaccion
INSERT INTO ApuestaTransaccion
(id_apuesta, id_transaccion) VALUES
(1, 58),
(2, 59),
(3, 60);

-- SOLO SE PONEN ALGUNOS EN EL INFORME POR QUE SON MUCHOS DATOS
GO

UPDATE Apuesta SET cuota_total = 3.47 WHERE id_apuesta = 13;
UPDATE Apuesta SET cuota_total = 2.20 WHERE id_apuesta = 14;
UPDATE Apuesta SET cuota_total = 4.31 WHERE id_apuesta = 16;
UPDATE Apuesta SET cuota_total = 3.63 WHERE id_apuesta = 28;
UPDATE Apuesta SET cuota_total = 3.24 WHERE id_apuesta = 35;
UPDATE Apuesta SET cuota_total = 3.52 WHERE id_apuesta = 39;
GO

```

4.3.2. Scripts de creación y carga de datos de la base de datos no relacional

4.3.2.1. Creación de base de datos y colecciones

```

// Creacion de base de datos

use("betcore_db");

// Creacion de colecciones
db.createCollection("apuestas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [
        "id_apuesta",
        "codigo_ticket",
        "usuario",
        "selecciones",

```

```

    "moneda",
    "monto_apostado",
    "estado_apuesta",
    "fecha_apuesta",
  ],
  properties: {
    _id: {
      bsonType: "objectId",
      description: "Identificador único del documento.",
    },
    id_apuesta: {
      bsonType: "int",
      minimum: 1,
      description: "Identificador interno de la apuesta.",
    },
    codigo_ticket: {
      bsonType: "string",
      minLength: 8,
      maxLength: 20,
      description: "Código único del ticket de apuesta.",
    },
    usuario: {
      bsonType: "object",
      required: [
        "id_usuario",
        "nombre_completo",
        "tipo_documento",
        "numero_documento",
      ],
      properties: {
        id_usuario: {
          bsonType: "int",
          minimum: 1,
          description: "Identificador del usuario que
realizó la apuesta.",
        },
        nombre_completo: {
          bsonType: "string",
          minLength: 3,
          maxLength: 100,
          description: "Nombre completo del usuario.",
        },
        tipo_documento: {
          bsonType: "string",
          enum: ["DNI", "CE", "PASAPORTE"],
          description: "Tipo de documento del usuario.",
        },
        numero_documento: {

```

```

        bsonType: "string",
        minLength: 8,
        maxLength: 12,
        description: "Número de documento del usuario.",
    },
    correo: {
        bsonType: "string",
        maxLength: 120,
        description: "Correo electrónico del usuario.",
    },
},
},
local: {
    bsonType: "object",
    properties: {
        id_local: {
            bsonType: "int",
            minimum: 1,
            description: "Identificador del local.",
        },
        nombre_local: {
            bsonType: "string",
            minLength: 3,
            maxLength: 80,
            description: "Nombre del local.",
        },
        distrito: {
            bsonType: "string",
            minLength: 3,
            maxLength: 80,
            description: "Distrito donde se ubica el local.",
        },
        ciudad: {
            bsonType: "string",
            minLength: 3,
            maxLength: 80,
            description: "Ciudad donde se ubica el local.",
        },
    },
},
selecciones: {
    bsonType: "array",
    minItems: 1,
    description: "Lista de selecciones realizadas en la
apuesta.",
    items: {
        bsonType: "object",
        required: [

```

```

        "deporte",
        "evento",
        "mercado",
        "opcion",
        "cuota",
        "resultado",
    ],
    properties: {
        deporte: {
            bsonType: "string",
            minLength: 3,
            maxLength: 50,
            description: "Deporte relacionado con la
apuesta.",
        },
        evento: {
            bsonType: "string",
            minLength: 5,
            maxLength: 120,
            description: "Evento deportivo seleccionado.",
        },
        mercado: {
            bsonType: "string",
            minLength: 3,
            maxLength: 80,
            description: "Mercado de apuesta seleccionado.",
        },
        opcion: {
            bsonType: "string",
            minLength: 2,
            maxLength: 100,
            description: "Opción elegida por el usuario.",
        },
        cuota: {
            bsonType: "double",
            minimum: 1,
            maximum: 100,
            description: "Valor de la cuota al momento de
apostar.",
        },
        resultado: {
            bsonType: "string",
            enum: ["PENDIENTE", "ACERTADO", "FALLADO",
"ANULADO"],
            description: "Resultado de la selección.",
        },
    },
},
},

```

```

    },
    moneda: {
      bsonType: "object",
      required: ["id_moneda", "codigo"],
      properties: {
        id_moneda: {
          bsonType: "int",
          minimum: 1,
          description: "Identificador de la moneda.",
        },
        codigo: {
          bsonType: "string",
          enum: ["PEN", "USD", "EUR"],
          description: "Código de la moneda.",
        },
        simbolo: {
          bsonType: "string",
          enum: ["S/", "$", "€"],
          description: "Símbolo de la moneda.",
        },
      },
    },
    monto_apostado: {
      bsonType: "double",
      minimum: 0,
      description: "Monto apostado por el usuario.",
    },
    monto_ganancia: {
      bsonType: "double",
      minimum: 0,
      description: "Monto ganado cuando corresponda.",
    },
    estado_apuesta: {
      bsonType: "string",
      enum: ["PENDIENTE", "GANADA", "PERDIDA", "ANULADA",
"CANCELADA"],
      description: "Estado actual de la apuesta.",
    },
    fecha_apuesta: {
      bsonType: "date",
      description: "Fecha y hora de registro de la
apuesta.",
    },
    created_at: {
      bsonType: "date",
      description: "Fecha de creación del documento.",
    },
    updated_at: {

```

```

        bsonType: "date",
        description: "Fecha de última actualización del
documento.",
    },
    fuente: {
        bsonType: "string",
        maxLength: 50,
        description: "Fuente u origen del registro.",
    },
},
},
},
validationLevel: "strict",
validationAction: "error",
});

db.createCollection("movimientos_financieros", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: [
                "id_transaccion",
                "usuario_id",
                "moneda",
                "tipo",
                "estado",
                "monto",
                "fecha_operacion",
            ],
            properties: {
                _id: {
                    bsonType: "objectId",
                    description: "Identificador único del documento.",
                },
                id_transaccion: {
                    bsonType: "int",
                    minimum: 1,
                    description: "Identificador interno de la transacción
financiera.",
                },
                usuario_id: {
                    bsonType: "int",
                    minimum: 1,
                    description:
                        "Referencia al usuario relacionado con el movimiento
financiero.",
                },
                apuesta_id: {

```

```

        bsonType: "int",
        minimum: 1,
        description:
            "Referencia a la apuesta relacionada con el
movimiento financiero.",
    },
    moneda: {
        bsonType: "object",
        required: ["codigo"],
        properties: {
            codigo: {
                bsonType: "string",
                enum: ["PEN", "USD", "EUR"],
                description: "Código de la moneda utilizada.",
            },
            simbolo: {
                bsonType: "string",
                enum: ["S/", "$", "€"],
                description: "Símbolo de la moneda.",
            },
        },
    },
},
tipo: {
    bsonType: "string",
    enum: [
        "APUESTA",
        "DEPOSITO",
        "RETIRO",
        "PREMIO",
        "REEMBOLSO",
        "COMISION",
        "AJUSTE",
    ],
    description: "Tipo de movimiento financiero.",
},
estado: {
    bsonType: "string",
    enum: ["PENDIENTE", "COMPLETADA", "FALLIDA",
"RECHAZADA"],
    description: "Estado del movimiento financiero.",
},
monto: {
    bsonType: "double",
    minimum: 0,
    description: "Monto principal del movimiento
financiero.",
},
saldo_anterior: {

```

```

        bsonType: "double",
        minimum: 0,
        description: "Saldo antes de aplicar el movimiento.",
    },
    saldo_posterior: {
        bsonType: "double",
        minimum: 0,
        description: "Saldo después de aplicar el
movimiento.",
    },
    comision_aplicada: {
        bsonType: "double",
        minimum: 0,
        description: "Comisión aplicada al movimiento.",
    },
    metodo_pago: {
        bsonType: "string",
        minLength: 3,
        maxLength: 60,
        description: "Método de pago utilizado.",
    },
    numero_comprobante: {
        bsonType: "string",
        minLength: 4,
        maxLength: 30,
        description: "Número de comprobante de la operación.",
    },
    descripcion: {
        bsonType: "string",
        maxLength: 200,
        description: "Descripción breve del movimiento
financiero.",
    },
    fecha_operacion: {
        bsonType: "date",
        description: "Fecha principal de la operación
financiera.",
    },
    created_at: {
        bsonType: "date",
        description: "Fecha de creación del documento.",
    },
    updated_at: {
        bsonType: "date",
        description: "Fecha de última actualización del
documento.",
    },
    },
},

```

```

    },
  },
  validationLevel: "strict",
  validationAction: "error",
});

```

4.3.2.2. Inserción de datos para las colecciones

```

// use("betcore_db");
db.apuestas.insertMany([
  {
    id_apuesta: 1001,
    codigo_ticket: "APU2024070001",
    usuario: {
      id_usuario: 501,
      nombre_completo: "Juan Carlos Mendoza Peña",
      tipo_documento: "DNI",
      numero_documento: "12345678",
      correo: "juan.mendoza@email.com",
    },
    local: {
      id_local: 15,
      nombre_local: "Casa de Apuestas Central",
      distrito: "San Isidro",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Fútbol",
        evento: "Perú vs Chile - Eliminatorias 2026",
        mercado: "Ganador del Partido",
        opcion: "Perú",
        cuota: 2.45,
        resultado: "PENDIENTE",
      },
      {
        deporte: "Tenis",
        evento: "Wimbledon - Final Masculina",
        mercado: "Ganador",
        opcion: "Jannik Sinner",
        cuota: 1.95,
        resultado: "ACERTADO",
      },
      {
        deporte: "Baloncesto",
        evento: "NBA - Lakers vs Celtics",
        mercado: "Over/Under Total Puntos",
        opcion: "Over 215",
      }
    ]
  }
]);

```

```

        cuota: 1.8,
        resultado: "PENDIENTE",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 150.5,
    monto_ganancia: 387.45,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-01T10:30:00Z"),
    created_at: new Date("2024-07-01T10:30:00Z"),
    updated_at: new Date(),
    fuente: "Presencial",
  },
  {
    id_apuesta: 1002,
    codigo_ticket: "APU2024070002",
    usuario: {
      id_usuario: 502,
      nombre_completo: "María Fernanda Torres Ríos",
      tipo_documento: "DNI",
      numero_documento: "23456789",
      correo: "maria.torres@email.com",
    },
    local: {
      id_local: 16,
      nombre_local: "ControlBet Miraflores",
      distrito: "Miraflores",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Fútbol",
        evento: "Alianza Lima vs Sporting Cristal",
        mercado: "Doble oportunidad",
        opcion: "Alianza Lima o empate",
        cuota: 1.6,
        resultado: "PENDIENTE",
      },
      {
        deporte: "Vóley",
        evento: "Perú vs Argentina",
        mercado: "Ganador del Partido",
        opcion: "Perú",
        cuota: 2.1,
      }
    ]
  }
]

```

```

        resultado: "PENDIENTE",
    },
],
moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
},
monto_apostado: 80.0,
monto_ganancia: 0.0,
estado_apuesta: "PENDIENTE",
fecha_apuesta: new Date("2024-07-02T11:30:00Z"),
created_at: new Date("2024-07-02T11:30:00Z"),
updated_at: new Date(),
fuente: "Web",
},
{
    id_apuesta: 1003,
    codigo_ticket: "APU2024070003",
    usuario: {
        id_usuario: 503,
        nombre_completo: "Luis Alberto Quispe Vargas",
        tipo_documento: "DNI",
        numero_documento: "34567890",
        correo: "luis.quispe@email.com",
    },
    local: {
        id_local: 17,
        nombre_local: "BetCore Norte",
        distrito: "Los Olivos",
        ciudad: "Lima",
    },
},
selecciones: [
    {
        deporte: "Fútbol",
        evento: "Universitario vs Melgar",
        mercado: "Ganador del Partido",
        opcion: "Universitario",
        cuota: 1.85,
        resultado: "ACERTADO",
    },
],
moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
},
monto_apostado: 120.0,

```

```

    monto_ganancia: 222.0,
    estado_apuesta: "GANADA",
    fecha_apuesta: new Date("2024-07-03T12:30:00Z"),
    created_at: new Date("2024-07-03T12:30:00Z"),
    updated_at: new Date(),
    fuente: "App móvil",
  },
  {
    id_apuesta: 1004,
    codigo_ticket: "APU2024070004",
    usuario: {
      id_usuario: 504,
      nombre_completo: "Andrea Milagros Salazar León",
      tipo_documento: "DNI",
      numero_documento: "45678901",
      correo: "andrea.salazar@email.com",
    },
    local: {
      id_local: 18,
      nombre_local: "Apuestas Express Sur",
      distrito: "Chorrillos",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Tenis",
        evento: "Roland Garros - Semifinal",
        mercado: "Total de sets",
        opcion: "Más de 3.5 sets",
        cuota: 1.75,
        resultado: "FALLADO",
      },
      {
        deporte: "Fútbol",
        evento: "Brasil vs Uruguay",
        mercado: "Ambos anotan",
        opcion: "Sí",
        cuota: 1.9,
        resultado: "ACERTADO",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 95.75,
    monto_ganancia: 0.0,
  },
]

```

```

    estado_apuesta: "PERDIDA",
    fecha_apuesta: new Date("2024-07-04T13:30:00Z"),
    created_at: new Date("2024-07-04T13:30:00Z"),
    updated_at: new Date(),
    fuente: "Presencial",
  },
  {
    id_apuesta: 1005,
    codigo_ticket: "APU2024070005",
    usuario: {
      id_usuario: 505,
      nombre_completo: "Carlos Eduardo Ramírez Soto",
      tipo_documento: "DNI",
      numero_documento: "56789012",
      correo: "carlos.ramirez@email.com",
    },
    local: {
      id_local: 19,
      nombre_local: "ControlBet Arequipa",
      distrito: "Cercado",
      ciudad: "Arequipa",
    },
    selecciones: [
      {
        deporte: "Baloncesto",
        evento: "NBA - Warriors vs Nuggets",
        mercado: "Ganador",
        opcion: "Warriors",
        cuota: 2.2,
        resultado: "PENDIENTE",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 60.0,
    monto_ganancia: 0.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-05T14:30:00Z"),
    created_at: new Date("2024-07-05T14:30:00Z"),
    updated_at: new Date(),
    fuente: "Web",
  },
  {
    id_apuesta: 1006,
    codigo_ticket: "APU2024070006",

```

```

    usuario: {
      id_usuario: 506,
      nombre_completo: "Valeria Nicole Herrera Díaz",
      tipo_documento: "DNI",
      numero_documento: "67890123",
      correo: "valeria.herrera@email.com",
    },
    local: {
      id_local: 15,
      nombre_local: "Casa de Apuestas Central",
      distrito: "San Isidro",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Fútbol",
        evento: "Real Madrid vs Barcelona",
        mercado: "Ganador del Partido",
        opcion: "Real Madrid",
        cuota: 2.05,
        resultado: "PENDIENTE",
      },
      {
        deporte: "Fútbol",
        evento: "Manchester City vs Liverpool",
        mercado: "Total goles",
        opcion: "Más de 2.5",
        cuota: 1.7,
        resultado: "PENDIENTE",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 200.0,
    monto_ganancia: 0.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-06T15:30:00Z"),
    created_at: new Date("2024-07-06T15:30:00Z"),
    updated_at: new Date(),
    fuente: "App móvil",
  },
  {
    id_apuesta: 1007,
    codigo_ticket: "APU2024070007",
    usuario: {

```

```

    id_usuario: 507,
    nombre_completo: "José Antonio Rojas Campos",
    tipo_documento: "DNI",
    numero_documento: "78901234",
    correo: "jose.rojas@email.com",
  },
  local: {
    id_local: 16,
    nombre_local: "ControlBet Miraflores",
    distrito: "Miraflores",
    ciudad: "Lima",
  },
  selecciones: [
    {
      deporte: "Boxeo",
      evento: "Pelea Internacional - Peso Ligero",
      mercado: "Ganador",
      opcion: "Boxeador A",
      cuota: 1.65,
      resultado: "ACERTADO",
    },
  ],
  moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
  },
  monto_apostado: 50.0,
  monto_ganancia: 82.5,
  estado_apuesta: "GANADA",
  fecha_apuesta: new Date("2024-07-07T16:30:00Z"),
  created_at: new Date("2024-07-07T16:30:00Z"),
  updated_at: new Date(),
  fuente: "Presencial",
},
{
  id_apuesta: 1008,
  codigo_ticket: "APU2024070008",
  usuario: {
    id_usuario: 508,
    nombre_completo: "Lucía Gabriela Castillo Ruiz",
    tipo_documento: "DNI",
    numero_documento: "89012345",
    correo: "lucia.castillo@email.com",
  },
  local: {
    id_local: 17,
    nombre_local: "BetCore Norte",

```

```

    distrito: "Los Olivos",
    ciudad: "Lima",
  },
  selecciones: [
    {
      deporte: "Fútbol",
      evento: "Argentina vs Colombia",
      mercado: "Resultado exacto",
      opcion: "2-1",
      cuota: 7.5,
      resultado: "PENDIENTE",
    },
  ],
  moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
  },
  monto_apostado: 35.0,
  monto_ganancia: 0.0,
  estado_apuesta: "PENDIENTE",
  fecha_apuesta: new Date("2024-07-08T17:30:00Z"),
  created_at: new Date("2024-07-08T17:30:00Z"),
  updated_at: new Date(),
  fuente: "Web",
},
{
  id_apuesta: 1009,
  codigo_ticket: "APU2024070009",
  usuario: {
    id_usuario: 509,
    nombre_completo: "Miguel Ángel Paredes Luna",
    tipo_documento: "DNI",
    numero_documento: "90123456",
    correo: "miguel.paredes@email.com",
  },
  local: {
    id_local: 18,
    nombre_local: "Apuestas Express Sur",
    distrito: "Chorrillos",
    ciudad: "Lima",
  },
  selecciones: [
    {
      deporte: "Tenis",
      evento: "US Open - Cuartos de Final",
      mercado: "Ganador",
      opcion: "Carlos Alcaraz",
    },
  ],
}

```

```

        cuota: 1.55,
        resultado: "ACERTADO",
    },
    {
        deporte: "Baloncesto",
        evento: "NBA - Heat vs Bulls",
        mercado: "Ganador",
        opcion: "Heat",
        cuota: 1.88,
        resultado: "ACERTADO",
    },
],
moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
},
monto_apostado: 110.25,
monto_ganancia: 207.27,
estado_apuesta: "GANADA",
fecha_apuesta: new Date("2024-07-09T18:30:00Z"),
created_at: new Date("2024-07-09T18:30:00Z"),
updated_at: new Date(),
fuente: "App móvil",
},
{
    id_apuesta: 1010,
    codigo_ticket: "APU2024070010",
    usuario: {
        id_usuario: 510,
        nombre_completo: "Camila Alejandra Flores Vega",
        tipo_documento: "DNI",
        numero_documento: "11223344",
        correo: "camila.flores@email.com",
    },
    local: {
        id_local: 19,
        nombre_local: "ControlBet Arequipa",
        distrito: "Cercado",
        ciudad: "Arequipa",
    },
    selecciones: [
        {
            deporte: "Fútbol",
            evento: "Atlético Madrid vs Sevilla",
            mercado: "Primera mitad",
            opcion: "Empate",
            cuota: 2.1,

```

```

        resultado: "FALLADO",
    },
],
moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
},
monto_apostado: 70.0,
monto_ganancia: 0.0,
estado_apuesta: "PERDIDA",
fecha_apuesta: new Date("2024-07-10T19:30:00Z"),
created_at: new Date("2024-07-10T19:30:00Z"),
updated_at: new Date(),
fuente: "Presencial",
},
{
    id_apuesta: 1011,
    codigo_ticket: "APU2024070011",
    usuario: {
        id_usuario: 511,
        nombre_completo: "Diego Fernando Morales Cruz",
        tipo_documento: "DNI",
        numero_documento: "22334455",
        correo: "diego.morales@email.com",
    },
    local: {
        id_local: 15,
        nombre_local: "Casa de Apuestas Central",
        distrito: "San Isidro",
        ciudad: "Lima",
    },
},
selecciones: [
    {
        deporte: "Fútbol",
        evento: "Bayern Múnich vs Dortmund",
        mercado: "Total goles",
        opcion: "Más de 3.5",
        cuota: 2.3,
        resultado: "PENDIENTE",
    },
],
moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
},
monto_apostado: 45.5,

```

```

    monto_ganancia: 0.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-11T10:30:00Z"),
    created_at: new Date("2024-07-11T10:30:00Z"),
    updated_at: new Date(),
    fuente: "Web",
  },
  {
    id_apuesta: 1012,
    codigo_ticket: "APU2024070012",
    usuario: {
      id_usuario: 512,
      nombre_completo: "Sofía Daniela Medina Ponce",
      tipo_documento: "DNI",
      numero_documento: "33445566",
      correo: "sofia.medina@email.com",
    },
    local: {
      id_local: 16,
      nombre_local: "ControlBet Miraflores",
      distrito: "Miraflores",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Vóley",
        evento: "Brasil vs Italia",
        mercado: "Ganador del Partido",
        opcion: "Brasil",
        cuota: 1.72,
        resultado: "ACERTADO",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 130.0,
    monto_ganancia: 223.6,
    estado_apuesta: "GANADA",
    fecha_apuesta: new Date("2024-07-12T11:30:00Z"),
    created_at: new Date("2024-07-12T11:30:00Z"),
    updated_at: new Date(),
    fuente: "App móvil",
  },
  {
    id_apuesta: 1013,

```

```

codigo_ticket: "APU2024070013",
usuario: {
  id_usuario: 513,
  nombre_completo: "Renato Alonso Núñez Prado",
  tipo_documento: "DNI",
  numero_documento: "44556677",
  correo: "renato.nunez@email.com",
},
local: {
  id_local: 17,
  nombre_local: "BetCore Norte",
  distrito: "Los Olivos",
  ciudad: "Lima",
},
selecciones: [
  {
    deporte: "Fútbol",
    evento: "PSG vs Marsella",
    mercado: "Ambos anotan",
    opcion: "Sí",
    cuota: 1.68,
    resultado: "PENDIENTE",
  },
  {
    deporte: "Tenis",
    evento: "ATP 500 - Final",
    mercado: "Ganador",
    opcion: "Daniil Medvedev",
    cuota: 2.05,
    resultado: "PENDIENTE",
  },
],
moneda: {
  id_moneda: 1,
  codigo: "PEN",
  simbolo: "S/",
},
monto_apostado: 90.0,
monto_ganancia: 0.0,
estado_apuesta: "PENDIENTE",
fecha_apuesta: new Date("2024-07-13T12:30:00Z"),
created_at: new Date("2024-07-13T12:30:00Z"),
updated_at: new Date(),
fuente: "Presencial",
},
{
  id_apuesta: 1014,
  codigo_ticket: "APU2024070014",

```

```

    usuario: {
      id_usuario: 514,
      nombre_completo: "Paola Vanessa Chávez Pino",
      tipo_documento: "DNI",
      numero_documento: "55667788",
      correo: "paola.chavez@email.com",
    },
    local: {
      id_local: 18,
      nombre_local: "Apuestas Express Sur",
      distrito: "Chorrillos",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Baloncesto",
        evento: "NBA - Celtics vs Knicks",
        mercado: "Handicap",
        opcion: "Celtics -5.5",
        cuota: 1.92,
        resultado: "ACERTADO",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 160.0,
    monto_ganancia: 307.2,
    estado_apuesta: "GANADA",
    fecha_apuesta: new Date("2024-07-14T13:30:00Z"),
    created_at: new Date("2024-07-14T13:30:00Z"),
    updated_at: new Date(),
    fuente: "Web",
  },
  {
    id_apuesta: 1015,
    codigo_ticket: "APU2024070015",
    usuario: {
      id_usuario: 515,
      nombre_completo: "Fabricio José Rivera Rupay",
      tipo_documento: "DNI",
      numero_documento: "66778899",
      correo: "fabricio.rivera@email.com",
    },
    local: {
      id_local: 19,

```

```

    nombre_local: "ControlBet Arequipa",
    distrito: "Cercado",
    ciudad: "Arequipa",
  },
  selecciones: [
    {
      deporte: "Fútbol",
      evento: "Sport Boys vs Cienciano",
      mercado: "Ganador del Partido",
      opcion: "Cienciano",
      cuota: 2.6,
      resultado: "FALLADO",
    },
  ],
  moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
  },
  monto_apostado: 40.0,
  monto_ganancia: 0.0,
  estado_apuesta: "PERDIDA",
  fecha_apuesta: new Date("2024-07-15T14:30:00Z"),
  created_at: new Date("2024-07-15T14:30:00Z"),
  updated_at: new Date(),
  fuente: "App móvil",
},
{
  id_apuesta: 1016,
  codigo_ticket: "APU2024070016",
  usuario: {
    id_usuario: 516,
    nombre_completo: "Brayan Benjamin Huerta Cárdenas",
    tipo_documento: "DNI",
    numero_documento: "77889900",
    correo: "brayan.huerta@email.com",
  },
  local: {
    id_local: 15,
    nombre_local: "Casa de Apuestas Central",
    distrito: "San Isidro",
    ciudad: "Lima",
  },
  selecciones: [
    {
      deporte: "Fútbol",
      evento: "Inter vs Juventus",
      mercado: "Ganador del Partido",

```

```

        opcion: "Inter",
        cuota: 1.95,
        resultado: "ACERTADO",
      },
      {
        deporte: "Boxeo",
        evento: "Campeonato Mundial - Peso Medio",
        mercado: "Ganador",
        opcion: "Boxeador B",
        cuota: 2.25,
        resultado: "PENDIENTE",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 220.0,
    monto_ganancia: 429.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-16T15:30:00Z"),
    created_at: new Date("2024-07-16T15:30:00Z"),
    updated_at: new Date(),
    fuente: "Presencial",
  },
  {
    id_apuesta: 1017,
    codigo_ticket: "APU2024070017",
    usuario: {
      id_usuario: 517,
      nombre_completo: "Giovanni Marcelo Gallegos De La Cruz",
      tipo_documento: "DNI",
      numero_documento: "88990011",
      correo: "giovanni.gallegos@email.com",
    },
    local: {
      id_local: 16,
      nombre_local: "ControlBet Miraflores",
      distrito: "Miraflores",
      ciudad: "Lima",
    },
  },
  selecciones: [
    {
      deporte: "Tenis",
      evento: "WTA - Final",
      mercado: "Ganadora",
      opcion: "Iga Swiatek",
    }
  ]
}

```

```

        cuota: 1.5,
        resultado: "ACERTADO",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 75.0,
    monto_ganancia: 112.5,
    estado_apuesta: "GANADA",
    fecha_apuesta: new Date("2024-07-17T16:30:00Z"),
    created_at: new Date("2024-07-17T16:30:00Z"),
    updated_at: new Date(),
    fuente: "Web",
  },
  {
    id_apuesta: 1018,
    codigo_ticket: "APU2024070018",
    usuario: {
      id_usuario: 518,
      nombre_completo: "Patricia Elena Córdova Sánchez",
      tipo_documento: "DNI",
      numero_documento: "99001122",
      correo: "patricia.cordova@email.com",
    },
    local: {
      id_local: 17,
      nombre_local: "BetCore Norte",
      distrito: "Los Olivos",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Fútbol",
        evento: "Ecuador vs Bolivia",
        mercado: "Ganador del Partido",
        opcion: "Ecuador",
        cuota: 1.45,
        resultado: "PENDIENTE",
      },
    ],
  },
  moneda: {
    id_moneda: 1,
    codigo: "PEN",
    simbolo: "S/",
  },
},

```

```

    monto_apostado: 55.0,
    monto_ganancia: 0.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-18T17:30:00Z"),
    created_at: new Date("2024-07-18T17:30:00Z"),
    updated_at: new Date(),
    fuente: "App móvil",
  },
  {
    id_apuesta: 1019,
    codigo_ticket: "APU2024070019",
    usuario: {
      id_usuario: 519,
      nombre_completo: "Roberto Martín Aguirre Molina",
      tipo_documento: "DNI",
      numero_documento: "10101010",
      correo: "roberto.aguirre@email.com",
    },
    local: {
      id_local: 18,
      nombre_local: "Apuestas Express Sur",
      distrito: "Chorrillos",
      ciudad: "Lima",
    },
    selecciones: [
      {
        deporte: "Baloncesto",
        evento: "NBA - Suns vs Mavericks",
        mercado: "Total puntos",
        opcion: "Under 228",
        cuota: 1.83,
        resultado: "PENDIENTE",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 100.0,
    monto_ganancia: 0.0,
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: new Date("2024-07-19T18:30:00Z"),
    created_at: new Date("2024-07-19T18:30:00Z"),
    updated_at: new Date(),
    fuente: "Presencial",
  },
  {

```

```

    id_apuesta: 1020,
    codigo_ticket: "APU2024070020",
    usuario: {
      id_usuario: 520,
      nombre_completo: "Daniela Antonella Ramos Ibarra",
      tipo_documento: "DNI",
      numero_documento: "20202020",
      correo: "daniela.ramos@email.com",
    },
    local: {
      id_local: 19,
      nombre_local: "ControlBet Arequipa",
      distrito: "Cercado",
      ciudad: "Arequipa",
    },
    selecciones: [
      {
        deporte: "Fútbol",
        evento: "Portugal vs Francia",
        mercado: "Clasifica",
        opcion: "Francia",
        cuota: 1.98,
        resultado: "ACERTADO",
      },
    ],
    moneda: {
      id_moneda: 1,
      codigo: "PEN",
      simbolo: "S/",
    },
    monto_apostado: 180.0,
    monto_ganancia: 356.4,
    estado_apuesta: "GANADA",
    fecha_apuesta: new Date("2024-07-20T19:30:00Z"),
    created_at: new Date("2024-07-20T19:30:00Z"),
    updated_at: new Date(),
    fuente: "Web",
  },
]);

```

```

// use("betcore_db");

db.movimientos_financieros.insertMany([
  {
    id_transaccion: 2001,
    usuario_id: 501,
    apuesta_id: 1001,

```

```

moneda: {
  codigo: "PEN",
  simbolo: "S/",
},
tipo: "APUESTA",
estado: "COMPLETADA",
monto: 150.5,
saldo_anterior: 500.0,
saldo_posterior: 349.5,
comision_aplicada: 0.0,
metodo_pago: "Saldo disponible",
numero_comprobante: "COMP-202407-0001",
descripcion:
  "Movimiento financiero de tipo apuesta registrado en
BetCore DB.",
fecha_operacion: new Date("2024-07-01T11:15:00Z"),
created_at: new Date("2024-07-01T11:15:00Z"),
updated_at: new Date(),
},
{
  id_transaccion: 2002,
  usuario_id: 502,
  apuesta_id: 1002,
  moneda: {
    codigo: "PEN",
    simbolo: "S/",
  },
  tipo: "APUESTA",
  estado: "COMPLETADA",
  monto: 80.0,
  saldo_anterior: 420.0,
  saldo_posterior: 340.0,
  comision_aplicada: 0.0,
  metodo_pago: "Saldo disponible",
  numero_comprobante: "COMP-202407-0002",
  descripcion:
    "Movimiento financiero de tipo apuesta registrado en
BetCore DB.",
  fecha_operacion: new Date("2024-07-02T12:15:00Z"),
  created_at: new Date("2024-07-02T12:15:00Z"),
  updated_at: new Date(),
},
{
  id_transaccion: 2003,
  usuario_id: 503,
  apuesta_id: 1003,
  moneda: {
    codigo: "PEN",

```

```

        simbolo: "S/",
    },
    tipo: "PREMIO",
    estado: "COMPLETADA",
    monto: 222.0,
    saldo_anterior: 300.0,
    saldo_posterior: 522.0,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0003",
    descripcion: "Pago de premio por apuesta ganada.",
    fecha_operacion: new Date("2024-07-03T13:15:00Z"),
    created_at: new Date("2024-07-03T13:15:00Z"),
    updated_at: new Date(),
},
{
    id_transaccion: 2004,
    usuario_id: 504,
    apuesta_id: 1004,
    moneda: {
        codigo: "PEN",
        simbolo: "S/",
    },
    tipo: "REEMBOLSO",
    estado: "COMPLETADA",
    monto: 25.0,
    saldo_anterior: 250.0,
    saldo_posterior: 275.0,
    comision_aplicada: 0.0,
    metodo_pago: "Yape",
    numero_comprobante: "COMP-202407-0004",
    descripcion: "Reembolso parcial registrado por ajuste de
apuesta.",
    fecha_operacion: new Date("2024-07-04T14:15:00Z"),
    created_at: new Date("2024-07-04T14:15:00Z"),
    updated_at: new Date(),
},
{
    id_transaccion: 2005,
    usuario_id: 505,
    apuesta_id: 1005,
    moneda: {
        codigo: "PEN",
        simbolo: "S/",
    },
    tipo: "DEPOSITO",
    estado: "COMPLETADA",
    monto: 100.0,

```

```

    saldo_anterior: 90.0,
    saldo_posterior: 190.0,
    comision_aplicada: 0.0,
    metodo_pago: "Tarjeta de débito",
    numero_comprobante: "COMP-202407-0005",
    descripcion: "Depósito de saldo para operaciones de
apuesta.",
    fecha_operacion: new Date("2024-07-05T15:15:00Z"),
    created_at: new Date("2024-07-05T15:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2006,
    usuario_id: 506,
    apuesta_id: 1006,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "RETIRO",
    estado: "PENDIENTE",
    monto: 60.0,
    saldo_anterior: 300.0,
    saldo_posterior: 237.5,
    comision_aplicada: 2.5,
    metodo_pago: "Transferencia bancaria",
    numero_comprobante: "COMP-202407-0006",
    descripcion: "Solicitud de retiro de saldo del usuario.",
    fecha_operacion: new Date("2024-07-06T16:15:00Z"),
    created_at: new Date("2024-07-06T16:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2007,
    usuario_id: 507,
    apuesta_id: 1007,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "PREMIO",
    estado: "COMPLETADA",
    monto: 82.5,
    saldo_anterior: 180.0,
    saldo_posterior: 262.5,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0007",
  }
]

```

```

    descripcion: "Pago de premio por apuesta ganada.",
    fecha_operacion: new Date("2024-07-07T17:15:00Z"),
    created_at: new Date("2024-07-07T17:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2008,
    usuario_id: 508,
    apuesta_id: 1008,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "COMISION",
    estado: "COMPLETADA",
    monto: 5.0,
    saldo_anterior: 140.0,
    saldo_posterior: 135.0,
    comision_aplicada: 5.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0008",
    descripcion: "Comisión aplicada por operación financiera.",
    fecha_operacion: new Date("2024-07-08T18:15:00Z"),
    created_at: new Date("2024-07-08T18:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2009,
    usuario_id: 509,
    apuesta_id: 1009,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "APUESTA",
    estado: "COMPLETADA",
    monto: 110.25,
    saldo_anterior: 400.0,
    saldo_posterior: 289.75,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0009",
    descripcion:
      "Movimiento financiero de tipo apuesta registrado en
BetCore DB.",
    fecha_operacion: new Date("2024-07-09T19:15:00Z"),
    created_at: new Date("2024-07-09T19:15:00Z"),
    updated_at: new Date(),
  },

```

```

    },
    {
      id_transaccion: 2010,
      usuario_id: 510,
      apuesta_id: 1010,
      moneda: {
        codigo: "PEN",
        simbolo: "S/",
      },
      tipo: "AJUSTE",
      estado: "COMPLETADA",
      monto: 20.0,
      saldo_anterior: 95.0,
      saldo_posterior: 115.0,
      comision_aplicada: 0.0,
      metodo_pago: "Ajuste administrativo",
      numero_comprobante: "COMP-202407-0010",
      descripcion: "Ajuste administrativo de saldo.",
      fecha_operacion: new Date("2024-07-10T11:15:00Z"),
      created_at: new Date("2024-07-10T11:15:00Z"),
      updated_at: new Date(),
    },
    {
      id_transaccion: 2011,
      usuario_id: 511,
      apuesta_id: 1011,
      moneda: {
        codigo: "PEN",
        simbolo: "S/",
      },
      tipo: "DEPOSITO",
      estado: "COMPLETADA",
      monto: 80.0,
      saldo_anterior: 100.0,
      saldo_posterior: 180.0,
      comision_aplicada: 0.0,
      metodo_pago: "Plin",
      numero_comprobante: "COMP-202407-0011",
      descripcion: "Depósito de saldo para operaciones de
apuesta.",
      fecha_operacion: new Date("2024-07-11T12:15:00Z"),
      created_at: new Date("2024-07-11T12:15:00Z"),
      updated_at: new Date(),
    },
    {
      id_transaccion: 2012,
      usuario_id: 512,
      apuesta_id: 1012,

```

```

moneda: {
  codigo: "PEN",
  simbolo: "S/",
},
tipo: "PREMIO",
estado: "COMPLETADA",
monto: 223.6,
saldo_anterior: 200.0,
saldo_posterior: 423.6,
comision_aplicada: 0.0,
metodo_pago: "Saldo disponible",
numero_comprobante: "COMP-202407-0012",
descripcion: "Pago de premio por apuesta ganada.",
fecha_operacion: new Date("2024-07-12T13:15:00Z"),
created_at: new Date("2024-07-12T13:15:00Z"),
updated_at: new Date(),
},
{
  id_transaccion: 2013,
  usuario_id: 513,
  apuesta_id: 1013,
  moneda: {
    codigo: "PEN",
    simbolo: "S/",
  },
  tipo: "RETIRO",
  estado: "RECHAZADA",
  monto: 90.0,
  saldo_anterior: 250.0,
  saldo_posterior: 250.0,
  comision_aplicada: 0.0,
  metodo_pago: "Transferencia bancaria",
  numero_comprobante: "COMP-202407-0013",
  descripcion: "Solicitud de retiro rechazada por validación
administrativa.",
  fecha_operacion: new Date("2024-07-13T14:15:00Z"),
  created_at: new Date("2024-07-13T14:15:00Z"),
  updated_at: new Date(),
},
{
  id_transaccion: 2014,
  usuario_id: 514,
  apuesta_id: 1014,
  moneda: {
    codigo: "PEN",
    simbolo: "S/",
  },
  tipo: "PREMIO",

```

```

    estado: "COMPLETADA",
    monto: 307.2,
    saldo_anterior: 320.0,
    saldo_posterior: 627.2,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0014",
    descripcion: "Pago de premio por apuesta ganada.",
    fecha_operacion: new Date("2024-07-14T15:15:00Z"),
    created_at: new Date("2024-07-14T15:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2015,
    usuario_id: 515,
    apuesta_id: 1015,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "APUESTA",
    estado: "COMPLETADA",
    monto: 40.0,
    saldo_anterior: 120.0,
    saldo_posterior: 80.0,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0015",
    descripcion:
      "Movimiento financiero de tipo apuesta registrado en
BetCore DB.",
    fecha_operacion: new Date("2024-07-15T16:15:00Z"),
    created_at: new Date("2024-07-15T16:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2016,
    usuario_id: 516,
    apuesta_id: 1016,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "APUESTA",
    estado: "COMPLETADA",
    monto: 220.0,
    saldo_anterior: 600.0,
    saldo_posterior: 380.0,
  },

```

```

    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0016",
    descripcion:
      "Movimiento financiero de tipo apuesta registrado en
BetCore DB.",
    fecha_operacion: new Date("2024-07-16T17:15:00Z"),
    created_at: new Date("2024-07-16T17:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2017,
    usuario_id: 517,
    apuesta_id: 1017,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "PREMIO",
    estado: "COMPLETADA",
    monto: 112.5,
    saldo_anterior: 210.0,
    saldo_posterior: 322.5,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0017",
    descripcion: "Pago de premio por apuesta ganada.",
    fecha_operacion: new Date("2024-07-17T18:15:00Z"),
    created_at: new Date("2024-07-17T18:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2018,
    usuario_id: 518,
    apuesta_id: 1018,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "REEMBOLSO",
    estado: "COMPLETADA",
    monto: 35.0,
    saldo_anterior: 95.0,
    saldo_posterior: 130.0,
    comision_aplicada: 0.0,
    metodo_pago: "Yape",
    numero_comprobante: "COMP-202407-0018",
  }
}

```

```

    descripcion: "Reembolso por anulación parcial de
operación.",
    fecha_operacion: new Date("2024-07-18T19:15:00Z"),
    created_at: new Date("2024-07-18T19:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2019,
    usuario_id: 519,
    apuesta_id: 1019,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "COMISION",
    estado: "COMPLETADA",
    monto: 10.0,
    saldo_anterior: 170.0,
    saldo_posterior: 160.0,
    comision_aplicada: 10.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0019",
    descripcion: "Comisión aplicada por operación financiera.",
    fecha_operacion: new Date("2024-07-19T11:15:00Z"),
    created_at: new Date("2024-07-19T11:15:00Z"),
    updated_at: new Date(),
  },
  {
    id_transaccion: 2020,
    usuario_id: 520,
    apuesta_id: 1020,
    moneda: {
      codigo: "PEN",
      simbolo: "S/",
    },
    tipo: "PREMIO",
    estado: "COMPLETADA",
    monto: 356.4,
    saldo_anterior: 280.0,
    saldo_posterior: 636.4,
    comision_aplicada: 0.0,
    metodo_pago: "Saldo disponible",
    numero_comprobante: "COMP-202407-0020",
    descripcion: "Pago de premio por apuesta ganada.",
    fecha_operacion: new Date("2024-07-20T12:15:00Z"),
    created_at: new Date("2024-07-20T12:15:00Z"),
    updated_at: new Date(),
  },
},

```

```
]);
```

4.4. Consultas

En esta sección se presentan las consultas implementadas para la base de datos relacional y no relacional de BetCore DB. Estas consultas tienen como finalidad validar que la estructura de datos propuesta permita obtener información útil para el control de apuestas, seguimiento de transacciones financieras, consulta de saldos, análisis de eventos deportivos y monitoreo de operaciones del negocio.

4.4.1. Consultas para la base de datos relacional

En esta sección se presentan consultas usando el DBMS SQL Server Management Studio (SSMS) orientadas a obtener información útil para la gestión operativa y financiera de BetCore DB. Estas consultas permiten analizar apuestas, usuarios, eventos deportivos, cuotas, transacciones, saldos, comprobantes y locales, con el objetivo de apoyar el control del negocio, reducir inconsistencias y facilitar la toma de decisiones.

4.4.1.1. Consultas y Subqueries

Consulta 1: Historial de apuestas registradas

Muestra las apuestas registradas durante el periodo de marzo a junio de 2024, incluyendo cliente, local, estado, monto apostado, cuota total y premio potencial.

Importancia:

Permite revisar las apuestas realizadas dentro del periodo operativo cargado en la base de datos y dar seguimiento a su estado.

```
SELECT
    a.id_apuesta,
    a.codigo_ticket,
    CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
    l.nombre_local,
    ea.nombre_estado AS estado_apuesta,
    a.tipo_apuesta,
    a.monto_apostado,
    a.cuota_total,
    ROUND(a.monto_apostado * a.cuota_total, 2) AS premio_potencial,
    m.codigo_moneda,
    a.fecha_apuesta
FROM Apuesta a
INNER JOIN Usuario u ON a.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
```

```

INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta
INNER JOIN Moneda m ON a.id_moneda = m.id_moneda
LEFT JOIN Local l ON a.id_local = l.id_local
WHERE a.fecha_apuesta BETWEEN '2024-03-01' AND '2024-06-30'
AND ea.nombre_estado IN ('PENDIENTE', 'GANADA', 'PERDIDA', 'ANULADA')
ORDER BY a.fecha_apuesta DESC;

```

Consulta 2: Resumen de apuestas por estado

Agrupar las apuestas según su estado, mostrando cantidad de apuestas, monto total, promedio, monto máximo y monto mínimo.

Importancia:

Ayuda a conocer la distribución de apuestas pendientes, ganadas, perdidas o anuladas

```

SELECT
    ea.nombre_estado AS estado_apuesta,
    COUNT(a.id_apuesta) AS cantidad_apuestas,
    SUM(a.monto_apostado) AS monto_total_apostado,
    ROUND(AVG(a.monto_apostado), 2) AS monto_promedio,
    MAX(a.monto_apostado) AS monto_maximo,
    MIN(a.monto_apostado) AS monto_minimo
FROM Apuesta a
INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta
GROUP BY ea.nombre_estado
HAVING COUNT(a.id_apuesta) > 0
ORDER BY monto_total_apostado DESC;

```

Consulta 3: Clientes con monto apostado superior al promedio general

Identifica a los clientes cuyo monto total apostado es mayor al promedio general de monto apostado por usuario.

Importancia:

Permite reconocer clientes con mayor actividad económica dentro de la casa de apuestas, lo cual puede servir para análisis de comportamiento, control de riesgo o estrategias de fidelización.

```

SELECT
    u.id_usuario,
    CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
    COUNT(a.id_apuesta) AS total_apuestas,
    SUM(a.monto_apostado) AS monto_total_apostado,
    ROUND(AVG(a.monto_apostado), 2) AS promedio_por_apuesta,
    MAX(a.monto_apostado) AS mayor_apuesta,
    MIN(a.monto_apostado) AS menor_apuesta
FROM Usuario u
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN Apuesta a ON u.id_usuario = a.id_usuario
GROUP BY u.id_usuario, p.nombres, p.apellidos
HAVING SUM(a.monto_apostado) > (

```

```

SELECT AVG(total_apostado)
FROM (
    SELECT SUM(monto_apostado) AS total_apostado
    FROM Apuesta
    GROUP BY id_usuario
) AS resumen_usuarios
ORDER BY monto_total_apostado DESC;

```

Consulta 4: Eventos deportivos con apuestas superiores al promedio

Muestra los eventos deportivos cuya cantidad de apuestas asociadas es mayor o igual al promedio de apuestas por evento.

Importancia:

Ayuda a identificar qué eventos deportivos generan más interés que el promedio, permitiendo priorizar su monitoreo y análisis comercial.

```

SELECT
    resumen.id_evento,
    resumen.nombre_evento,
    resumen.nombre_deporte,
    resumen.nombre_liga,
    resumen.cantidad_apuestas,
    resumen.cuota_promedio
FROM (
    SELECT
        e.id_evento,
        e.nombre_evento,
        d.nombre_deporte,
        l.nombre_liga,
        COUNT(DISTINCT a.id_apuesta) AS cantidad_apuestas,
        ROUND(AVG(da.valor_cuota), 2) AS cuota_promedio
    FROM Evento e
    INNER JOIN Liga l ON e.id_liga = l.id_liga
    INNER JOIN Deporte d ON l.id_deporte = d.id_deporte
    LEFT JOIN MercadoApuesta ma ON e.id_evento = ma.id_evento
    LEFT JOIN Cuota c ON ma.id_mercado = c.id_mercado
    LEFT JOIN DetalleApuesta da ON c.id_cuota = da.id_cuota
    LEFT JOIN Apuesta a ON da.id_apuesta = a.id_apuesta
    GROUP BY e.id_evento, e.nombre_evento, d.nombre_deporte,
    l.nombre_liga
) AS resumen
WHERE resumen.cantidad_apuestas >= (
    SELECT AVG(CAST(cantidad_apuestas AS DECIMAL(10,2)))
    FROM (
        SELECT
            e.id_evento,
            COUNT(DISTINCT a.id_apuesta) AS cantidad_apuestas
        FROM Evento e
        LEFT JOIN MercadoApuesta ma ON e.id_evento = ma.id_evento
        LEFT JOIN Cuota c ON ma.id_mercado = c.id_mercado
        LEFT JOIN DetalleApuesta da ON c.id_cuota = da.id_cuota
        LEFT JOIN Apuesta a ON da.id_apuesta = a.id_apuesta
        GROUP BY e.id_evento
    ) AS promedio_eventos
)
AND resumen.cantidad_apuestas > 0

```

```
ORDER BY resumen.cantidad_apuestas DESC;
```

Consulta 5: Cuotas activas para eventos de Universitario

Lista las cuotas activas asociadas a eventos deportivos que contienen la palabra “Universitario”.

Importancia:

Permite revisar opciones de apuesta disponibles para eventos de alta demanda.

```
SELECT
    e.nombre_evento,
    ma.nombre_mercado,
    c.nombre_opcion,
    c.valor AS valor_cuota,
    c.estado AS estado_cuota
FROM Evento e
INNER JOIN MercadoApuesta ma ON e.id_evento = ma.id_evento
INNER JOIN Cuota c ON ma.id_mercado = c.id_mercado
WHERE e.nombre_evento LIKE '%Universitario%'
      AND c.estado = 'ACTIVA'
ORDER BY e.nombre_evento, ma.nombre_mercado, c.valor DESC;
```

Consulta 6: Apuestas con monto superior al promedio general

Identifica apuestas cuyo monto apostado es mayor al promedio general registrado.

Importancia:

Ayuda a detectar operaciones de mayor valor que podrían requerir seguimiento especial.

```
SELECT
    a.id_apuesta,
    a.codigo_ticket,
    CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
    ea.nombre_estado,
    a.monto_apostado,
    a.fecha_apuesta
FROM Apuesta a
INNER JOIN Usuario u ON a.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta
WHERE a.monto_apostado > (
    SELECT AVG(monto_apostado)
    FROM Apuesta
)
AND ea.nombre_estado IN ('PENDIENTE', 'GANADA', 'PERDIDA')
ORDER BY a.monto_apostado DESC;
```

Consulta 7: Resumen de transacciones por tipo y estado

Agrupar las transacciones financieras por tipo y estado, mostrando cantidad, monto total y promedio.

Importancia:

Permite controlar depósitos, retiros, apuestas, premios, bonificaciones y devoluciones.

```
SELECT
    tt.nombre_tipo AS tipo_transaccion,
    et.nombre_estado AS estado_transaccion,
    COUNT(tf.id_transaccion) AS cantidad_transacciones,
    SUM(tf.monto_total) AS monto_total,
    ROUND(AVG(tf.monto_total), 2) AS monto_promedio
FROM TransaccionFinanciera tf
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
GROUP BY tt.nombre_tipo, et.nombre_estado
HAVING COUNT(tf.id_transaccion) > 0
ORDER BY monto_total DESC;
```

Consulta 8: Transacciones pendientes por revisar

Muestra transacciones pendientes cuyo monto total es igual o superior a S/ 100.

Importancia:

Permite priorizar operaciones pendientes que requieren validación del área financiera.

```
SELECT
    tf.id_transaccion,
    CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
    tt.nombre_tipo AS tipo_transaccion,
    mp.nombre_metodo AS metodo_pago,
    tf.monto_total,
    m.codigo_moneda,
    tf.fecha_solicitud,
    et.nombre_estado
FROM TransaccionFinanciera tf
INNER JOIN Usuario u ON tf.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
INNER JOIN MetodoPago mp ON tf.id_metodo_pago = mp.id_metodo_pago
INNER JOIN Moneda m ON tf.id_moneda = m.id_moneda
WHERE et.nombre_estado = 'PENDIENTE'
AND tf.monto_total >= 100
ORDER BY tf.monto_total DESC, tf.fecha_solicitud ASC;
```

Consulta 9: Saldos disponibles y retenidos por usuario

Consulta el saldo disponible, retenido y total de cada usuario.

Importancia:

Permite controlar los fondos disponibles de los usuarios y detectar montos retenidos por operaciones pendientes.

```
SELECT
    u.id_usuario,
    CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
    m.codigo_moneda,
    s.monto_disponible,
    s.monto_retenido,
    s.monto_total,
    s.ultima_actualizacion
FROM Saldo s
INNER JOIN Usuario u ON s.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN Moneda m ON s.id_moneda = m.id_moneda
WHERE m.codigo_moneda IN ('PEN', 'USD', 'EUR')
    AND s.monto_total >= 0
ORDER BY s.monto_total DESC;
```

Consulta 10: Mayores variaciones de saldo

Muestra los cambios más significativos en el saldo de los usuarios.

Importancia:

Permite auditar movimientos importantes de dinero y detectar operaciones inusuales.

```
SELECT TOP 10
    hs.id_historial_saldo,
    CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
    hs.saldo_anterior,
    hs.saldo_posterior,
    ROUND(hs.saldo_posterior - hs.saldo_anterior, 2) AS variacion_saldo,
    hs.fecha_registro,
    tf.id_transaccion
FROM HistorialSaldo hs
INNER JOIN Saldo s ON hs.id_saldo = s.id_saldo
INNER JOIN Usuario u ON s.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN TransaccionFinanciera tf ON hs.id_transaccion =
    tf.id_transaccion
WHERE ABS(hs.saldo_posterior - hs.saldo_anterior) > 0
ORDER BY ABS(hs.saldo_posterior - hs.saldo_anterior) DESC;
```

Consulta 11: Ingresos y egresos diarios del negocio

Resume los montos diarios según la naturaleza de la transacción.

Importancia:

Facilita el control financiero diario y permite evaluar el balance operativo.

```
SELECT
    CAST(tf.fecha_proceso AS DATE) AS fecha,
```

```

SUM(CASE WHEN tt.naturaleza = 'INGRESO' THEN tf.monto_netto ELSE 0
END) AS total_ingresos,
SUM(CASE WHEN tt.naturaleza = 'EGRESO' THEN tf.monto_netto ELSE 0
END) AS total_egresos,
ROUND(
SUM(CASE WHEN tt.naturaleza = 'INGRESO' THEN tf.monto_netto ELSE
0 END) -
SUM(CASE WHEN tt.naturaleza = 'EGRESO' THEN tf.monto_netto ELSE 0
END), 2
) AS balance_dia
FROM TransaccionFinanciera tf
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
WHERE tf.fecha_proceso IS NOT NULL
AND et.nombre_estado = 'COMPLETADA'
GROUP BY CAST(tf.fecha_proceso AS DATE)
ORDER BY fecha DESC;

```

Consulta 12: Locales con mayor volumen de apuestas

Identifica los locales con mayor cantidad de apuestas y mayor monto total apostado.

Importancia:

Permite comparar el rendimiento operativo de cada local.

```

SELECT
l.id_local,
l.nombre_local,
l.districto,
l.ciudad,
COUNT(a.id_apuesta) AS cantidad_apuestas,
SUM(a.monto_apostado) AS monto_total_apostado,
ROUND(AVG(a.monto_apostado), 2) AS promedio_apuesta
FROM Local l
INNER JOIN Apuesta a ON l.id_local = a.id_local
WHERE l.estado = 1
GROUP BY l.id_local, l.nombre_local, l.districto, l.ciudad
HAVING COUNT(a.id_apuesta) >= 1
ORDER BY monto_total_apostado DESC;

```

Consulta 13: Trabajadores asignados por local y rol

Lista los trabajadores asignados a cada local, incluyendo su rol y estado de asignación.

Importancia:

Permite controlar qué trabajadores operan en cada sede y verificar responsabilidades internas.

```

SELECT
l.nombre_local,
l.districto,
CONCAT(p.nombres, ' ', p.apellidos) AS trabajador,

```

```

r.nombre_rol,
txl.fecha_asignacion,
CASE
    WHEN txl.estado = 1 THEN 'ACTIVO'
    ELSE 'INACTIVO'
END AS estado_asignacion
FROM TrabajadorxLocal txl
INNER JOIN Trabajador t ON txl.id_trabajador = t.id_trabajador
INNER JOIN Persona p ON t.id_persona = p.id_persona
INNER JOIN Rol r ON t.id_rol = r.id_rol
INNER JOIN Local l ON txl.id_local = l.id_local
WHERE txl.estado = 1
ORDER BY l.nombre_local, r.nombre_rol, trabajador;

```

Consulta 14: Deportes y ligas con mayor actividad de apuestas

Agrupar las apuestas por deporte y liga, mostrando cantidad de eventos y apuestas asociadas.

Importancia:

Permite identificar qué deportes o competiciones generan mayor interés.

```

SELECT
    d.nombre_deporte,
    l.nombre_liga,
    COUNT(DISTINCT e.id_evento) AS cantidad_eventos,
    COUNT(DISTINCT a.id_apuesta) AS cantidad_apuestas,
    ROUND(AVG(da.valor_cuota), 2) AS cuota_promedio
FROM Deporte d
INNER JOIN Liga l ON d.id_deporte = l.id_deporte
INNER JOIN Evento e ON l.id_liga = e.id_liga
INNER JOIN MercadoApuesta ma ON e.id_evento = ma.id_evento
INNER JOIN Cuota c ON ma.id_mercado = c.id_mercado
INNER JOIN DetalleApuesta da ON c.id_cuota = da.id_cuota
INNER JOIN Apuesta a ON da.id_apuesta = a.id_apuesta
WHERE d.estado = 1
GROUP BY d.nombre_deporte, l.nombre_liga
HAVING COUNT(DISTINCT a.id_apuesta) > 0
ORDER BY cantidad_apuestas DESC;

```

Consulta 15: Comprobantes emitidos por método de pago

Muestra comprobantes emitidos, relacionándolos con la transacción financiera y el método de pago utilizado.

Importancia:

Facilita la auditoría de operaciones económicas y permite validar la documentación asociada.

```

SELECT
    c.id_comprobante,
    c.numero_comprobante,
    c.fecha_emision,
    tf.id_transaccion,
    tt.nombre_tipo AS tipo_transaccion,

```

```

        mp.nombre_metodo AS metodo_pago,
        tf.monto_total,
        m.codigo_moneda,
        et.nombre_estado AS estado_transaccion
FROM Comprobante c
INNER JOIN TransaccionFinanciera tf ON c.id_transaccion =
tf.id_transaccion
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN MetodoPago mp ON tf.id_metodo_pago = mp.id_metodo_pago
INNER JOIN Moneda m ON tf.id_moneda = m.id_moneda
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
WHERE c.numero_comprobante LIKE 'COMP%'
      AND et.nombre_estado IN ('COMPLETADA', 'PENDIENTE')
ORDER BY c.fecha_emision DESC;

```

4.4.1.2. Procedimientos almacenados

A continuación, se presentan procedimientos almacenados orientados a la gestión y análisis de información en BetCore DB. Estos procedimientos permiten consultar apuestas, transacciones, saldos, eventos deportivos, comprobantes y trabajadores, aportando valor al negocio mediante reportes reutilizables, filtros por parámetros y control de información operativa y financiera.

Procedimiento 1: Listar apuestas ganadas

Lista las apuestas que se encuentran en estado ganado, mostrando el cliente, código de ticket, monto apostado, cuota total y premio potencial.

Importancia:

Permite identificar rápidamente las apuestas ganadas y estimar los pagos que podrían corresponder a los usuarios.

```

CREATE OR ALTER PROCEDURE dbo.usp_ListarApuestasGanadas
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.id_apuesta,
        a.codigo_ticket,
        CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
        a.tipo_apuesta,
        a.monto_apostado,
        a.cuota_total,
        ROUND(a.monto_apostado * a.cuota_total, 2) AS premio_potencial,
        a.fecha_apuesta
    FROM Apuesta a
    INNER JOIN Usuario u ON a.id_usuario = u.id_usuario
    INNER JOIN Persona p ON u.id_persona = p.id_persona
    INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta

```

```

        WHERE ea.nombre_estado = 'GANADA'
        ORDER BY a.fecha_apuesta DESC;
END;
GO

EXEC dbo.usp_ListarApuestasGanadas;
GO

```

Procedimiento 2: Consultar apuestas entre fechas

Consulta las apuestas registradas dentro de un rango de fechas determinado.

Importancia:

Facilita el análisis de apuestas por periodos específicos, útil para reportes diarios, semanales o mensuales.

```

CREATE OR ALTER PROCEDURE dbo.usp_ConsultarApuestasPorFechas
    @fecha_inicio DATETIME,
    @fecha_fin DATETIME
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.id_apuesta,
        a.codigo_ticket,
        CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
        ea.nombre_estado AS estado_apuesta,
        a.tipo_apuesta,
        a.monto_apostado,
        a.cuota_total,
        a.fecha_apuesta
    FROM Apuesta a
    INNER JOIN Usuario u ON a.id_usuario = u.id_usuario
    INNER JOIN Persona p ON u.id_persona = p.id_persona
    INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta
    WHERE a.fecha_apuesta >= @fecha_inicio
        AND a.fecha_apuesta < DATEADD(DAY, 1, @fecha_fin)
    ORDER BY a.fecha_apuesta DESC;
END;
GO

EXEC dbo.usp_ConsultarApuestasPorFechas
    @fecha_inicio = '2024-01-01',
    @fecha_fin = '2024-12-31';
GO

```

Procedimiento 3: Resumen de apuestas por estado

Agrupar las apuestas según su estado y muestra cantidad, monto total, monto promedio, monto máximo y monto mínimo.

Importancia:

Ayuda a conocer la distribución de apuestas pendientes, ganadas, perdidas, anuladas o canceladas.

```

CREATE OR ALTER PROCEDURE dbo.usp_ResumenApuestasPorEstado
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        ea.nombre_estado AS estado_apuesta,
        COUNT(a.id_apuesta) AS cantidad_apuestas,
        SUM(a.monto_apostado) AS monto_total_apostado,
        ROUND(AVG(a.monto_apostado), 2) AS monto_promedio,
        MAX(a.monto_apostado) AS monto_maximo,
        MIN(a.monto_apostado) AS monto_minimo
    FROM Apuesta a
    INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
    ea.id_estado_apuesta
    GROUP BY ea.nombre_estado
    ORDER BY monto_total_apostado DESC;
END;
GO

EXEC dbo.usp_ResumenApuestasPorEstado
GO

```

Procedimiento 4: Clientes con mayor monto apostado

Muestra los clientes que acumulan mayor monto apostado, considerando cantidad de apuestas y promedio apostado.

Importancia:

Permite identificar clientes con alta actividad y analizar posibles estrategias comerciales o de control.

```

CREATE OR ALTER PROCEDURE dbo.usp_ClientesMayorMontoApostado
    @top_clientes INT = 10
AS
BEGIN
    SET NOCOUNT ON;

    SELECT TOP (@top_clientes)
        u.id_usuario,
        CONCAT(p.nombres, ' ', p.apellidos) AS cliente,
        COUNT(a.id_apuesta) AS total_apuestas,
        SUM(a.monto_apostado) AS monto_total_apostado,
        ROUND(AVG(a.monto_apostado), 2) AS promedio_por_apuesta,
        MAX(a.monto_apostado) AS mayor_apuesta,
        MIN(a.monto_apostado) AS menor_apuesta
    FROM Usuario u
    INNER JOIN Persona p ON u.id_persona = p.id_persona
    INNER JOIN Apuesta a ON u.id_usuario = a.id_usuario
    GROUP BY u.id_usuario, p.nombres, p.apellidos
    HAVING SUM(a.monto_apostado) > 0
    ORDER BY monto_total_apostado DESC;
END;
GO

EXEC dbo.usp_ClientesMayorMontoApostado
    @top_clientes = 1
GO

```

Procedimiento 5: Consultar detalle de apuestas por usuario

Muestra las apuestas de un usuario específico, incluyendo ticket, estado, monto, cuota y local.

Importancia:

Facilita la atención de reclamos o consultas de clientes sobre su historial de apuestas.

```
CREATE OR ALTER PROCEDURE dbo.usp_DetalleApuestasPorUsuario
    @id_usuario INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.id_apuesta,
        a.codigo_ticket,
        ea.nombre_estado AS estado_apuesta,
        a.tipo_apuesta,
        a.monto_apostado,
        a.cuota_total,
        ROUND(a.monto_apostado * a.cuota_total, 2) AS premio_potencial,
        ISNULL(lo.nombre_local, 'ONLINE') AS local_registro,
        a.fecha_apuesta
    FROM Apuesta a
    INNER JOIN EstadoApuesta ea ON a.id_estado_apuesta =
ea.id_estado_apuesta
    LEFT JOIN Local lo ON a.id_local = lo.id_local
    WHERE a.id_usuario = @id_usuario
    ORDER BY a.fecha_apuesta DESC;
END;
GO

EXEC dbo.usp_DetalleApuestasPorUsuario
    @id_usuario = 44
GO
```

Procedimiento 6: Eventos con mayor actividad de apuestas

Lista los eventos deportivos con mayor cantidad de apuestas asociadas.

Importancia:

Permite identificar qué partidos o competencias generan mayor interés y volumen de operación.

```
CREATE OR ALTER PROCEDURE dbo.usp_EventosConMayorActividad
    @top_eventos INT = 10
AS
BEGIN
    SET NOCOUNT ON;

    SELECT TOP (@top_eventos)
        e.id_evento,
```

```

        e.nombre_evento,
        d.nombre_deporte,
        li.nombre_liga,
        COUNT(DISTINCT a.id_apuesta) AS cantidad_apuestas,
        ROUND(AVG(da.valor_cuota), 2) AS cuota_promedio
    FROM Apuesta a
    INNER JOIN DetalleApuesta da ON a.id_apuesta = da.id_apuesta
    INNER JOIN Cuota c ON da.id_cuota = c.id_cuota
    INNER JOIN MercadoApuesta ma ON c.id_mercado = ma.id_mercado
    INNER JOIN Evento e ON ma.id_evento = e.id_evento
    INNER JOIN Liga li ON e.id_liga = li.id_liga
    INNER JOIN Deporte d ON li.id_deporte = d.id_deporte
    GROUP BY e.id_evento, e.nombre_evento, d.nombre_deporte,
    li.nombre_liga
    HAVING COUNT(DISTINCT a.id_apuesta) > 0
    ORDER BY cantidad_apuestas DESC;
END;
GO

EXEC dbo.usp_EventosConMayorActividad
    @top_eventos = 1;
GO

```

Procedimiento 7: Consultar cuotas activas por evento

Consulta las cuotas activas de eventos cuyo nombre coincida con un texto ingresado.

Importancia:

Permite revisar rápidamente las opciones de apuesta disponibles para eventos relevantes.

```

CREATE OR ALTER PROCEDURE dbo.usp_CuotasActivasPorEvento
    @nombre_evento VARCHAR(150)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        e.nombre_evento,
        ma.nombre_mercado,
        c.nombre_opcion,
        c.valor AS valor_cuota,
        c.estado AS estado_cuota
    FROM Evento e
    INNER JOIN MercadoApuesta ma ON e.id_evento = ma.id_evento
    INNER JOIN Cuota c ON ma.id_mercado = c.id_mercado
    WHERE e.nombre_evento LIKE '%' + @nombre_evento + '%'
        AND c.estado = 'ACTIVA'
    ORDER BY e.nombre_evento, ma.nombre_mercado, c.valor DESC;
END;
GO

EXEC dbo.usp_CuotasActivasPorEvento
    @nombre_evento = 'Alianza'
GO

```

Procedimiento 8: Resumen de transacciones por tipo y estado

Agrupar las transacciones financieras según tipo y estado, mostrando cantidad, monto total y promedio.

Importancia:

Ayuda a controlar depósitos, retiros, premios, devoluciones, comisiones y apuestas desde el punto de vista financiero.

```
CREATE OR ALTER PROCEDURE dbo.usp_ResumenTransaccionesPorTipoEstado
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        tt.nombre_tipo AS tipo_transaccion,
        et.nombre_estado AS estado_transaccion,
        COUNT(tf.id_transaccion) AS cantidad_transacciones,
        SUM(tf.monto_total) AS monto_total,
        ROUND(AVG(tf.monto_total), 2) AS monto_promedio
    FROM TransaccionFinanciera tf
    INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
    INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
    GROUP BY tt.nombre_tipo, et.nombre_estado
    ORDER BY monto_total DESC;
END;
GO

EXEC dbo.usp_ResumenTransaccionesPorTipoEstado
GO
```

Procedimiento 9: Transacciones pendientes por monto mínimo

Lista las transacciones pendientes que superan un monto mínimo definido por el usuario.

Importancia:

Permite priorizar operaciones financieras pendientes que requieren validación o seguimiento.

```
CREATE OR ALTER PROCEDURE dbo.usp_TransaccionesPendientesPorMonto
    @monto_minimo DECIMAL(12,2) = 100.00
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        tf.id_transaccion,
        CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
        tt.nombre_tipo AS tipo_transaccion,
        mp.nombre_metodo AS metodo_pago,
        tf.monto_total,
        m.codigo_moneda,
        tf.fecha_solicitud,
        et.nombre_estado AS estado_transaccion
    FROM TransaccionFinanciera tf
```

```

INNER JOIN Usuario u ON tf.id_usuario = u.id_usuario
INNER JOIN Persona p ON u.id_persona = p.id_persona
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
INNER JOIN MetodoPago mp ON tf.id_metodo_pago = mp.id_metodo_pago
INNER JOIN Moneda m ON tf.id_moneda = m.id_moneda
WHERE et.nombre_estado = 'PENDIENTE'
AND tf.monto_total >= @monto_minimo
ORDER BY tf.monto_total DESC, tf.fecha_solicitud ASC;
END;
GO

EXEC dbo.usp_TransaccionesPendientesPorMonto
GO

```

Procedimiento 10: Consultar saldo por usuario

Consulta el saldo disponible, retenido y total de un usuario determinado.

Importancia:

Permite verificar los fondos del cliente antes de procesar apuestas, retiros o pagos.

```

CREATE OR ALTER PROCEDURE dbo.usp_ConsultarSaldoPorUsuario
    @id_usuario INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        u.id_usuario,
        CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
        m.codigo_moneda,
        s.monto_disponible,
        s.monto_retenido,
        s.monto_total,
        s.ultima_actualizacion
    FROM Saldo s
    INNER JOIN Usuario u ON s.id_usuario = u.id_usuario
    INNER JOIN Persona p ON u.id_persona = p.id_persona
    INNER JOIN Moneda m ON s.id_moneda = m.id_moneda
    WHERE u.id_usuario = @id_usuario
    ORDER BY m.codigo_moneda;
END;
GO

EXEC dbo.usp_ConsultarSaldoPorUsuario
    @id_usuario = 20
GO

```

Procedimiento 11: Historial de saldo por usuario

Muestra los cambios de saldo de un usuario, indicando saldo anterior, saldo posterior y variación.

Importancia:

Permite auditar movimientos de dinero y resolver inconsistencias financieras.

```
CREATE OR ALTER PROCEDURE dbo.usp_HistorialSaldoPorUsuario
    @id_usuario INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        hs.id_historial_saldo,
        CONCAT(p.nombres, ' ', p.apellidos) AS usuario,
        hs.saldo_anterior,
        hs.saldo_posterior,
        ROUND(hs.saldo_posterior - hs.saldo_anterior, 2) AS
    variacion_saldo,
        hs.fecha_registro,
        tf.id_transaccion
    FROM HistorialSaldo hs
    INNER JOIN Saldo s ON hs.id_saldo = s.id_saldo
    INNER JOIN Usuario u ON s.id_usuario = u.id_usuario
    INNER JOIN Persona p ON u.id_persona = p.id_persona
    INNER JOIN TransaccionFinanciera tf ON hs.id_transaccion =
    tf.id_transaccion
    WHERE u.id_usuario = @id_usuario
    ORDER BY hs.fecha_registro DESC;
END;
GO

EXEC dbo.usp_ConsultarSaldoPorUsuario
    @id_usuario = 30
GO
```

Procedimiento 12: Balance financiero diario por rango de fechas

Calcula ingresos, egresos y balance diario de transacciones completadas dentro de un rango de fechas.

Importancia:

Permite evaluar el flujo financiero del negocio y detectar días con mayores ingresos o egresos.

```
CREATE OR ALTER PROCEDURE dbo.usp_BalanceDiarioPorRangoFechas
    @fecha_inicio DATE,
    @fecha_fin DATE
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        CAST(tf.fecha_proceso AS DATE) AS fecha,
        SUM(CASE WHEN tt.naturaleza = 'INGRESO' THEN tf.monto_netto ELSE
0 END) AS total_ingresos,
        SUM(CASE WHEN tt.naturaleza = 'EGRESO' THEN tf.monto_netto ELSE 0
END) AS total_egresos,
        ROUND(
            SUM(CASE WHEN tt.naturaleza = 'INGRESO' THEN tf.monto_netto
ELSE 0 END) -
```

```

        SUM(CASE WHEN tt.naturaleza = 'EGRESO' THEN tf.monto_netos
ELSE 0 END), 2
    ) AS balance_dia
FROM TransaccionFinanciera tf
INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
WHERE tf.fecha_proceso >= @fecha_inicio
      AND tf.fecha_proceso < DATEADD(DAY, 1, @fecha_fin)
      AND et.nombre_estado = 'COMPLETADA'
GROUP BY CAST(tf.fecha_proceso AS DATE)
ORDER BY fecha DESC;
END;
GO

EXEC dbo.usp_BalanceDiarioPorRangoFechas
    @fecha_inicio = '2024-01-01',
    @fecha_fin = '2024-12-31';
GO

```

Procedimiento 13: Comprobantes emitidos por rango de fechas

Listas comprobantes emitidos dentro de un rango de fechas, asociados a transacciones financieras.

Importancia:

Facilita la auditoría documental de pagos, depósitos, retiros o premios.

```

CREATE OR ALTER PROCEDURE dbo.usp_ComprobantesPorRangoFechas
    @fecha_inicio DATE,
    @fecha_fin DATE
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        c.id_comprobante,
        c.numero_comprobante,
        c.fecha_emision,
        tf.id_transaccion,
        tt.nombre_tipo AS tipo_transaccion,
        mp.nombre_metodo AS metodo_pago,
        tf.monto_total,
        m.codigo_moneda,
        et.nombre_estado AS estado_transaccion
    FROM Comprobante c
    INNER JOIN TransaccionFinanciera tf ON c.id_transaccion =
tf.id_transaccion
    INNER JOIN TipoTransaccion tt ON tf.id_tipo_transaccion =
tt.id_tipo_transaccion
    INNER JOIN MetodoPago mp ON tf.id_metodo_pago = mp.id_metodo_pago
    INNER JOIN Moneda m ON tf.id_moneda = m.id_moneda
    INNER JOIN EstadoTransaccion et ON tf.id_estado = et.id_estado
    WHERE c.fecha_emision >= @fecha_inicio
          AND c.fecha_emision < DATEADD(DAY, 1, @fecha_fin)
    ORDER BY c.fecha_emision DESC;
END;
GO

```

```
EXEC dbo.usp_ComprobantesPorRangoFechas
    @fecha_inicio = '2024-01-01',
    @fecha_fin     = '2024-12-31';
GO
```

Procedimiento 14: Trabajadores asignados a un local

Lista los trabajadores asignados a un local específico, incluyendo su rol y estado de asignación.

Importancia:

Permite controlar qué trabajadores operan en cada sede y verificar responsabilidades internas.

```
CREATE OR ALTER PROCEDURE dbo.usp_TrabajadoresPorLocal
    @id_local INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        lo.id_local,
        lo.nombre_local,
        lo.distrrito,
        CONCAT(p.nombres, ' ', p.apellidos) AS trabajador,
        r.nombre_rol,
        txl.fecha_asignacion,
        CASE
            WHEN txl.estado = 1 THEN 'ACTIVO'
            ELSE 'INACTIVO'
        END AS estado_asignacion
    FROM TrabajadorxLocal txl
    INNER JOIN Trabajador t ON txl.id_trabajador = t.id_trabajador
    INNER JOIN Persona p ON t.id_persona = p.id_persona
    INNER JOIN Rol r ON t.id_rol = r.id_rol
    INNER JOIN Local lo ON txl.id_local = lo.id_local
    WHERE lo.id_local = @id_local
    ORDER BY r.nombre_rol, trabajador;
END;
GO

EXEC dbo.usp_TrabajadoresPorLocal
    @id_local = 3
GO
```

Procedimiento 15: Retornar cantidad y monto total de apuestas por año

Calcula la cantidad de apuestas y el monto total apostado en un año determinado, devolviendo ambos resultados mediante parámetros de salida.

Importancia:

Permite obtener indicadores anuales de operación y comparar el comportamiento del negocio por periodo.

```
CREATE OR ALTER PROCEDURE dbo.usp_ResumenApuestasPorAnio
```

```

        @anio INT,
        @cantidad_apuestas INT OUTPUT,
        @monto_total DECIMAL(12,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @cantidad_apuestas = COUNT(*),
        @monto_total = ISNULL(SUM(monto_apostado), 0)
    FROM Apuesta
    WHERE YEAR(fecha_apuesta) = @anio;
END;
GO

DECLARE @cantidad INT;
DECLARE @monto DECIMAL(12,2);

EXEC dbo.usp_ResumenApuestasPorAnio
    @anio = 2024,
    @cantidad_apuestas = @cantidad OUTPUT,
    @monto_total = @monto OUTPUT;

-- Ver los resultados
SELECT
    @cantidad AS cantidad_apuestas,
    @monto AS monto_total;
GO

```

4.4.1.3. Funciones

A continuación, se presentan funciones SQL orientadas a obtener indicadores útiles para la gestión de BetCore DB. Estas funciones permiten calcular información relacionada con apuestas, saldos, transacciones, cuotas, comprobantes y comportamiento financiero del negocio.

Función 1: Cantidad de apuestas por estado

Retorna la cantidad de apuestas registradas según un estado específico, como GANADA, PERDIDA, PENDIENTE o ANULADA.

Importancia:

Permite conocer rápidamente cuántas apuestas se encuentran en cada estado, facilitando el seguimiento operativo.

```

CREATE OR ALTER FUNCTION dbo.fn_CantidadApuestasPorEstado
(
    @estado VARCHAR(30)
)
RETURNS INT
AS
BEGIN
    DECLARE @cantidad INT;

    SELECT @cantidad = COUNT(*)

```

```

FROM Apuesta a
INNER JOIN EstadoApuesta ea
    ON a.id_estado_apuesta = ea.id_estado_apuesta
WHERE ea.nombre_estado = @estado;

RETURN ISNULL(@cantidad, 0);
END;
GO

PRINT 'Cantidad de apuestas ganadas: ' +
CAST(dbo.fn_CantidadApuestasPorEstado('GANADA') AS VARCHAR(20));
GO

```

Función 2: Total apostado por moneda

Calcula el monto total apostado según el código de moneda ingresado, por ejemplo, PEN, USD o EUR.

Importancia:

Ayuda a conocer el volumen económico apostado por tipo de moneda, útil para control financiero.

```

CREATE OR ALTER FUNCTION dbo.fn_TotalApostadoPorMoneda
(
    @codigo_moneda VARCHAR(10)
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @total DECIMAL(12,2);

    SELECT @total = SUM(a.monto_apostado)
    FROM Apuesta a
    INNER JOIN Moneda m
        ON a.id_moneda = m.id_moneda
    WHERE m.codigo_moneda = @codigo_moneda;

    RETURN ISNULL(@total, 0);
END;
GO

PRINT 'Total apostado en PEN: ' +
CAST(dbo.fn_TotalApostadoPorMoneda('PEN') AS VARCHAR(30));
GO

```

Función 3: Promedio de monto apostado por tipo de apuesta

Calcula el monto promedio apostado según el tipo de apuesta, como SIMPLE o COMBINADA.

Importancia:

Permite identificar cuánto suelen apostar los usuarios según la modalidad de apuesta.

```

CREATE OR ALTER FUNCTION dbo.fn_PromedioMontoPorTipoApuesta
(
    @tipo_apuesta VARCHAR(30)
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @promedio DECIMAL(12,2);

    SELECT @promedio = AVG(monto_apostado)
    FROM Apuesta
    WHERE tipo_apuesta = @tipo_apuesta;

    RETURN ISNULL(@promedio, 0);
END;
GO

PRINT 'Promedio de apuesta SIMPLE: ' +
CAST(dbo.fn_PromedioMontoPorTipoApuesta('SIMPLE') AS VARCHAR(30));
GO

```

Función 4: Premio potencial de una apuesta

Calcula el premio potencial de una apuesta específica multiplicando el monto apostado por la cuota total.

Importancia:

Permite estimar el monto que podría pagarse al cliente si la apuesta resulta ganadora.

```

CREATE OR ALTER FUNCTION dbo.fn_PremioPotencialApuesta
(
    @id_apuesta INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @premio DECIMAL(12,2);

    SELECT @premio = ROUND(monto_apostado * cuota_total, 2)
    FROM Apuesta
    WHERE id_apuesta = @id_apuesta;

    RETURN ISNULL(@premio, 0);
END;
GO

PRINT 'Premio potencial de la apuesta 1: ' +
CAST(dbo.fn_PremioPotencialApuesta(1) AS VARCHAR(30));
GO

```

Función 5: Saldo disponible de un usuario

Retorna el saldo disponible de un usuario específico.

Importancia:

Permite verificar si un usuario cuenta con fondos disponibles para realizar apuestas o retiros.

```
CREATE OR ALTER FUNCTION dbo.fn_SaldoDisponibleUsuario
(
    @id_usuario INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @saldo DECIMAL(12,2);

    SELECT @saldo = SUM(monto_disponible)
    FROM Saldo
    WHERE id_usuario = @id_usuario;

    RETURN ISNULL(@saldo, 0);
END;
GO

PRINT 'Saldo disponible del usuario 1: ' +
CAST(dbo.fn_SaldoDisponibleUsuario(1) AS VARCHAR(30));
GO
```

Función 6: Saldo retenido de un usuario

Retorna el monto retenido asociado a un usuario.

Importancia:

Permite controlar fondos bloqueados por operaciones pendientes, como apuestas activas o transacciones en proceso.

```
CREATE OR ALTER FUNCTION dbo.fn_SaldoRetenidoUsuario
(
    @id_usuario INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @saldo_retenido DECIMAL(12,2);

    SELECT @saldo_retenido = SUM(monto_retenido)
    FROM Saldo
    WHERE id_usuario = @id_usuario;

    RETURN ISNULL(@saldo_retenido, 0);
END;
GO

PRINT 'Saldo retenido del usuario 2: ' +
CAST(dbo.fn_SaldoRetenidoUsuario(2) AS VARCHAR(30));
GO
```

Función 7: Total de transacciones por tipo

Calcula el monto total de transacciones según el tipo indicado, por ejemplo, DEPOSITO, RETIRO, APUESTA o PREMIO.

Importancia:

Ayuda a controlar los movimientos financieros según su clasificación.

```
CREATE OR ALTER FUNCTION dbo.fn_TotalTransaccionesPorTipo
(
    @tipo_transaccion VARCHAR(50)
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @total DECIMAL(12,2);

    SELECT @total = SUM(tf.monto_total)
    FROM TransaccionFinanciera tf
    INNER JOIN TipoTransaccion tt
        ON tf.id_tipo_transaccion = tt.id_tipo_transaccion
    WHERE tt.nombre_tipo = @tipo_transaccion;

    RETURN ISNULL(@total, 0);
END;
GO

PRINT 'Total de transacciones tipo DEPOSITO: ' +
CAST(dbo.fn_TotalTransaccionesPorTipo('DEPOSITO') AS VARCHAR(30));
GO
```

Función 8: Cantidad de transacciones por estado

Retorna la cantidad de transacciones financieras según su estado, como COMPLETADA, PENDIENTE o FALLIDA.

Importancia:

Permite monitorear operaciones financieras pendientes o fallidas que requieren atención.

```
CREATE OR ALTER FUNCTION dbo.fn_CantidadTransaccionesPorEstado
(
    @estado VARCHAR(30)
)
RETURNS INT
AS
BEGIN
    DECLARE @cantidad INT;

    SELECT @cantidad = COUNT(*)
    FROM TransaccionFinanciera tf
    INNER JOIN EstadoTransaccion et
        ON tf.id_estado = et.id_estado
    WHERE et.nombre_estado = @estado;

    RETURN ISNULL(@cantidad, 0);
END;
GO

PRINT 'Cantidad de transacciones pendientes: ' +
CAST(dbo.fn_CantidadTransaccionesPorEstado('PENDIENTE') AS VARCHAR(20));
GO
```

Función 9: Balance financiero por rango de fechas

Calcula la diferencia entre ingresos y egresos completados dentro de un rango de fechas.

Importancia:

Permite evaluar si el negocio tuvo balance positivo o negativo en un periodo determinado.

```
CREATE OR ALTER FUNCTION dbo.fn_BalanceFinancieroPorFechas
(
    @fecha_inicio DATE,
    @fecha_fin DATE
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @balance DECIMAL(12,2);

    SELECT @balance =
        SUM(CASE WHEN tt.naturaleza = 'INGRESO' THEN tf.monto_netto ELSE
0 END) -
        SUM(CASE WHEN tt.naturaleza = 'EGRESO' THEN tf.monto_netto ELSE 0
END)
    FROM TransaccionFinanciera tf
    INNER JOIN TipoTransaccion tt
        ON tf.id_tipo_transaccion = tt.id_tipo_transaccion
    INNER JOIN EstadoTransaccion et
        ON tf.id_estado = et.id_estado
    WHERE tf.fecha_proceso >= @fecha_inicio
        AND tf.fecha_proceso < DATEADD(DAY, 1, @fecha_fin)
        AND et.nombre_estado = 'COMPLETADA';

    RETURN ISNULL(@balance, 0);
END;
GO

PRINT 'Balance financiero: ' +
CAST(dbo.fn_BalanceFinancieroPorFechas('2024-01-01', '2024-12-31') AS
VARCHAR(30));
GO
```

Función 10: Cantidad de apuestas por año

Retorna la cantidad de apuestas registradas en un año específico.

Importancia:

Permite medir el volumen anual de operaciones y comparar el crecimiento del negocio.

```
CREATE OR ALTER FUNCTION dbo.fn_CantidadApuestasPorAnio
(
    @anio INT
)
RETURNS INT
AS
```

```

BEGIN
    DECLARE @cantidad INT;

    SELECT @cantidad = COUNT(*)
    FROM Apuesta
    WHERE YEAR(fecha_apuesta) = @anio;

    RETURN ISNULL(@cantidad, 0);
END;
GO

PRINT 'Cantidad de apuestas en 2024: ' +
CAST(dbo.fn_CantidadApuestasPorAnio(2024) AS VARCHAR(20));
GO

```

Función 11: Monto máximo apostado por usuario

Retorna el mayor monto apostado por un usuario específico.

Importancia:

Ayuda a identificar el nivel máximo de exposición o actividad económica de un cliente.

```

CREATE OR ALTER FUNCTION dbo.fn_MontoMaximoApostadoUsuario
(
    @id_usuario INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @maximo DECIMAL(12,2);

    SELECT @maximo = MAX(monto_apostado)
    FROM Apuesta
    WHERE id_usuario = @id_usuario;

    RETURN ISNULL(@maximo, 0);
END;
GO

PRINT 'Mayor apuesta del usuario 1: ' +
CAST(dbo.fn_MontoMaximoApostadoUsuario(1) AS VARCHAR(30));
GO

```

Función 12: Cantidad de cuotas activas por evento

Retorna la cantidad de cuotas activas asociadas a un evento deportivo.

Importancia:

Permite verificar si un evento cuenta con opciones disponibles para apostar.

```

CREATE OR ALTER FUNCTION dbo.fn_CuotasActivasPorEvento
(
    @id_evento INT
)
RETURNS INT
AS

```

```

BEGIN
    DECLARE @cantidad INT;

    SELECT @cantidad = COUNT(c.id_cuota)
    FROM Evento e
    INNER JOIN MercadoApuesta ma
        ON e.id_evento = ma.id_evento
    INNER JOIN Cuota c
        ON ma.id_mercado = c.id_mercado
    WHERE e.id_evento = @id_evento
        AND c.estado = 'ACTIVA';

    RETURN ISNULL(@cantidad, 0);
END;
GO

PRINT 'Cuotas activas del evento 6: ' +
CAST(dbo.fn_CuotasActivasPorEvento(6) AS VARCHAR(20));
GO

```

Función 13: Cuota promedio por evento

Calcula el valor promedio de las cuotas asociadas a un evento deportivo.

Importancia:

Permite analizar el comportamiento de las cuotas ofrecidas en cada evento.

```

CREATE OR ALTER FUNCTION dbo.fn_CuotaPromedioPorEvento
(
    @id_evento INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @promedio DECIMAL(12,2);

    SELECT @promedio = AVG(c.valor)
    FROM Evento e
    INNER JOIN MercadoApuesta ma
        ON e.id_evento = ma.id_evento
    INNER JOIN Cuota c
        ON ma.id_mercado = c.id_mercado
    WHERE e.id_evento = @id_evento;

    RETURN ISNULL(@promedio, 0);
END;
GO

PRINT 'Cuota promedio del evento 1: ' +
CAST(dbo.fn_CuotaPromedioPorEvento(1) AS VARCHAR(30));
GO

```

Función 14: Cantidad de comprobantes emitidos por año

Retorna la cantidad de comprobantes emitidos durante un año específico.

Importancia:

Permite controlar la emisión documental asociada a transacciones financieras.

```

CREATE OR ALTER FUNCTION dbo.fn_ComprobantesEmitidosPorAnio
(
    @anio INT
)
RETURNS INT
AS
BEGIN
    DECLARE @cantidad INT;

    SELECT @cantidad = COUNT(*)
    FROM Comprobante
    WHERE YEAR(fecha_emision) = @anio;

    RETURN ISNULL(@cantidad, 0);
END;
GO

PRINT 'Comprobantes emitidos en 2024: ' +
CAST(dbo.fn_ComprobantesEmitidosPorAnio(2024) AS VARCHAR(20));
GO

```

Función 15: Nivel de riesgo de una apuesta

Clasifica una apuesta según su monto apostado en BAJO, MEDIO o ALTO.

Importancia:

Permite identificar operaciones que podrían requerir mayor supervisión por su impacto económico.

```

CREATE OR ALTER FUNCTION dbo.fn_NivelRiesgoApuesta
(
    @id_apuesta INT
)
RETURNS VARCHAR(20)
AS
BEGIN
    DECLARE @monto DECIMAL(12,2);
    DECLARE @nivel VARCHAR(20);

    SELECT @monto = monto_apostado
    FROM Apuesta
    WHERE id_apuesta = @id_apuesta;

    SET @nivel =
        CASE
            WHEN @monto IS NULL THEN 'NO EXISTE'
            WHEN @monto >= 500 THEN 'ALTO'
            WHEN @monto >= 100 THEN 'MEDIO'
            ELSE 'BAJO'
        END;

    RETURN @nivel;
END;
GO

PRINT 'Nivel de riesgo de la apuesta 1: ' +
dbo.fn_NivelRiesgoApuesta(1);
GO

```

4.4.1.4. Triggers

A continuación, se presentan triggers diseñados para reforzar reglas de negocio, validar datos críticos, proteger operaciones sensibles y mantener la trazabilidad de información en BetCore DB. Estos triggers se ejecutan automáticamente ante eventos de inserción, actualización o eliminación en tablas relevantes de la base de datos.

Trigger 1: Validar mayoría de edad y correo de la persona

Valida que una persona registrada no tenga fecha de nacimiento futura y, cuando se registre un correo, que tenga un formato básico válido.

Importancia:

Ayuda a mantener datos confiables de clientes y trabajadores. En el contexto de apuestas deportivas, la edad es un dato relevante porque permite verificar que el usuario cumpla con la mayoría de edad.

```
CREATE OR ALTER TRIGGER dbo.trg_Persona_ValidarEdadCorreo
ON dbo.Persona
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted
            WHERE fecha_nacimiento > CAST(GETDATE() AS DATE)
        )
        BEGIN
            THROW 50001, 'La fecha de nacimiento no puede ser mayor a la
fecha actual.', 1;
        END;

        IF EXISTS (
            SELECT 1
            FROM inserted
            WHERE correo IS NOT NULL
            AND correo NOT LIKE '%@%.%'
        )
        BEGIN
            THROW 50002, 'El correo registrado no tiene un formato
válido.', 1;
        END;
    END TRY
END TRY
```

```

BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
END;
GO

```

Trigger 2: Validar que el usuario sea mayor de edad

Valida que una persona asociada a un usuario tenga al menos 18 años al momento de registrarse o actualizarse.

Importancia:

Evita registrar usuarios apostadores menores de edad, reduciendo riesgos legales y operativos para la casa de apuestas.

```

CREATE OR ALTER TRIGGER dbo.trg_Usuario_ValidarMayorEdad
ON dbo.Usuario
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN Persona p ON i.id_persona = p.id_persona
            WHERE DATEDIFF(YEAR, p.fecha_nacimiento, GETDATE()) < 18
        )
        BEGIN
            THROW 50003, 'No se puede registrar un usuario menor de
edad.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 3: Evitar doble asignación activa de trabajador

Evita que un mismo trabajador tenga más de una asignación activa al mismo local.

Importancia:

Permite mantener un control ordenado del personal asignado a cada local y evita duplicidades en la gestión operativa.

```

CREATE OR ALTER TRIGGER
dbo.trg_TrabajadorxLocal_EvitarAsignacionDuplicada
ON dbo.TrabajadorxLocal
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT txl.id_trabajador, txl.id_local
            FROM TrabajadorxLocal txl
            WHERE txl.estado = 1
            GROUP BY txl.id_trabajador, txl.id_local
            HAVING COUNT(*) > 1
        )
        BEGIN
            THROW 50004, 'El trabajador ya tiene una asignación activa
en este local.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 4: Evitar eliminación de locales con operaciones

Impide eliminar un local si tiene apuestas registradas o trabajadores asignados.

Importancia:

Protege la trazabilidad histórica de las operaciones realizadas en cada establecimiento.

```

CREATE OR ALTER TRIGGER dbo.trg_Local_PrevenirEliminacionConOperaciones
ON dbo.Local
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM deleted d
            WHERE EXISTS (
                SELECT 1
                FROM Apuesta a
                WHERE a.id_local = d.id_local
            )
        )
        OR EXISTS (
            SELECT 1
            FROM TrabajadorxLocal txl
            WHERE txl.id_local = d.id_local
        )
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

```

        )
    )
    BEGIN
        THROW 50005, 'No se puede eliminar un local con apuestas o
trabajadores asociados.', 1;
    END;

    DELETE l
    FROM Local l
    INNER JOIN deleted d ON l.id_local = d.id_local;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
END;
GO

```

Trigger 5: Validar monto y cuota de una apuesta

Valida que el monto apostado sea mayor que cero y que la cuota total sea mayor o igual a 1.

Importancia:

Evita registrar apuestas inválidas que puedan generar errores financieros o inconsistencias en el cálculo de premios.

```

CREATE OR ALTER TRIGGER dbo.trg_Apuesta_ValidarMontoCuota
ON dbo.Apuesta
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted
            WHERE monto_apostado <= 0
                OR cuota_total < 1
        )
        BEGIN
            THROW 50006, 'El monto apostado debe ser mayor a cero y la
cuota total debe ser mayor o igual a 1.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 6: Validar saldo disponible antes de registrar apuesta

Valida que el usuario tenga saldo disponible suficiente para cubrir el monto de la apuesta registrada.

Importancia:

Evita que se registren apuestas sin fondos suficientes, reduciendo riesgos de pérdidas monetarias.

```
CREATE OR ALTER TRIGGER dbo.trg_Apuesta_ValidarSaldoDisponible
ON dbo.Apuesta
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN Saldo s
            ON i.id_usuario = s.id_usuario
            AND i.id_moneda = s.id_moneda
            WHERE i.monto_apostado > s.monto_disponible
        )
        BEGIN
            THROW 50007, 'El usuario no cuenta con saldo disponible
suficiente para registrar la apuesta.', 1;
        END;

        IF EXISTS (
            SELECT 1
            FROM inserted i
            WHERE NOT EXISTS (
                SELECT 1
                FROM Saldo s
                WHERE s.id_usuario = i.id_usuario
                AND s.id_moneda = i.id_moneda
            )
        )
        BEGIN
            THROW 50008, 'El usuario no tiene saldo registrado para la
moneda de la apuesta.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO
```

Trigger 7: Evitar modificación de apuestas cerradas

Impide modificar el monto apostado o la cuota total de una apuesta que ya se encuentra en estado final, como GANADA, PERDIDA o ANULADA.

Importancia:

Protege la integridad de apuestas ya resueltas y evita alteraciones posteriores que puedan afectar pagos o auditorías.

```
CREATE OR ALTER TRIGGER dbo.trg_Apuesta_PrevenirCambioApuestaCerrada
ON dbo.Apuesta
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN deleted d ON i.id_apuesta = d.id_apuesta
            INNER JOIN EstadoApuesta ea ON d.id_estado_apuesta =
ea.id_estado_apuesta
            WHERE ea.nombre_estado IN ('GANADA', 'PERDIDA', 'ANULADA')
            AND (
                i.monto_apostado <> d.monto_apostado
                OR i.cuota_total <> d.cuota_total
            )
        )
        BEGIN
            THROW 50009, 'No se puede modificar el monto o la cuota de
una apuesta cerrada.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO
```

Trigger 8: Validar que la cuota seleccionada esté activa

Valida que la cuota seleccionada en el detalle de una apuesta se encuentre en estado ACTIVA.

Importancia:

Evita que los clientes apuesten sobre cuotas cerradas, suspendidas o no disponibles.

```
CREATE OR ALTER TRIGGER dbo.trg_DetalleApuesta_ValidarCuotaActiva
ON dbo.DetalleApuesta
AFTER INSERT, UPDATE
```

```

AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN Cuota c ON i.id_cuota = c.id_cuota
            WHERE c.estado <> 'ACTIVA'
        )
        BEGIN
            THROW 50010, 'No se puede registrar una apuesta con una
cuota que no se encuentra activa.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 9: Copiar valor actual de cuota en el detalle de apuesta

Copia automáticamente el valor actual de la cuota al campo valor_cuota del detalle de apuesta cuando este no se registra correctamente.

Importancia:

Permite conservar el valor de la cuota al momento de la apuesta, evitando que cambios posteriores en la cuota afecten el historial de la operación.

```

CREATE OR ALTER TRIGGER dbo.trg_DetalleApuesta_CopiarValorCuota
ON dbo.DetalleApuesta
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        UPDATE da
        SET da.valor_cuota = c.valor
        FROM DetalleApuesta da
        INNER JOIN inserted i
            ON da.id_apuesta = i.id_apuesta
            AND da.id_cuota = i.id_cuota
        INNER JOIN Cuota c
            ON i.id_cuota = c.id_cuota
        WHERE i.valor_cuota IS NULL
            OR i.valor_cuota <= 0;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
    END CATCH;
END;

```

```

        THROW;
    END CATCH;
END;
GO

```

Trigger 10: Validar valor de cuota

Valida que toda cuota registrada tenga un valor mayor o igual a 1.

Importancia:

Evita registrar cuotas inválidas que puedan afectar el cálculo de premios y la consistencia de las apuestas.

```

CREATE OR ALTER TRIGGER dbo.trg_Cuota_ValidarValor
ON dbo.Cuota
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted
            WHERE valor < 1
        )
        BEGIN
            THROW 50011, 'El valor de la cuota debe ser mayor o igual a
1.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 11: Evitar eliminación de cuotas usadas en apuestas

Impide eliminar cuotas que ya hayan sido utilizadas en detalles de apuestas.

Importancia:

Mantiene la trazabilidad histórica de las apuestas y evita que se pierda información asociada a operaciones realizadas.

```

CREATE OR ALTER TRIGGER dbo.trg_Cuota_PrevenirEliminacionConApuestas
ON dbo.Cuota
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;

```

```

BEGIN TRY
    IF EXISTS (
        SELECT 1
        FROM deleted d
        INNER JOIN DetalleApuesta da ON d.id_cuota = da.id_cuota
    )
    BEGIN
        THROW 50012, 'No se puede eliminar una cuota que ya fue
utilizada en una apuesta.', 1;
    END;

    DELETE c
    FROM Cuota c
    INNER JOIN deleted d ON c.id_cuota = d.id_cuota;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
END;
GO

```

Trigger 12: Validar montos de transacción financiera

Valida que el monto total de una transacción sea mayor que cero y que el monto neto no sea negativo ni mayor al monto total.

Importancia:

Evita inconsistencias en depósitos, retiros, premios, devoluciones y otros movimientos financieros.

```

CREATE OR ALTER TRIGGER dbo.trg_TransaccionFinanciera_ValidarMontos
ON dbo.TransaccionFinanciera
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted
            WHERE monto_total <= 0
               OR monto_neto < 0
               OR monto_neto > monto_total
        )
        BEGIN
            THROW 50013, 'Los montos de la transacción financiera no son
válidos.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
    END CATCH;
END;

```

```

        THROW;
    END CATCH;
END;
GO

```

Trigger 13: Validar retiro contra saldo disponible

Valida que una transacción de tipo RETIRO no supere el saldo disponible del usuario.

Importancia:

Evita retiros superiores al saldo disponible, protegiendo la consistencia financiera de la base de datos.

```

CREATE OR ALTER TRIGGER dbo.trg_TransaccionFinanciera_ValidarRetiroSaldo
ON dbo.TransaccionFinanciera
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN TipoTransaccion tt
                ON i.id_tipo_transaccion = tt.id_tipo_transaccion
            INNER JOIN Saldo s
                ON i.id_usuario = s.id_usuario
                AND i.id_moneda = s.id_moneda
            WHERE tt.nombre_tipo = 'RETIRO'
                AND i.monto_total > s.monto_disponible
        )
        BEGIN
            THROW 50014, 'El monto del retiro no puede superar el saldo
disponible del usuario.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

Trigger 14: Actualizar saldo e historial por transacción completada

Cuando una transacción cambia o se registra en estado COMPLETADA, actualiza el saldo del usuario según la naturaleza de la transacción (INGRESO o EGRESO) y registra el cambio en HistorialSaldo.

Importancia:

Automatiza el control financiero, mantiene actualizado el saldo del usuario y genera trazabilidad de los movimientos monetarios.

```

CREATE OR ALTER TRIGGER
dbo.trg_TransaccionFinanciera_ActualizarSaldoHistorial
ON dbo.TransaccionFinanciera
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        ;WITH TransaccionesCompletadas AS (
            SELECT
                i.id_transaccion,
                i.id_usuario,
                i.id_moneda,
                i.monto_netto,
                tt.naturaleza,
                CASE
                    WHEN tt.naturaleza = 'INGRESO' THEN i.monto_netto
                    WHEN tt.naturaleza = 'EGRESO' THEN i.monto_netto * -1
                    ELSE 0
                END AS variacion
            FROM inserted i
            INNER JOIN TipoTransaccion tt
                ON i.id_tipo_transaccion = tt.id_tipo_transaccion
            INNER JOIN EstadoTransaccion et
                ON i.id_estado = et.id_estado
            LEFT JOIN deleted d
                ON i.id_transaccion = d.id_transaccion
            LEFT JOIN EstadoTransaccion etd
                ON d.id_estado = etd.id_estado
            WHERE et.nombre_estado = 'COMPLETADA'
            AND tt.afecta_saldo = 1
            AND (
                d.id_transaccion IS NULL
                OR ISNULL(etd.nombre_estado, '') <> 'COMPLETADA'
            )
        )
        UPDATE s
        SET
            s.monto_disponible = s.monto_disponible + tc.variacion,
            s.monto_total = s.monto_total + tc.variacion,
            s.ultima_actualizacion = GETDATE()
        FROM Saldo s
        INNER JOIN TransaccionesCompletadas tc
            ON s.id_usuario = tc.id_usuario
            AND s.id_moneda = tc.id_moneda;

        ;WITH TransaccionesCompletadas AS (
            SELECT
                i.id_transaccion,
                i.id_usuario,
                i.id_moneda,
                i.monto_netto,
                tt.naturaleza,
                CASE
                    WHEN tt.naturaleza = 'INGRESO' THEN i.monto_netto

```

```

        WHEN tt.naturaleza = 'EGRESO' THEN i.monto_neto * -1
        ELSE 0
    END AS variacion
FROM inserted i
INNER JOIN TipoTransaccion tt
    ON i.id_tipo_transaccion = tt.id_tipo_transaccion
INNER JOIN EstadoTransaccion et
    ON i.id_estado = et.id_estado
LEFT JOIN deleted d
    ON i.id_transaccion = d.id_transaccion
LEFT JOIN EstadoTransaccion etd
    ON d.id_estado = etd.id_estado
WHERE et.nombre_estado = 'COMPLETADA'
    AND tt.afecta_saldo = 1
    AND (
        d.id_transaccion IS NULL
        OR ISNULL(etd.nombre_estado, '') <> 'COMPLETADA'
    )
)
INSERT INTO HistorialSaldo (
    saldo_anterior,
    saldo_posterior,
    fecha_registro,
    id_saldo,
    id_transaccion
)
SELECT
    s.monto_total - tc.variacion AS saldo_anterior,
    s.monto_total AS saldo_posterior,
    GETDATE() AS fecha_registro,
    s.id_saldo,
    tc.id_transaccion
FROM TransaccionesCompletadas tc
INNER JOIN Saldo s
    ON tc.id_usuario = s.id_usuario
    AND tc.id_moneda = s.id_moneda;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
END;
GO

```

Trigger 15: Validar comprobante de transacción completada

Valida que un comprobante solo pueda asociarse a una transacción financiera en estado COMPLETADA.

Importancia:

Evita generar comprobantes sobre transacciones pendientes o fallidas, garantizando mayor confiabilidad documental.

```

CREATE OR ALTER TRIGGER dbo.trg_Comprobante_ValidarTransaccionCompletada
ON dbo.Comprobante

```

```

AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        IF EXISTS (
            SELECT 1
            FROM inserted i
            INNER JOIN TransaccionFinanciera tf
                ON i.id_transaccion = tf.id_transaccion
            INNER JOIN EstadoTransaccion et
                ON tf.id_estado = et.id_estado
            WHERE et.nombre_estado <> 'COMPLETADA'
        )
        BEGIN
            THROW 50015, 'Solo se pueden emitir comprobantes para
transacciones completadas.', 1;
        END;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH;
END;
GO

```

4.4.1.4. Índices

En esta sección se presentan los índices diseñados para optimizar el rendimiento de las consultas relacionales de BetCore DB. Los índices permiten mejorar la búsqueda, filtrado, agrupamiento y ordenamiento de información en tablas críticas como Apuesta, TransaccionFinanciera, Saldo, HistorialSaldo, Cuota, Comprobante y TrabajadorxLocal.

Su implementación se relaciona directamente con la necesidad del negocio de consultar información operativa y financiera en escenarios donde se requiere analizar apuestas por estado y fecha, revisar transacciones pendientes, consultar saldos por usuario, monitorear cuotas activas o generar reportes históricos. Sin embargo, los índices deben aplicarse de forma controlada, ya que un uso excesivo puede afectar operaciones de inserción, actualización y eliminación de datos.

Relación con los casos prácticos de índices

Caso	¿Aplica al proyecto?	¿Por qué se relaciona?	Tablas principales
Caso 1: Estadísticas desactualizadas	Sí	BetCore DB registra apuestas, transacciones, saldos, historial y comprobantes. Si se insertan muchos datos, SQL Server puede estimar mal las filas y afectar el rendimiento de las consultas.	Apuesta, TransaccionFinanciera, HistorialSaldo, Comprobante
Caso 2: Estadísticas de columnas múltiples	Sí, es el principal	Las consultas del proyecto filtran por combinaciones como estado y fecha, usuario y fecha, mercado y estado, o saldo por usuario y moneda.	Apuesta, TransaccionFinanciera, Cuota, Saldo
Caso 3: Datos ascendentes	Sí, es muy importante	Se utilizan fechas que crecen constantemente, como fecha_apuesta, fecha_proceso,	Apuesta, TransaccionFinanciera, Comprobante, HistorialSaldo

		fecha_emision y fecha_registro.	
Caso 4: Parameter Sniffing	Aplica de forma secundaria	Los procedimientos almacenados usan parámetros como @id_usuario, @fecha_inicio, @fecha_fin, @monto_minimo o @id_local.	Procedimientos almacenados
Caso 5: Variables de tabla	No aplica directamente	En las consultas principales no se trabaja con variables de tabla masivas tipo @Tabla.	No prioritario
Caso 1: Estadísticas desactualizadas	Sí	BetCore DB registra apuestas, transacciones, saldos, historial y comprobantes. Si se insertan muchos datos, SQL Server puede estimar mal las filas y afectar el rendimiento de las consultas.	Apuesta, TransaccionFinanciera, HistorialSaldo, Comprobante

Índice 1: Apuestas por estado y fecha

Crea un índice no agrupado sobre las columnas `id_estado_apuesta` y `fecha_apuesta` de la tabla `Apuesta`.

Importancia:

Permite optimizar consultas que buscan apuestas por estado y rango de fechas, por ejemplo, apuestas pendientes, ganadas, perdidas o anuladas en un periodo determinado.

```
CREATE NONCLUSTERED INDEX IX_Apuesta_Estado_Fecha
ON dbo.Apuesta(id_estado_apuesta, fecha_apuesta);
GO
```

Índice 2: Apuestas por fecha

Crea un índice sobre la columna `fecha_apuesta` de la tabla `Apuesta`.

Importancia:

Mejora el rendimiento de reportes históricos o consultas por periodos, como apuestas registradas por día, semana o mes.

```
CREATE NONCLUSTERED INDEX IX_Apuesta_Fecha
ON dbo.Apuesta(fecha_apuesta);
GO
```

Índice 3: Apuestas por usuario y fecha

Crea un índice compuesto sobre `id_usuario` y `fecha_apuesta`.

Importancia:

Optimiza consultas de historial de apuestas por cliente, permitiendo revisar rápidamente las operaciones realizadas por un usuario en un rango de tiempo.

```
CREATE NONCLUSTERED INDEX IX_Apuesta_Usuario_Fecha
ON dbo.Apuesta(id_usuario, fecha_apuesta);
GO
```

Índice 4: Apuestas por monto apostado

Crea un índice sobre la columna `monto_apostado` de la tabla `Apuesta`.

Importancia:

Facilita la identificación de apuestas con montos altos, operaciones fuera de parámetros habituales o análisis de riesgo financiero.

```
CREATE NONCLUSTERED INDEX IX_Apuesta_Monto
ON dbo.Apuesta(monto_apostado);
GO
```

Índice 5: Apuestas por local

Crea un índice sobre la columna id_local de la tabla Apuesta.

Importancia:

Permite analizar el rendimiento de cada local, identificando qué establecimientos concentran mayor cantidad de apuestas o mayor monto apostado.

```
CREATE NONCLUSTERED INDEX IX_Apuesta_Local
ON dbo.Apuesta(id_local);
GO
```

Índice 6: Detalle de apuesta por cuota

Crea un índice sobre la columna id_cuota de la tabla DetalleApuesta.

Importancia:

Optimiza consultas que relacionan apuestas con cuotas seleccionadas, permitiendo analizar qué opciones de apuesta fueron utilizadas con mayor frecuencia.

```
CREATE NONCLUSTERED INDEX IX_DetalleApuesta_Cuota
ON dbo.DetalleApuesta(id_cuota);
GO
```

Índice 7: Mercados de apuesta por evento

Crea un índice sobre la columna id_evento de la tabla MercadoApuesta.

Importancia:

Mejora las consultas que buscan mercados disponibles para un evento deportivo específico, como ganador del partido, total de goles o marcador exacto.

```
CREATE NONCLUSTERED INDEX IX_MercadoApuesta_Evento
ON dbo.MercadoApuesta(id_evento);
GO
```

Índice 8: Cuotas por mercado, estado y valor

Crea un índice compuesto sobre id_mercado, estado y valor en la tabla Cuota.

Importancia:

Optimiza la búsqueda de cuotas activas dentro de un mercado de apuesta y permite ordenarlas por valor, lo cual es útil para mostrar opciones disponibles al usuario.

```
CREATE NONCLUSTERED INDEX IX_Cuota_Mercado_Estado_Valor
ON dbo.Cuota(id_mercado, estado, valor);
GO
```

Índice 9: Transacciones por estado, monto y fecha de solicitud

Crea un índice compuesto sobre id_estado, monto_total y fecha_solicitud en la tabla TransaccionFinanciera.

Importancia:

Permite identificar transacciones pendientes o sensibles por monto, facilitando el control financiero y la priorización de operaciones por revisar.

```
CREATE NONCLUSTERED INDEX IX_Transaccion_Estado_Monto_FechaSolicitud
ON dbo.TransaccionFinanciera(id_estado, monto_total, fecha_solicitud);
GO
```

Índice 10: Transacciones por estado y fecha de proceso

Crea un índice compuesto sobre id_estado y fecha_proceso.

Importancia:

Mejora reportes financieros que analizan transacciones completadas, fallidas o pendientes dentro de un periodo determinado.

```
CREATE NONCLUSTERED INDEX IX_Transaccion_Estado_FechaProceso
ON dbo.TransaccionFinanciera(id_estado, fecha_proceso);
GO
```

Índice 11: Transacciones por tipo y estado

Crea un índice compuesto sobre id_tipo_transaccion y id_estado.

Importancia:

Optimiza consultas que agrupan movimientos financieros por tipo, como depósitos, retiros, apuestas, premios o devoluciones, y por estado de procesamiento.

```
CREATE NONCLUSTERED INDEX IX_Transaccion_Tipo_Estado
ON dbo.TransaccionFinanciera(id_tipo_transaccion, id_estado);
GO
```

Índice 12: Saldos por usuario y moneda

Crea un índice compuesto sobre id_usuario y id_moneda en la tabla Saldo.

Importancia:

Permite consultar rápidamente el saldo disponible, retenido y total de un usuario según la moneda utilizada.

```
CREATE NONCLUSTERED INDEX IX_Saldo_Usuario_Moneda
ON dbo.Saldo(id_usuario, id_moneda);
GO
```

Índice 13: Historial de saldo por saldo y fecha

Crea un índice compuesto sobre id_saldo y fecha_registro en la tabla HistorialSaldo.

Importancia:

Optimiza consultas de auditoría financiera, permitiendo revisar la evolución del saldo de un usuario en orden cronológico.

```
CREATE NONCLUSTERED INDEX IX_HistorialSaldo_Saldo_Fecha
ON dbo.HistorialSaldo(id_saldo, fecha_registro);
GO
```

Índice 14: Comprobantes por fecha de emisión

Crea un índice sobre la columna fecha_emision de la tabla Comprobante.

Importancia:

Permite recuperar comprobantes emitidos en determinados periodos, facilitando auditorías, validaciones y control documental.

```
CREATE NONCLUSTERED INDEX IX_Comprobante_Fecha
ON dbo.Comprobante(fecha_emision);
GO
```

Índice 15: Trabajadores por local y estado

Crea un índice compuesto sobre id_local y estado en la tabla TrabajadorxLocal.

Importancia:

Mejora las consultas que identifican trabajadores activos asignados a cada local, apoyando el control operativo del personal.

```
CREATE NONCLUSTERED INDEX IX_TrabajadorxLocal_Local_Estado
ON dbo.TrabajadorxLocal(id_local, estado);
GO
```

4.4.1.5. Estadísticas

En esta sección se presentan las estadísticas creadas para optimizar el rendimiento de las consultas en la base de datos relacional de BetCore DB. Las estadísticas permiten que SQL Server estime de manera más precisa la cantidad de registros que serán procesados en una consulta, especialmente cuando existen filtros por estado, fecha, usuario, monto, moneda o tipo de transacción.

Su uso es importante porque ayuda al optimizador de consultas a elegir mejores planes de ejecución, reduciendo lecturas innecesarias y mejorando el tiempo de respuesta en reportes operativos, financieros y de seguimiento de apuestas.

Estadística 1: Estadística de apuestas por estado y fecha

Crea una estadística sobre las columnas `id_estado_apuesta` y `fecha_apuesta` de la tabla `Apuesta`.

Importancia:

Permite optimizar consultas que filtran apuestas por estado y fecha, como apuestas pendientes, ganadas, perdidas o anuladas dentro de un periodo determinado.

```
CREATE STATISTICS ST_Apuesta_Estado_Fecha
ON dbo.Apuesta(id_estado_apuesta, fecha_apuesta);
GO

EXEC sp_helpstats 'dbo.Apuesta', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Apuesta', 'ST_Apuesta_Estado_Fecha');
GO
```

Estadística 2: Estadística de apuestas por usuario y fecha

Crea una estadística sobre las columnas `id_usuario` y `fecha_apuesta` de la tabla `Apuesta`.

Importancia:

Ayuda a mejorar consultas relacionadas con el historial de apuestas de un usuario en un rango de fechas.

```
CREATE STATISTICS ST_Apuesta_Usuario_Fecha
ON dbo.Apuesta(id_usuario, fecha_apuesta);
GO

EXEC sp_helpstats 'dbo.Apuesta', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Apuesta', 'ST_Apuesta_Usuario_Fecha');
GO
```

GO

Estadística 3: Estadística de apuestas por monto apostado

Crea una estadística sobre la columna monto_apostado de la tabla Apuesta.

Importancia:

Permite mejorar consultas que buscan apuestas de montos altos, operaciones superiores al promedio o apuestas que requieren mayor control financiero.

```
CREATE STATISTICS ST_Apuesta_Monto
ON dbo.Apuesta(monto_apostado);
GO

EXEC sp_helpstats 'dbo.Apuesta', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Apuesta', 'ST_Apuesta_Monto');
GO
```

Estadística 4: Estadística de transacciones por estado, monto y fecha de solicitud

Crea una estadística sobre las columnas id_estado, monto_total y fecha_solicitud de la tabla TransaccionFinanciera.

Importancia:

Optimiza consultas que identifican transacciones pendientes, operaciones de monto alto o movimientos financieros que requieren revisión.

```
CREATE STATISTICS ST_Transaccion_Estado_Monto_FechaSolicitud
ON dbo.TransaccionFinanciera(id_estado, monto_total, fecha_solicitud);
GO

EXEC sp_helpstats 'dbo.TransaccionFinanciera', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.TransaccionFinanciera',
'ST_Transaccion_Estado_Monto_FechaSolicitud');
GO
```

Estadística 5: Estadística de transacciones por estado y fecha de proceso

Crea una estadística sobre las columnas id_estado y fecha_proceso de la tabla TransaccionFinanciera.

Importancia:

Mejora consultas de reportes financieros que analizan transacciones completadas, pendientes o fallidas en un periodo específico.

```
CREATE STATISTICS ST_Transaccion_Estado_FechaProceso
ON dbo.TransaccionFinanciera(id_estado, fecha_proceso);
GO

EXEC sp_helpstats 'dbo.TransaccionFinanciera', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.TransaccionFinanciera',
'ST_Transaccion_Estado_FechaProceso');
GO
```

Estadística 6: Estadística de transacciones por tipo y estado

Crea una estadística sobre las columnas id_tipo_transaccion y id_estado de la tabla TransaccionFinanciera.

Importancia:

Permite optimizar consultas que agrupan o filtran movimientos financieros por tipo de transacción y estado, como depósitos, retiros, apuestas, premios o devoluciones.

```
CREATE STATISTICS ST_Transaccion_Tipo_Estado
ON dbo.TransaccionFinanciera(id_tipo_transaccion, id_estado);
GO

EXEC sp_helpstats 'dbo.TransaccionFinanciera', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.TransaccionFinanciera',
'ST_Transaccion_Tipo_Estado');
GO
```

Estadística 7: Estadística de cuotas por mercado y estado

Crea una estadística sobre las columnas id_mercado y estado de la tabla Cuota.

Importancia:

Ayuda a optimizar consultas que buscan cuotas activas o cerradas dentro de un mercado de apuesta específico.

```

CREATE STATISTICS ST_Cuota_Mercado_Estado
ON dbo.Cuota(id_mercado, estado);
GO

EXEC sp_helpstats 'dbo.Cuota', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Cuota', 'ST_Cuota_Mercado_Estado');
GO

```

Estadística 8: Estadística de saldos por usuario y moneda

Crea una estadística sobre las columnas id_usuario y id_moneda de la tabla Saldo.

Importancia:

Permite mejorar consultas que verifican el saldo disponible, retenido o total de un usuario según la moneda utilizada.

```

CREATE STATISTICS ST_Saldo_Usuario_Moneda
ON dbo.Saldo(id_usuario, id_moneda);
GO

EXEC sp_helpstats 'dbo.Saldo', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Saldo', 'ST_Saldo_Usuario_Moneda');
GO

```

Estadística 9: Estadística de historial de saldo por saldo y fecha

Crea una estadística sobre las columnas id_saldo y fecha_registro de la tabla HistorialSaldo.

Importancia:

Optimiza consultas de auditoría que revisan cambios en los saldos de los usuarios a través del tiempo.

```

CREATE STATISTICS ST_HistorialSaldo_Saldo_Fecha
ON dbo.HistorialSaldo(id_saldo, fecha_registro);
GO

EXEC sp_helpstats 'dbo.HistorialSaldo', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.HistorialSaldo',
'ST_HistorialSaldo_Saldo_Fecha');
GO

```

Estadística 10: Estadística de comprobantes por fecha de emisión

Crea una estadística sobre la columna fecha_emision de la tabla Comprobante.

Importancia:

Mejora consultas relacionadas con comprobantes emitidos en periodos específicos, facilitando auditorías y validaciones documentales.

```
CREATE STATISTICS ST_Comprobante_Fecha
ON dbo.Comprobante(fecha_emision);
GO

EXEC sp_helpstats 'dbo.Comprobante', 'ALL';
GO

DBCC SHOW_STATISTICS ('dbo.Comprobante', 'ST_Comprobante_Fecha');
GO
```

4.4.2. Consultas para la base de datos no relacional

En esta sección se presentan las consultas desarrolladas para la base de datos no relacional de BetCore DB. Las consultas fueron elaboradas en VS code con la extension de MongoDB y permiten obtener información relevante para la gestión operativa, financiera y comercial de una casa de apuestas deportivas.

Cada consulta aporta valor a la empresa porque permite analizar apuestas pendientes, movimientos financieros, montos apostados, desempeño por local, usuarios con mayor actividad y operaciones económicas registradas. Estas consultas utilizan las colecciones apuestas y movimientos_financieros.

```
use("betcore_db");
```

Consulta 1: Listar apuestas pendientes

Esta consulta muestra las apuestas que actualmente se encuentran con estado PENDIENTE, incluyendo el código de ticket, usuario, monto apostado y fecha de registro.

Importancia:

Permite identificar las apuestas que aún no han sido resueltas. Esto es útil para el seguimiento operativo, ya que ayuda a controlar qué apuestas todavía no han sido ganadas, perdidas, anuladas o canceladas.

```
db.apuestas
.find(
  {
    estado_apuesta: "PENDIENTE",
  },
  {
    _id: 0,
    id_apuesta: 1,
    codigo_ticket: 1,
    "usuario.nombre_completo": 1,
    monto_apostado: 1,
    estado_apuesta: 1,
    fecha_apuesta: 1,
  },
)
.sort({
  fecha_apuesta: -1,
});
```

Consulta 2: Buscar apuestas con monto mayor o igual a S/ 100

Esta consulta obtiene las apuestas cuyo monto apostado es mayor o igual a S/ 100 y que se encuentran registradas en moneda nacional.

Importancia:

Permite identificar apuestas de mayor valor económico. Es importante para el área financiera y de control de riesgos, ya que las apuestas con montos altos pueden requerir mayor seguimiento.

```
db.apuestas
.find(
  {
    monto_apostado: {
      $gte: 100,
    },
    "moneda.codigo": "PEN",
  },
  {
    _id: 0,
    id_apuesta: 1,
    codigo_ticket: 1,
```

```

        "usuario.nombre_completo": 1,
        monto_apostado: 1,
        "moneda.codigo": 1,
        estado_apuesta: 1,
    },
)
.sort({
    monto_apostado: -1,
});

```

Consulta 3: Consultar el historial de apuestas de un usuario

Esta consulta permite buscar las apuestas registradas por un usuario específico mediante su número de documento.

Importancia:

Es útil para atención al cliente, auditoría interna y seguimiento del comportamiento de los apostadores, ya que permite revisar el historial de apuestas de un usuario determinado.

```

db.apuestas
.find(
  {
    "usuario.numero_documento": "12345678",
  },
  {
    _id: 0,
    id_apuesta: 1,
    codigo_ticket: 1,
    "usuario.nombre_completo": 1,
    selecciones: 1,
    monto_apostado: 1,
    monto_ganancia: 1,
    estado_apuesta: 1,
    fecha_apuesta: 1,
  },
)
.sort({
  fecha_apuesta: -1,
});

```

Consulta 4: Listar selecciones pendientes del deporte fútbol

Esta consulta descompone el arreglo selecciones para mostrar únicamente las selecciones relacionadas con fútbol que aún tienen resultado pendiente.

Importancia:

Ayuda a conocer qué eventos futbolísticos todavía no han sido resueltos. Esto es importante porque el fútbol suele ser uno de los deportes con mayor volumen de apuestas dentro del negocio.

```
db.apuestas.aggregate([
  {
    $unwind: "$selecciones",
  },
  {
    $match: {
      "selecciones.deporte": "Fútbol",
      "selecciones.resultado": "PENDIENTE",
    },
  },
  {
    $project: {
      _id: 0,
      id_apuesta: 1,
      codigo_ticket: 1,
      usuario: "$usuario.nombre_completo",
      evento: "$selecciones.evento",
      mercado: "$selecciones.mercado",
      opcion: "$selecciones.opcion",
      cuota: "$selecciones.cuota",
      monto_apostado: 1,
      estado_apuesta: 1,
    },
  },
]);
```

Consulta 5: Calcular el total apostado por local

Esta consulta agrupa las apuestas por local y calcula el monto total apostado y la cantidad de apuestas registradas en cada punto de atención.

Importancia:

Permite comparar el rendimiento económico de los locales. Es útil para tomar decisiones comerciales, administrativas y operativas sobre los puntos de atención con mayor actividad.

```
db.apuestas.aggregate([
  {
    $group: {
      _id: "$local.nombre_local",
    },
  },
]);
```

```

        total_apostado: {
            $sum: "$monto_apostado",
        },
        cantidad_apuestas: {
            $sum: 1,
        },
    },
},
{
    $project: {
        _id: 0,
        local: "$_id",
        total_apostado: 1,
        cantidad_apuestas: 1,
    },
},
{
    $sort: {
        total_apostado: -1,
    },
},
]);

```

Consulta 6: Identificar los usuarios con mayor monto apostado

Esta consulta agrupa las apuestas por usuario y calcula el monto total apostado por cada uno, mostrando los cinco usuarios con mayor actividad económica.

Importancia:

Permite identificar a los clientes más activos o de mayor valor para la empresa. Esta información puede utilizarse para estrategias de fidelización, promociones o análisis comercial.

```

db.apuestas.aggregate([
{
    $group: {
        _id: {
            id_usuario: "$usuario.id_usuario",
            nombre_completo: "$usuario.nombre_completo",
        },
        total_apostado: {
            $sum: "$monto_apostado",
        },
        cantidad_apuestas: {
            $sum: 1,
        },
    },
}
]);

```

```

    },
  },
  {
    $project: {
      _id: 0,
      id_usuario: "$_id.id_usuario",
      nombre_completo: "$_id.nombre_completo",
      total_apostado: 1,
      cantidad_apuestas: 1,
    },
  },
  {
    $sort: {
      total_apostado: -1,
    },
  },
  {
    $limit: 5,
  },
]);

```

Consulta 7: Listar movimientos financieros pendientes o rechazados

Esta consulta muestra los movimientos financieros cuyo estado sea PENDIENTE o RECHAZADA, incluyendo tipo de operación, monto, método de pago y fecha.

Importancia:

Permite identificar operaciones económicas que requieren revisión. Es importante para el control financiero, especialmente en casos de retiros, depósitos o movimientos que no fueron completados correctamente.

```

db.movimientos_financieros
.find(
  {
    estado: {
      $in: ["PENDIENTE", "RECHAZADA"],
    },
  },
  {
    _id: 0,
    id_transaccion: 1,
    usuario_id: 1,
    apuesta_id: 1,
    tipo: 1,
    estado: 1,
  }
)

```

```

        monto: 1,
        metodo_pago: 1,
        fecha_operacion: 1,
    },
)
.sort({
    fecha_operacion: -1,
});

```

Consulta 8: Calcular el monto total por tipo de movimiento financiero

Esta consulta agrupa los movimientos financieros según su tipo y calcula el monto total y la cantidad de movimientos registrados por cada categoría.

Importancia:

Permite conocer el comportamiento financiero del negocio, diferenciando operaciones como apuestas, premios, depósitos, retiros, comisiones, reembolsos y ajustes.

```

db.movimientos_financieros.aggregate([
{
    $group: {
        _id: "$tipo",
        total_monto: {
            $sum: "$monto",
        },
        cantidad_movimientos: {
            $sum: 1,
        },
    },
},
{
    $project: {
        _id: 0,
        tipo_movimiento: "$_id",
        total_monto: 1,
        cantidad_movimientos: 1,
    },
},
{
    $sort: {
        total_monto: -1,
    },
},
]);

```

Consulta 9: Consultar movimientos financieros de una apuesta específica

Esta consulta busca los movimientos financieros asociados a una apuesta específica mediante el campo `apuesta_id`.

Importancia:

Es útil para auditoría, atención de reclamos y validación de operaciones económicas relacionadas con una apuesta determinada, como pagos, premios o reembolsos.

```
db.movimientos_financieros
.find(
  {
    apuesta_id: 1001,
  },
  {
    _id: 0,
    id_transaccion: 1,
    apuesta_id: 1,
    usuario_id: 1,
    tipo: 1,
    estado: 1,
    monto: 1,
    saldo_anterior: 1,
    saldo_posterior: 1,
    fecha_operacion: 1,
  },
)
.sort({
  fecha_operacion: -1,
});
```

Consulta 10: Relacionar movimientos financieros con la información de la apuesta

Esta consulta utiliza `$lookup` para relacionar los documentos de `movimientos_financieros` con la colección `apuestas`, mostrando información financiera junto con los datos principales del ticket.

Importancia:

Permite visualizar en una sola consulta el movimiento económico y la apuesta relacionada. Es útil para auditoría, control interno y revisión de operaciones financieras asociadas a tickets específicos.

```

db.movimientos_financieros.aggregate([
  {
    $lookup: {
      from: "apuestas",
      localField: "apuesta_id",
      foreignField: "id_apuesta",
      as: "detalle_apuesta",
    },
  },
  {
    $unwind: "$detalle_apuesta",
  },
  {
    $project: {
      _id: 0,
      id_transaccion: 1,
      tipo: 1,
      estado: 1,
      monto: 1,
      fecha_operacion: 1,
      apuesta_id: 1,
      codigo_ticket: "$detalle_apuesta.codigo_ticket",
      usuario: "$detalle_apuesta.usuario.nombre_completo",
      monto_apostado: "$detalle_apuesta.monto_apostado",
      estado_apuesta: "$detalle_apuesta.estado_apuesta",
    },
  },
  {
    $sort: {
      fecha_operacion: -1,
    },
  },
]);

```

CONCLUSIONES

Conclusiones y recomendaciones

Conclusiones

El desarrollo de BetCore DB permitió diseñar una solución de base de datos orientada a la gestión y monitoreo de información para una casa de apuestas deportivas. A lo largo del trabajo se abordó tanto el enfoque relacional como el no relacional, lo que permitió analizar cómo organizar la información del negocio desde dos perspectivas complementarias.

En primer lugar, el modelo relacional desarrollado en SQL Server permitió estructurar la información de manera organizada mediante tablas, claves primarias, claves foráneas y relaciones entre entidades. Este enfoque resulta adecuado para controlar datos operativos y transaccionales que requieren consistencia, integridad y trazabilidad, como usuarios, apuestas, eventos deportivos, transacciones, locales y monedas. De esta manera, el diseño relacional contribuye a reducir la duplicidad de datos y a mantener reglas claras entre las entidades principales del negocio.

En segundo lugar, la implementación de consultas, procedimientos almacenados, funciones, triggers, índices y estadísticas permitió fortalecer el uso de la base de datos relacional. Estos elementos no solo facilitaron la consulta de información, sino que también aportaron al control de operaciones, automatización de procesos, validación de datos y mejora del rendimiento. Por ello, SQL Server representa una alternativa sólida para administrar la información estructurada y crítica de BetCore DB.

Por otro lado, el diseño no relacional permitió representar la información mediante un modelo documental en MongoDB. Este enfoque fue adecuado para organizar documentos relacionados con apuestas y movimientos financieros, utilizando estructuras flexibles, documentos embebidos, arreglos y referencias lógicas. La colección apuestas permitió agrupar información operativa como usuario, local, selecciones, moneda, montos y estado; mientras que la colección movimientos_financieros permitió registrar operaciones económicas vinculadas al negocio.

Asimismo, el modelo no relacional demostró ser útil para consultas más flexibles y orientadas al análisis de documentos completos. A través de MongoDB se pudieron realizar búsquedas, agregaciones y relaciones mediante \$lookup, lo que permitió obtener información relevante sobre apuestas pendientes, montos apostados, usuarios con mayor actividad,

desempeño por local y movimientos financieros. Esto evidencia que el modelo documental puede complementar al modelo relacional cuando se requiere flexibilidad y rapidez en la consulta de información agrupada.

En conclusión, BetCore DB integra una propuesta coherente para una casa de apuestas deportivas, ya que combina la consistencia del modelo relacional con la flexibilidad del modelo no relacional. El enfoque relacional permite mantener control e integridad sobre los datos principales del negocio, mientras que el enfoque documental facilita representar operaciones agrupadas y consultar información de manera más directa. Esta combinación fortalece la capacidad de la empresa para gestionar apuestas, controlar movimientos financieros y apoyar la toma de decisiones.

Recomendaciones

Se recomienda mantener el modelo relacional en SQL Server para los procesos principales que requieren alta consistencia, control de integridad y relaciones formales entre entidades. Esto resulta especialmente importante en operaciones sensibles como el registro de usuarios, apuestas, transacciones, saldos, pagos y auditoría financiera.

También se recomienda utilizar el modelo no relacional en MongoDB como apoyo para escenarios donde se necesite consultar información agrupada de forma rápida y flexible. Por ejemplo, puede emplearse para visualizar el detalle completo de una apuesta, analizar selecciones deportivas, revisar movimientos financieros o generar reportes operativos sin depender de múltiples uniones entre tablas.

Asimismo, se recomienda actualizar periódicamente los índices y estadísticas de la base de datos relacional, ya que estos elementos ayudan a mejorar el rendimiento de las consultas. En una casa de apuestas deportivas, donde puede existir un alto volumen de

operaciones, es importante que las búsquedas por usuario, evento, apuesta, fecha, estado y transacción se realicen de manera eficiente.

En el caso de MongoDB, se recomienda crear índices sobre campos utilizados frecuentemente en las consultas, como estado_apuesta, usuario.id_usuario, usuario.numero_documento, fecha_apuesta, apuesta_id, usuario_id, tipo y estado. Esto permitiría mejorar el rendimiento de las consultas sobre apuestas y movimientos financieros.

Se recomienda reforzar la seguridad de la información, considerando que una casa de apuestas maneja datos personales, financieros y operativos. Para ello, es importante establecer roles de usuario, permisos de acceso, políticas de respaldo, auditoría de operaciones y mecanismos de protección de datos sensibles.

Finalmente, se recomienda que BetCore DB sea considerado como una solución escalable. A futuro, el sistema podría incorporar nuevas colecciones o tablas relacionadas con promociones, límites de apuesta, historial de cuotas, validaciones antifraude, reportes regulatorios y comportamiento de usuarios. Esto permitiría que la base de datos evolucione de acuerdo con el crecimiento de la empresa y las necesidades del negocio.

Video About-the-Team

Link del video: <https://youtu.be/qCM1BG-g3SI>

Duración: 00:00 – 10 minutos y 16 segundos

Integrante	Minuto y segundo
Macalupu Marchan, Yeissen Beckam	04:14 - 05:13
Chavez Pino, Paula Brenda	05:14 - 06:28
Huerta Cardenas, Brayan Benjamin	06:29 - 07:32
Rivera Rupay, Fabricio Jose	07:33 - 08:55
Gallegos De La Cruz, Giovanni Marcelo	08:56 -10:16

Video About The Team 1

Evidencia de las reuniones en grupo por medio de discord

Reunión 2

Esta reunion se desarrollo en discord, donde se hablo de las estructura adecuada para la creacion de base de datos en MongoDB.



Nota. Captura tomada de la reunión grupal para el proyecto.

Exposición del Equipo

Link del video: <https://youtu.be/51x3B481Ygs>

Duración: 00:00 – 13 minutos y 32 segundos

Exposición en Equipo 1

Evidencia de la exposición ya subida en YouTube

Principales consultas

```
// Consulta 1: Listar apuestas pendientes
db.apuestas
.find(
  {
    estado_apuesta: "PENDIENTE",
  },
  {
    _id: 0,
    id_apuesta: 1,
    codigo_ticket: 1,
    "usuario.nombre_completo": 1,
    monto_apostado: 1,
    estado_apuesta: 1,
    fecha_apuesta: 1,
  },
).sort({
  fecha_apuesta: -1,
});
```

Esta consulta muestra las apuestas que actualmente se encuentran con estado PENDIENTE, incluyendo el código de ticket, usuario, monto apostado y fecha de registro.

Resultado

```
{
  id_apuesta: NumberInt("1819"),
  codigo_ticket: "APU2024070019",
  usuario: {
    nombre_completo: "Usuario",
    monto_apostado: NumberInt("100"),
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: ISODate(
      "2024-07-19T18:30:00.000Z"
    )
  },
}

{
  id_apuesta: NumberInt("1818"),
  codigo_ticket: "APU2024070018",
  usuario: {
    nombre_completo: "Usuario",
    monto_apostado: NumberInt("55"),
    estado_apuesta: "PENDIENTE",
    fecha_apuesta: ISODate(
      "2024-07-19T17:30:00.000Z"
    )
  },
}
```



Finalidad

Permite identificar las apuestas que aún no han sido resueltas. Esto es útil para el seguimiento operativo, ya que ayuda a controlar qué apuestas todavía no han sido ganadas, perdidas, anuladas o canceladas.

Nota. Captura tomada del video de exposicion.

Bibliografía

- Ministerio de Comercio Exterior y Turismo. (2025, 17 de diciembre). *Mincetur: recaudación por juegos y apuestas a distancia supera los S/ 419 millones en 2025* [Nota de prensa]. gob.pe. <https://www.gob.pe/institucion/mincetur/noticias/1314117-mincetur-recaudacion-por-juegos-y-apuestas-a-distancia-supera-los-s-419-millones-en-2025>
- MySQL AB. (s.f.). Manual de referencia de MySQL 5.0. Documentation & Help. <https://documentation.help/MySQL-5.0-es/>
- TutorialMySQL.org. (s.f.). InnoDB. <https://tutorialmysql.org/motores-de-almacenamiento/innodb>
- PostgreSQL Global Development Group. (s.f.). Main Page/es. PostgreSQL Wiki. Recuperado el 10 de mayo de 2026, de https://wiki.postgresql.org/wiki/Main_Page/es
- Raja, Y. (2021, 19 de agosto). Administración de usuarios y roles de PostgreSQL. Blog de Amazon Web Services (AWS). <https://aws.amazon.com/es/blogs/aws-spanish/managing-postgresql-users-and-roles/>
- Microsoft. (s.f.). ¿Qué es SQL Server? Microsoft Learn. <https://learn.microsoft.com/es-es/sql/sql-server/what-is-sql-server?view=sql-server-ver17>

Anexos

Anexo A: Evidencia de entrevistas realizadas**Segmento 1: Propietarios, Asistentes u Encargados de casas de apuestas**

Entrevista 1	
Entrevistado	Gamaniel Cruz Valerio
Edad	27 años
Sexo	Masculino
Ocupación	Encargado de local de apuestas ("Te Apuesto" / La Tinka)
Distrito	San Juan de Lurigancho, Lima
Duración	18 minutos 35 segundos
Enlace al video	https://youtu.be/FOZ8yaOpJDI

Figura A1

Evidencia de entrevista realizada al encargado de casa de apuestas



Nota. Captura tomada del video de entrevista elaborado por el equipo.

Entrevistado 2	
Entrevistado	Manuel Sanchez
Edad	- años
Sexo	Masculino
Ocupación	Administrador de local de apuestas
Distrito	Surquillo, Lima
Duración	3 minutos 28 segundos
Enlace al video	https://youtu.be/sZN-aLleVYY?si=95O2da4x-jWfa_FX

Figura A2

Evidencia de entrevista realizada al administrador de casa de apuestas



Nota. Captura tomada del video de entrevista elaborado por el equipo.

Segmento 2: Clientes o personas que realizan apuestas deportivas

Entrevistado 1	
Entrevistado	Byron Tirado
Edad	- años
Sexo	Masculino
Ocupación	Administrador de negocio familiar
Distrito	San Juan de Lurigancho, Lima
Duración	4 minutos 26 segundos
Enlace al video	https://youtu.be/ohdtMp_Q5e4?si=Nxul8DlfHF42eebd

Figura A3

Evidencia de entrevista realizada a persona que realiza apuestas



Nota. Captura tomada del video de entrevista elaborado por el equipo.

Entrevistado 2	
Entrevistado	Jefferson Pérez Pino
Edad	- años
Sexo	Masculino
Ocupación	Docente de Inglés
Distrito	El Tambo, Huancayo

Duración	4 minutos 13 segundos
Enlace al video	https://youtu.be/X34vMN5ummw

Figura A4

Evidencia de entrevista realizada a persona que realiza apuestas



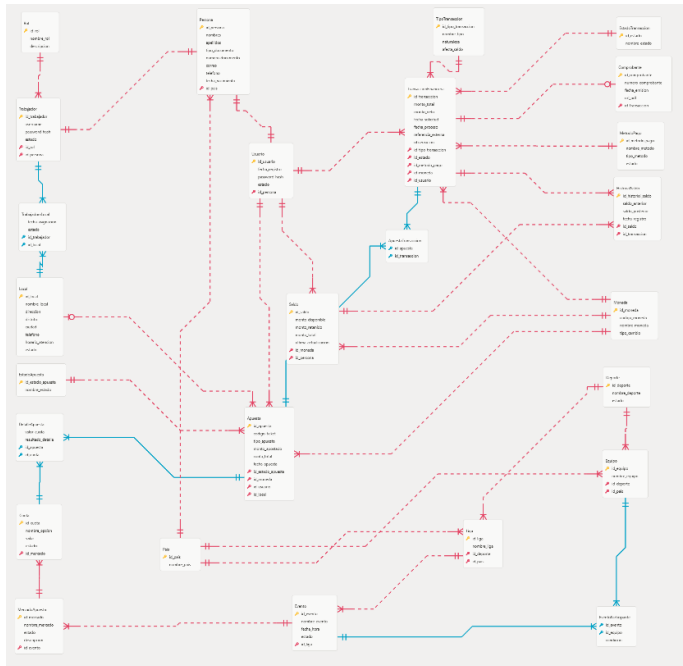
Nota. Captura tomada del video de entrevista elaborado por el equipo.

Entrevistado 3	
Entrevistado	Luis Fernando Ramírez Torres
Edad	28 años
Sexo	Masculino
Ocupación	Técnico administrativo
Distrito	Los Olivos, Lima
Duración	7 minutos 54 segundos
Enlace al video	https://youtu.be/lDQ-6H2F2Go

Anexo C: Diagrama entidad-relación lógico

Figura C1

Diagrama entidad-relación lógico de BetCore DB.

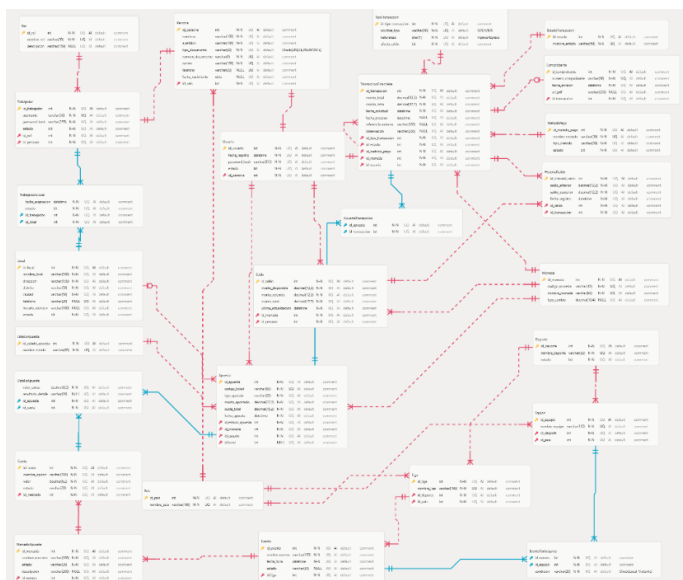


Nota. Elaborado por el equipo como parte del diseño lógico de la base de datos para la gestión de información en casas de apuestas deportivas.

Anexo D: Diagrama entidad-relación físico

Figura D1

Diagrama entidad-relación físico implementado para BetCore DB.

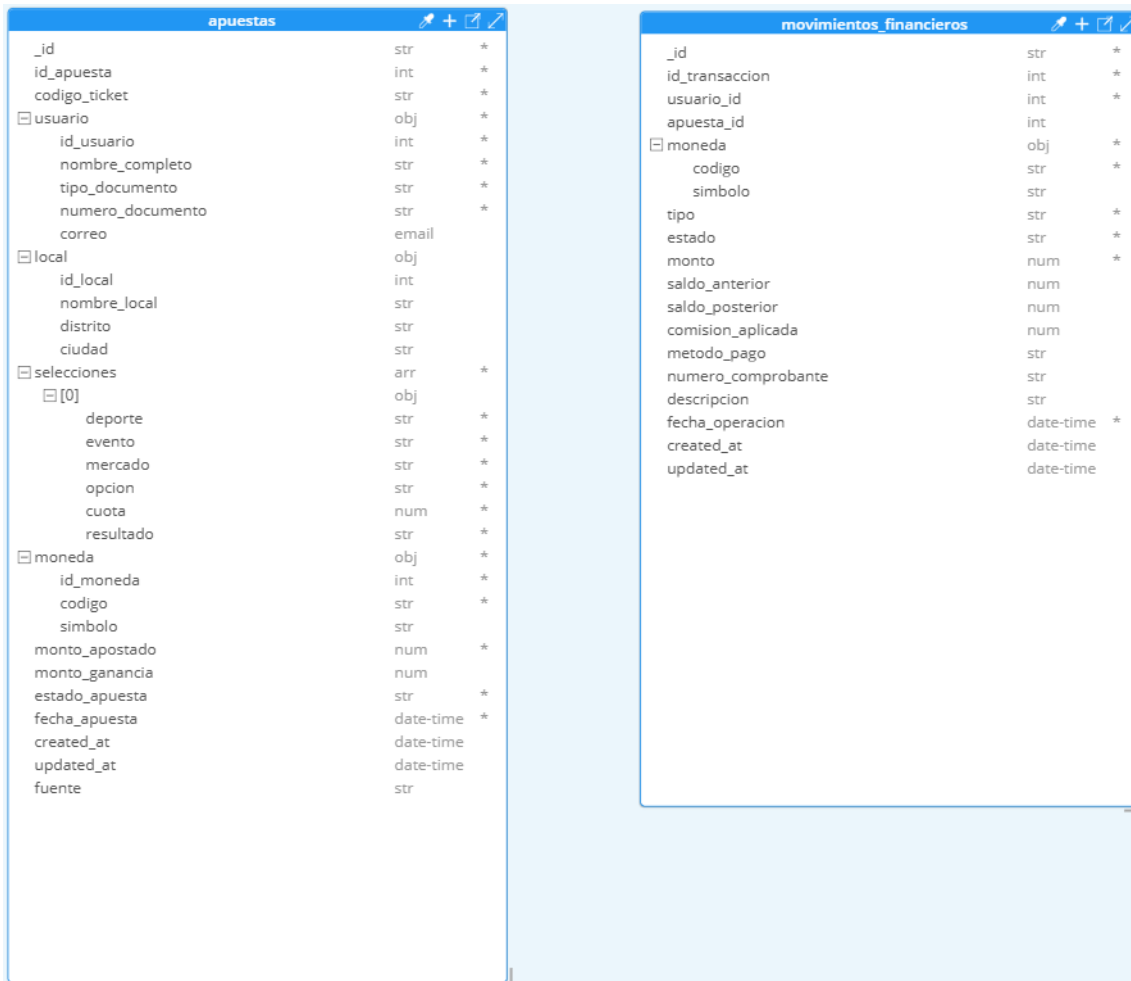


Nota. El diagrama representa la estructura física de la base de datos relacional diseñada para en Visual Studio Code .ERD.

Anexo E: Diagrama entidad-relación físico

Figura E1

Diagrama de MongoDB en Hackolade



Nota. El diagrama representa la estructura de la base de datos no relacional diseñada en Hackolade.

Anexo F: Enlaces de video de exposición

Video correspondiente al TB1

Video de exposición	
Título	upc-pre-202610-1asi0787-6008-controlbet-expo-tp1
Duración	14 minutos 35 segundos
Enlace al video	https://www.youtube.com/watch?v=wLq9tri9Hjs

Video correspondiente al TF1

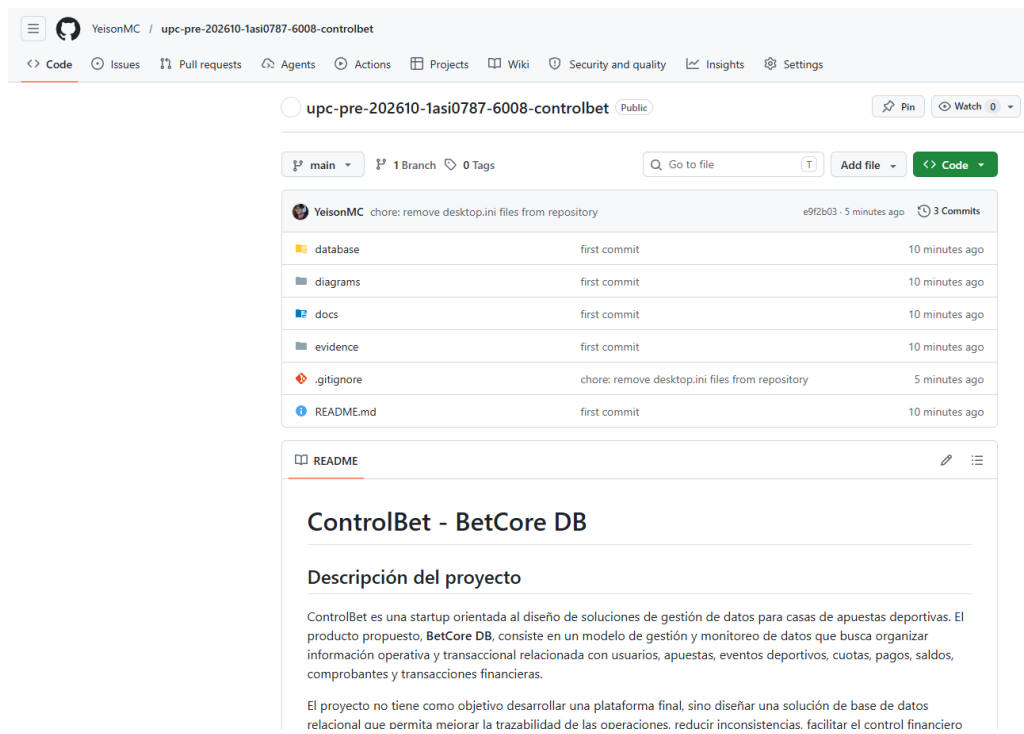
Video de exposición	
Titulo	upc-pre-202610-1asi0787-6008-controlbet-expo-tf1
Duración	13 minutos 32 segundos
Enlace al video	https://youtu.be/51x3B481Ygs

Anexo G: Enlaces a evidencias y artefactos del proyecto

Repositorio de GitHub: <https://github.com/YeisonMC/upc-pre-202610-1asi0787-6008-controlbet>

Figura G1

Repositorio de GitHub del proyecto ControlBet.



Nota. La figura muestra la estructura del repositorio utilizado para organizar los documentos, diagramas, y evidencias del proyecto BetCore DB.